1st Stability Workshop

Proceedings

15 December 2025

Preface

The Stability Workshop was created as a venue for discussing research aligned with one of the original goals of blockchains and cryptocurrencies: stability.

Although blockchains and cryptocurrencies were invented as a reaction to the global financial crisis of 2008 and promised to provide a more stable foundation for global monetary systems, this promise has not been fulfilled yet and recent developments (e.g. DegenFi, Memecoins, Scam ICOs, ...) have even taken the field in a direction opposite of what was originally intended.

This workshop is for all those who are bravely pursuing the original goals of the blockchains and cryptocurrencies and pursuing a more stable world.

In this first edition, we have received 14 submissions, of which 10 were accepted for publication at the workshop, after a thorough peer review process. On average, each paper was reviewed by 4 program committee members, with no paper paper receiving less than 3 reviews and some papers receiving even as much as 6 reviews. This proceedings includes all accepted submitted papers. All 10 accepted papers will still undergo a second independent round of reviews after the workshop, before final publication at the Journal of Financial Technology.

Dr. Bruno Woltzenlogel Paleo

15th of December 2025

Strategic Complementarity, Liquidity Tools, and Stale Pricing in Stablecoins

Vaibhav Keshav (MIT)

Abstract

I propose liquidity fees and buffers as a policy to stabilize stablecoins. Liquidity fees apply solely in the primary market and serve dual purposes: deterring panic withdrawals and replenishing reserves. Liquidity ratios, by contrast, directly limit redemption capacity and affect solvency perceptions. In a surprising result, an anticipatory behavior by arbitrageurs compresses arbitrage margins and lowers secondary market prices even before policies take effect, triggering ex-ante redemptions in the secondary market. Finally, stale pricing weakens arbitrage discipline, amplifying fragility. These results underscore the need to coordinate fee design and liquidity management to ensure stablecoin ecosystem resilience.

JEL Classification Code: D14, G11, G23

Keywords: Stablecoins, Strategic Complementarity, DeFi, Liquidity, Fee.

1 Introduction

In recent years, fiat-backed stablecoins have emerged as a critical component of decentralized financial infrastructure, offering on-chain settlement and dollar-denominated liquidity to a wide range of decentralized and institutional users (Ma et al. (2023)). Despite their promise of price stability, these instruments are structurally fragile: they issue short-term claims redeemable at par while holding portfolios that often include illiquid or volatile assets. This liquidity mismatch, coupled with fixed $1 redemption guarantees, creates the conditions for the run dynamics that are amplified by coordination failures and arbitrage frictions. When investors lose confidence in the underlying backing or redemption process, even temporarily, large-scale redemptions can be triggered, overwhelming reserve buffers and forcing the issuer to liquidate assets at fire-sale prices. The collapse of *TerraUSD* (Liu et al. (2023)) and instability episodes involving major fiat-backed stablecoins (see, Anadu et al. (2023); Uhlig (2022), among others) have underscored the urgency of designing credible and incentive-compatible mechanisms that preempt runs without sacrificing price stability (U.S. Congress (2025)). This paper addresses key policy questions: What tools can stabilize fiat-backed stablecoins, where should they be applied in the market structure, and how do they affect strategic behavior, price volatility, and ecosystem-wide fragility?

To answer these research questions, I develop a dynamic model of stablecoin fragility. The model features three distinct economic agents—investors, arbitrageurs, and the stablecoin issuer—interacting over time under uncertainty about asset fundamentals. Investors are heterogeneous in their information and face private and public signals about the issuer's solvency (Liu et al. (2023)). They redeem strategically based on a dynamic coordination

problem, weighing the benefit of continued participation against the risk of being left behind in a run. Arbitrageurs engage in primary-market redemption and secondary-market trading but are subject to inventory costs and balance sheet frictions, which constrain their ability to stabilize prices. The issuer dynamically chooses a liquidity fee charged only in the primary market at redemption, the number of arbitrageurs it contracts with, the amount of stablecoins minted, and the size of the liquidity buffer. These instruments jointly determine the system's fragility and price stability. The model nests a global games structure within a dynamic contracting framework, allowing for tractable equilibrium characterization and explicit welfare analysis.

The analysis delivers several novel insights about the design and interaction of stablecoin stabilization tools. First, I show that a primary-market liquidity fee and an ex-ante liquidity buffer are strategic complements in mitigating run risk: increasing either reduces the marginal effectiveness of the other, as both operate through the investor's continuation value and shift the redemption threshold. This interaction gives rise to non-linear policy tradeoffs, whereby aggressive deployment of one tool can partially offset the need for the other. Second, I find that arbitrage capacity plays a dual role. On one hand, deeper arbitrage enhances price stability by narrowing the deviation between the secondary market price and the primary market redemption value. On the other hand, it accelerates the propagation of bad news into redemption behavior, thereby amplifying run incentives through faster price adjustment. Third, I endogenize the issuer's selection of all key policy levers and solve for closed-form optimal policies in a two-period environment. The comparative statics reveal that optimal fees increase with redemption pressure and arbitrage inefficiency; optimal arbitrage capacity balances stabilization benefits against

balance sheet costs; and minting is optimally constrained when liquidity is close to the regulatory threshold. Finally, I show that naively combining tools, such as layering liquidity buffers on top of high redemption fees, can generate incentives when strategic cross-effects are not internalized. These findings offer practical implications for stablecoin design and regulation, especially under prospective stress conditions where fragility must be managed in real time.

Although most studies of stablecoin fragility focus on contemporaneous price deviations or realized redemptions, I show that the anticipation of regulatory intervention, particularly the imposition of liquidity fees or binding liquidity ratios, can itself reshape market behavior in advance. In my model, arbitrageurs internalize expected future changes in redemption economics, adjusting their pricing thresholds and inventory allocations before policies take effect. This forward-looking behavior influences secondary market prices, which in turn affect investor incentives to redeem preemptively. Thus, even when liquidity fees apply *solely* in the primary market, their announcement compresses arbitrage margins and shifts the investor's expected payoff profile. Anticipatory runs may then occur not because fundamentals have deteriorated, but because the arbitrage boundary is mechanically altered by future fee implementation. I further show that issuers must endogenously respond to these expectations: unless the liquidity buffer is sufficiently strong to absorb front-loaded withdrawals, or unless the policy transition is credibly smoothed, the pre-announcement of a stabilizing tool can paradoxically induce ex-ante instability. These insights underscore the importance of strategic timing and credibility in the design of stablecoin redemption mechanisms and regulatory frameworks.

In the last part of the model, I integrate stale pricing into the analysis of stablecoin dynamics. In conventional settings, secondary market prices reflect up-to-date asset valuations through continuous arbitrage. However, when arbitrageurs are balance-sheet constrained or face redemption frictions, such as inventory costs, delayed settlements, or impending liquidity fees, the pass-through from primary market conditions to secondary market prices becomes imperfect. I show that such frictions can induce *stale pricing* behavior, where secondary market prices fail to fully adjust to changes in the issuer's fundamentals. This misalignment dampens the price signal observed by investors and distorts their redemption incentives. In my framework, stale prices widen the gap between perceived and actual redemption value, potentially masking deterioration in asset quality or, conversely, delaying recovery signals during stabilization. The endogenous emergence of stale pricing, driven by constrained arbitrage, generates a new amplification channel: by obscuring shifts in fundamentals, it can either delay runs or trigger abrupt repricing once constraints ease. These dynamics have critical implications for the credibility of price-based stabilization mechanisms in stablecoin ecosystems.

To put these results in context, a (payment) stablecoin "refers to a digital asset that is or is designed to be used as a means of payment or settlement and whose issuer is obligated to convert, redeem, or repurchase it for a fixed amount of monetary value, and represents or creates a reasonable expectation that it will maintain a stable value relative to such monetary value. This definition excludes any national currency and any security issued by an investment company registered under section 8(a) of the Investment Company Act of 1940 (U.S. Congress (2025))."[1] Permitted payment stablecoin issuers are required to

[1]Although while writing this paper, the bill on the Guiding and Establishing National Innovation for US Stablecoins of 2025 (GENIUS Act of 2025) is still pending, I use it to

maintain reserves backing their outstanding payment stablecoins on at least a one-to-one basis. These reserves must consist of high-quality, liquid assets, including: (i) United States coins and currency (such as Federal Reserve notes); (ii) funds held as demand deposits or other withdrawable deposits at insured depository institutions or regulated foreign depository institutions, subject to prudential limitations established by relevant supervisory authorities to ensure institutional safety and soundness; (iii) Treasury bills, notes, or bonds with either a remaining maturity of 93 days or less, or originally issued with such a maturity; (iv) repurchase agreements maturing in 7 days or less that are backed by Treasury bills maturing in 90 days or less; and (v) reverse repurchase agreements maturing in 7 days or less, collateralized by Treasury securities on an overnight basis and subject to overcollateralization consistent with prevailing market practices, provided such agreements are either tri-party, centrally cleared, or bilateral with adequately creditworthy counterparties even under conditions of severe market stress, (vi) money market funds, invested solely in underlying assets, among others.

Reserves backing permitted payment stablecoins may not be pledged, rehypothecated, or reused, except to the extent necessary to generate liquidity for meeting reasonable expectations of redemption requests. Specifically, reserves held in the form of Treasury bills may be used as collateral in repurchase agreements with a maturity of 90 days or less, provided that either: (A) such repurchase agreements are cleared through a central clearing counterparty that has been approved by the issuer's primary Federal or State payment stablecoin regulator; or (B) the issuer has obtained prior approval from its primary Federal or State regulator to engage in such transactions.

---

explain the structure of the stablecoin ecosystem.

The primary Federal payment stablecoin regulators—or, in the case of a State-qualified issuer, the relevant State regulator—shall issue: (i) capital requirements for permitted payment stablecoin issuers that are no more than necessary to ensure the issuer's continued operations; (ii) liquidity and interest rate risk management standards that are sufficient to preserve the financial integrity of the issuer and its ability to meet all financial obligations, including redemption demands; and (iii) operational, compliance, and information technology risk management standards—such as those addressing Bank Secrecy Act and sanctions compliance—appropriately tailored to the issuer's business model and risk profile, and aligned with applicable legal authorities.

This paper contributes to many streams of literature. First, this paper contributes the foundational literature on financial intermediary runs and liquidity management (Allen and Gale (1998); Bernardo and Welch (2004); Diamond and Dybvig (1983); Goldstein and Pauzner (2005)) and subsequent models of panic-based and information-driven runs, (see, Chari and Jagannathan (1988); Jacklin and Bhattacharya (1988)) into the context of decentralized digital assets.[2] Like traditional banks, stablecoins face liquidity mismatches and coordination risk, but without formal deposit contracts or government insurance.[3] Furthermore, in an influential work Morris and Shin (1998, 2003, 2004) demonstrate that introducing even slight uncertainty into agents' information about fundamentals ensures a unique equilibrium. This approach transforms the

[2]Owing to the extensive and continually evolving nature of the literature in this area, a comprehensive review is beyond the scope of this paper. For a detailed treatment of the relevant theoretical foundations, readers are referred to Brunnermeier and Oehmke (2013) for a detailed survey.

[3]In a related study, Bryant (1980) shows how fractional reserves and deposit insurance can mitigate the social costs of illiquidity under incomplete information—an insight echoed in my model of stablecoins, where issuers lack formal insurance and must instead rely on internal mechanisms like liquidity fees and reserve buffers.

notion of self-fulfilling runs from a multiplicity problem into a probabilistic, policy-relevant structure. Abreu and Brunnermeier (2003) further highlight how even rational arbitrageurs may fail to prevent bubbles or crashes due to synchronization failures—agents wait for others to act, thereby delaying correction. Finally, recent work by Goldstein and Pauzner (2005) endogenize run equilibria using private signals about fundamentals. They show that liquidity provision and run incentives must be balanced through contract design. These insights apply directly to stablecoins, where arbitrage capacity is often constrained, coordination is fragile, and peg maintenance depends on both fundamental backing and the strategic behavior of investors and arbitrageurs. By integrating these threads, my paper embeds stablecoin fragility within a rigorously modeled environment of forward-looking behavior, incomplete information, and market segmentation, thereby unifying classic run theory with modern global games and arbitrage frictions.

Second, this paper contributes to the literature on money market funds (MMFs) by leveraging the institutional similarity between the primary market of stablecoins and the constant net asset value (NAV) structure of (a subset of) MMFs in the US, European Union, and the United Kingdom. During the Global Financial Crisis (GFC), MMFs suffered large-scale redemptions as investors fled to safety, highlighting the structural fragility of fixed NAV instruments (Kacperczyk and Schnabl (2010, 2013)). Subsequent studies emphasize the central role of information acquisition and redemption incentives in amplifying run dynamics (Gallagher et al. (2020); Schmidt et al. (2016)). In response to the GFC, the SEC implemented a range of regulations designed to limit *ex-post* run amplification (SEC (2014)).[4] One of the most

[4]See Hanson et al. (2015) for evaluations of potential policy tools.

consequential provisions of the SEC's post-crisis reforms was the requirement that prime institutional MMFs adopt a floating NAV regime. This structural change prompted a substantial reallocation of assets: approximately $1 trillion flowed out of prime institutional MMFs into government MMFs (Cipriani and La Spada (2021)). In parallel, many investors redirected capital to MMFs domiciled in Europe, which continued to offer constant NAV structures and were thus perceived as more stable substitutes (Fricke et al. (2024)).[5]

However, even after these reforms, prime MMFs experienced renewed run pressures that were exacerbated by liquidity gate provisions in March 2020 (Li et al. (2021)) and in March 2022 (Keshav and Saha (2023)), revealing the limitations of *ex-post* regulatory mechanisms. As a result of this run, the SEC implemented provisions of liquidity fee and liquidity ratios (SEC (2023)).[6] Keshav (2024) develops a dynamic model quantifying the impact of these tools. Furthermore, Keshav (2025) examines the behavior of MMFs in the prime and dollar during the US-China trade shock of 2025, highlighting how liquidity fees and regulatory liquidity thresholds shaped portfolio reallocations without triggering widespread redemptions, thus emphasizing a selective flight to quality within the dollar ecosystem.[7]

[5] Parlatore (2016) studies the effect of sponsor support in the US MMF industry.

[6] The reforms also eliminated the use of redemption gates, which previously allowed funds to temporarily suspend redemptions during periods of market stress.

[7] Related scholarship examines the transmission of European sovereign and banking shocks to MMFs. Chernenko and Sunderam (2014) studies how the European banking crisis affected US prime MMFs, identifying key funding frictions arising from cross-border exposures. Keshav and Winters (2025a) documents that MMFs with constant net asset value domiciled in Europe experienced substantial flows in response to the United Kingdom's September 2022 fiscal announcement, the so-called 'Growth Plan', highlighting the sensitivity of NAV-stable funds to sovereign credibility. More recently, Keshav and Winters (2025b) exploited the German political crisis of 2024 as a natural experiment to show that sovereign political risk can be transmitted to the private sector through reallocations of the MMF portfolio. The study finds that, despite stable investor flows, fund managers rebalanced away from euro-denominated private instruments toward sovereign securities, thereby contracting the credit supply to private borrowers. Together, these studies reveal that while post-2008 regulatory reforms have bolstered investor-side resilience, MMFs continue to serve

My contribution builds on this foundation by modeling how only sophisticated investors, those who internalize arbitrage frictions and redemption constraints, choose to run, while others do not. This selective response generates equilibrium segmentation and highlights the conditions under which ex-post fees and buffers stabilize redemptions versus when they inadvertently exacerbate instability.[8]

Third, this paper contributes to a growing literature to model the risks and potential regulatory framework to contain those risks (a non-exhaustive list would include, Gorton et al. (2023); Gorton and Zhang (2021); Hu et al. (2019); Li and Mann (2018); Liu and Tsyvinski (2021), among others).[9] Abadi and Brunnermeier (2018) introduce the "blockchain trilemma," framing limits on decentralization, efficiency, and transferability. D'Avernas et al. (2022) and Gorton et al. (2022) model stablecoin pegs under collateral and seigniorage tradeoffs, showing their vulnerability to demand shocks and the role of equity holders. Cong et al. (2021) emphasize endogenous adoption driven by transactional demand, while Sockin and Xiong (2023a,b) explore how token design and user-platform conflicts shape fragility. Ma et al. (2023) highlight that concentrated arbitrage authorization helps manage peg enforcement but increases systemic risk. Li and Mayer (2021) and Gryglewicz et al. (2021) build dy-

as a transmission channel for sovereign and financial stress through adjustments on the asset side.

[8] Stablecoin prices may deviate from their nominal peg of $1 when secondary market imbalances are not fully offset by arbitrage activity. Specifically, the market price falls below $1 when selling pressure exceeds the absorption capacity of authorized arbitrageurs. Conversely, it rises above $1 when buying pressure outpaces their ability or willingness to supply liquidity. In this regard, fluctuations in stablecoin prices reflect a structural feature analogous to that observed in exchange-traded funds (ETFs), whereby share prices trade at a premium or discount to their NAVs due to frictions in primary and secondary market arbitrage. ETFs, in contrast to MMFs, are less susceptible to runs (see Allen and Gale (2004); Farhi et al. (2009); Jacklin (1987); Koont et al. (2022), among others).

[9] See, Harvey et al. (2021); John et al. (2022); Makarov and Schoar (2022) for survey.

namic models showing persistent instability and optimal financing structures in token ecosystems.

My paper is the closest to Ma et al. (2023) with notable variations. Although I build on their rich theoretical framework, my model introduces a broader policy design space, allowing the issuer to choose two stabilization tools: liquidity fees and liquidity buffers. This richer policy environment permits a detailed analysis of how these tools interact, both ex post and ex ante, to shape redemption incentives and secondary market pricing. In particular, I show how anticipated changes in redemption frictions can induce endogenous run behavior even before policies take effect, a channel that arises from the forward-looking behavior of arbitrageurs and investors. Furthermore, by incorporating stale pricing and misperceptions of arbitrage, the model provides new theoretical foundations for persistent price deviations and wedge amplification in pegged asset markets. In doing so, it offers a unified framework for understanding both price volatility and run risk under dynamic, policy-responsive market conditions.

Finally, this paper also relates to the larger body of research on cryptocurrencies and decentralized finance. Given the extensive and rapidly evolving nature of this literature, a comprehensive treatment is beyond the scope of this study. However, several notable studies, for example Harvey et al. (2021), John et al. (2022), and Makarov and Schoar (2021) offer systematic surveys and overviews of key developments in the field.

2 Mathematical Model

We consider a discrete-time economy indexed by $t = 0, 1, \ldots, T$, in which a fiat-backed financial intermediary, such as a stablecoin issuer or a constant NAV MMF, faces fragility arising from liquidity transformation, investor coordination risk, and arbitrage constraints. The intermediary issues tokens or fund shares that are redeemable at par in a designated primary market. These instruments also trade in a secondary market at a floating price. The system evolves under uncertainty about the fundamental value of the reserve asset θ_t and faces dynamic pressures from investor redemptions and arbitrage flows.

There are four types of agents in the economy: informed investors, uninformed investors, arbitrageurs, and the issuer. *Informed investors* represent a continuum of mass $w_1 \in (0, 1)$. These investors receive both a private signal x_i and a public signal z_t about the fundamental θ_t, and they form posterior beliefs accordingly (Schmidt et al. (2016)). Informed investors make dynamic decisions about when to redeem based on their beliefs about the fundamentals and expectations about aggregate redemption behavior (Liu et al. (2023)). If they redeem, they incur an idiosyncratic cost c_i (Keshav (2024)). In cases of liquidity shortfall, redemptions may also trigger penalties associated with breaching the liquidity ratio. *Uninformed investors* comprise the residual mass $1 - w_1$. These agents do not observe private signals and follow mechanical or delayed redemption heuristics, e.g., passively reacting to secondary market conditions or mimicking informed withdrawals.

Arbitrageurs, indexed by $n \in \mathbb{N}$, participate in both the secondary and primary markets. They are able to purchase or sell tokens in the floating-price secondary market and simultaneously access the primary market, where

redemptions and creations are conducted at a fixed nominal value of \$1, provided the issuer remains solvent. Arbitrageurs are subject to quadratic inventory costs, capturing balance sheet constraints, and must clear positions daily. Their activity connects the secondary market price to the fundamental value through redemption and creation arbitrage.

The issuer (or fund manager) manages a portfolio comprising both liquid and illiquid assets. The issuer chooses the liquidity fee ϕ_t applied only in the primary market at redemption, and controls the liquidity buffer L_t. To satisfy redemptions, the issuer draws on liquid reserves, and if necessary, liquidates illiquid assets at a discount, incurring a cost. Regulatory or internal policy constraints impose a minimum liquidity ratio $L_{\min}$, below which the issuer may face additional costs or reputational damage.

State Variables and Market Structure

At each date t, the system is characterized by a set of state variables. The economic fundamental $\theta_t \in \mathbb{R}$ captures the solvency of the issuer or the quality of the reserve asset portfolio. The liquidity buffer $L_t \in [0, 1]$ denotes the ratio of liquid assets to total liabilities, while the variable $A_t \in [0, 1]$ represents the fraction of investors redeeming at time t. Redemption decisions may reflect strategic considerations, private information, or exogenous liquidity shocks.

In the primary market, the issuer imposes a liquidity fee $\phi_t \in \mathbb{R}_+$, which is applied only upon redemption. This fee serves to internalize the liquidity cost imposed by redeeming investors (SEC (2023)). Tokens or shares also trade in a secondary market at price p_t, determined endogenously through arbitrage

and investor demand under frictions.

Fundamentals and Information Structure

The fundamental evolves according to a stationary AR(1) process:

$$\theta_t = \rho\theta_{t-1} + \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, \sigma_\theta^2), \tag{1}$$

where $\rho \in (0, 1)$ governs the persistence of solvency conditions. Investors do not observe θ_t directly. Instead, each informed investor i receives a private signal:

$$x_{i,t} = \theta_t + \epsilon_{i,t}, \quad \epsilon_{i,t} \sim \mathcal{N}(0, \sigma_x^2), \tag{2}$$

while all investors observe a public signal:

$$z_t = \theta_t + \eta_t, \quad \eta_t \sim \mathcal{N}(0, \sigma_z^2). \tag{3}$$

Informed investors update their beliefs based on Bayes' rule and assess the expected payoff from redeeming versus staying invested. Their actions depend on expectations about future fundamentals and the behavior of other investors. Uninformed investors lack private signals and may respond to public trends or redeem with delay, contributing to coordination failures in periods of stress.

Liquidity Dynamics

The issuer's liquidity buffer evolves as:

$$L_{t+1} = L_t + R_t - \lambda A_t, \tag{4}$$

where R_t is the return on the reserve portfolio, and $\lambda > 0$ measures the liquidity drain associated with redemptions. This specification captures the

essential tradeoff of liquidity transformation: asset returns replenish the buffer, while redemptions deplete it.

If the liquidity ratio falls below a binding regulatory threshold $L_{\min}$, the issuer incurs additional frictions, including regulatory penalties, forced liquidation of illiquid holdings, or administrative redemption controls. These constraints introduce nonlinearities into the issuer's optimization and influence investors' beliefs about run risk. The resulting dynamics shape both price stability and systemic fragility.

2.1 Informed Investors' Problem

At each date t, an informed investor chooses whether to redeem immediately or remain invested. Immediate redemption yields a deterministic payoff net of a liquidity fee ϕ_t, an idiosyncratic cost c_i, and a liquidity penalty $\gamma \cdot \max\{0, L_{\min} - L_t\}$ if the fund's liquidity buffer falls below a regulatory threshold $L_{\min}$ (Keshav (2024)). Staying invested entitles the investor to a per-period benefit η—interpreted as liquidity yield, convenience, or expected return—conditional on the fund's survival. Let $\beta \in (0, 1)$ denote the investor's discount factor.

Let $V_{i,t}$ denote the investor's value function. If the investor redeems, utility is:

$$U_t^{\text{redeem}} = 1 - c_i - \gamma \cdot \max\{0, L_{\min} - L_t\}.$$

If the investor stays, they receive η in each period $s > t$ with survival probability $P_{i,t}^{\text{surv}}(s)$. Thus, expected continuation value is:

$$U_t^{\text{stay}} = \sum_{s=t+1}^{T} \beta^{s-t} \cdot P_{i,t}^{\text{surv}}(s) \cdot \eta.$$

The investor chooses to redeem if and only if:

$$U_t^{\text{redeem}} \geq U_t^{\text{stay}}.$$

Investor beliefs over solvency θ_t are formed based on their private signal $x_{i,t} = \theta_t + \epsilon_{i,t}$ with $\epsilon_{i,t} \sim \mathcal{N}(0, \sigma_x^2)$ and a public signal $z_t = \theta_t + \eta_t$ with $\eta_t \sim \mathcal{N}(0, \sigma_z^2)$. Assuming a Gaussian prior $\theta_t \sim \mathcal{N}(\theta_0, \sigma_0^2)$, the posterior belief is normal with mean:

$$\mu_t = \omega_x x_{i,t} + \omega_z z_t,$$

where

$$\omega_x = \frac{\sigma_z^2}{\sigma_x^2 + \sigma_z^2}, \quad \omega_z = \frac{\sigma_x^2}{\sigma_x^2 + \sigma_z^2}.$$

Given these beliefs, there exists a threshold signal $x_t^*(z_t)$ such that the investor is indifferent between redeeming and staying. This threshold solves:

$$1 - c_i - \gamma \cdot \max\{0, L_{\min} - L_t\} = \sum_{s=t+1}^{T} \beta^{s-t} \cdot \Phi\left(\frac{\mu_t - A_t}{\sigma_\theta}\right) \cdot \eta,$$

where $\Phi(\cdot)$ denotes the standard normal CDF and reflects the belief that the aggregate redemption level A_t remains below the fundamental θ_t, ensuring survival. All investors with $x_{i,t} \leq x_t^*(z_t)$ redeem, generating endogenous runs when beliefs deteriorate or when liquidity thresholds bind.

The investor's dynamic decision thus captures coordination motives, forward-looking survival risk, and the endogenous impact of market-based redemption frictions, regulated liquidity buffers, and information heterogeneity.

2.1.1 Belief About Fund Survival

Informed investors contemplating redemption at time t must assess the likelihood that the fund remains solvent in the next period. Fund survival hinges on two interrelated risks: redemption-induced distress and insufficient *ex-ante* liquidity. Formally, the investor anticipates default or fund failure if either of the following conditions is met:

$$A_t > \theta_t \quad \text{(Coordination failure: excessive redemptions)} \tag{5}$$

$$L_t < L_{\min} \quad \text{(Liquidity breach)} \tag{6}$$

Here, A_t denotes aggregate redemptions by all investors (excluding the investor herself), θ_t represents the unobserved fundamental threshold beyond which the manager is unable to meet redemptions without incurring fire-sale losses, and L_t is the fund's realized liquidity ratio compared to a critical threshold, $L_{\min}$.

Since the fundamental θ_t is latent, investors rely on a noisy public signal μ_t, which aggregates relevant macroeconomic and fund-level information. Following standard Bayesian updating under normality, the investor's posterior over θ_t induces a perceived survival probability given by:

$$P_{\text{surv}}(\mu_t) = \mathbb{P}(A_t \leq \theta_t \mid \mu_t) = \Phi\left(\frac{\mu_t - A_t}{\sigma_\theta}\right) \tag{7}$$

where $\Phi(\cdot)$ is the standard normal cumulative distribution function (CDF), and σ_θ denotes the perceived volatility (inverse precision) of the fundamental threshold.

This posterior survival belief plays a pivotal role in the investor's decision-making. It enters directly into the expected utility from remaining invested:

$$U_t^{\text{stay}} = \beta \cdot P_{\text{surv}}(\mu_t) \cdot \eta = \beta \cdot \Phi\left(\frac{\mu_t - A_t}{\sigma_\theta}\right) \cdot \eta, \tag{8}$$

where $\beta \in (0, 1)$ is the investor's discount factor and η captures the continuation value conditional on fund survival, representing yield, convenience, or strategic complementarities from fund liquidity services. The investor redeems when the expected value of staying, as captured in Equation 8, falls below the value of redeeming immediately, subject to any applicable liquidity fee and redemption friction.

In summary, the endogenous nature of A_t, influenced by the beliefs and actions of other investors, creates a strategic complementarity in redemption behavior. As such, the survival belief in Equation 7 not only governs the investor's own action but also generates fragility through higher-order belief effects, akin to global games in the tradition of Morris and Shin (2003) and in MMF contexts by Schmidt et al. (2016).

2.2 Arbitrageur Problem

At each time t, arbitrageurs observe the secondary market price p_t and can transact in the primary market at a fixed price of \$1. Minting incurs no fee, while redemptions are subject to a liquidity fee ϕ_t. Arbitrageurs take positions to exploit deviations from the peg and are subject to quadratic inventory costs. This subsection characterizes their optimal behavior and the resulting price impact.

2.2.1 Setup and Payoff Structure

Let $z_{j,t}$ denote the net arbitrage position of arbitrageur j. A positive value $z_{j,t} > 0$ indicates buying tokens in the secondary market and redeeming them in the primary market at net value $1 - \phi_t$. A negative value $z_{j,t} < 0$ represents minting tokens at cost \$1 and selling them in the secondary market at p_t. Arbitrageurs face an inventory cost of $z_{j,t}^2/(2\chi)$, where $\chi > 0$ governs risk tolerance. The arbitrageur's payoff is:

$$\Pi_{j,t} = \begin{cases} (1 - \phi_t - p_t) \cdot z_{j,t} - \dfrac{z_{j,t}^2}{2\chi}, & z_{j,t} > 0 \quad \text{(redeem)} \\ (p_t - 1) \cdot |z_{j,t}| - \dfrac{z_{j,t}^2}{2\chi}, & z_{j,t} < 0 \quad \text{(mint and sell)} \end{cases} \tag{9}$$

2.2.2 Optimal Arbitrage Response

We derive the arbitrageur's optimal response by solving the first-order condition:

Case A: Secondary Market Discount ($p_t < 1 - \phi_t$) Arbitrageurs buy in the secondary market and redeem in the primary market (paying ϕ_t). The optimal arbitrage volume satisfies:

$$\frac{d\Pi_{j,t}}{dz_{j,t}} = (1 - \phi_t - p_t) - \frac{z_{j,t}}{\chi} = 0 \quad \Rightarrow \quad z_{j,t}^* = \chi(1 - \phi_t - p_t)$$

Case B: Secondary Market Premium ($p_t > 1$) Arbitrageurs mint at cost 1 (no fee) and sell in the secondary market. The optimal arbitrage volume satisfies:

$$\frac{d\Pi_{j,t}}{dz_{j,t}} = -(p_t - 1) - \frac{z_{j,t}}{\chi} = 0 \quad \Rightarrow \quad z_{j,t}^* = -\chi(p_t - 1)$$

Case C: No Arbitrage Region ($p_t \in [1-\phi_t, 1]$) Neither minting nor redeeming is profitable. Hence:

$$z^*_{j,t} = 0$$

2.2.3 Aggregate Arbitrage Supply

Summing across n symmetric arbitrageurs, total arbitrage supply Z_t becomes:

$$Z_t = \begin{cases} n\chi(1-\phi_t-p_t), & p_t < 1-\phi_t \\ -n\chi(p_t-1), & p_t > 1 \\ 0, & p_t \in [1-\phi_t, 1] \end{cases} \tag{10}$$

This creates a kinked supply function, with a dead zone of width ϕ_t below the peg where redemption arbitrage is unprofitable. Figures 1- 4 provide graphical representation of Z_t.

2.2.4 Price Formation and Market Clearing

Assume arbitrageurs absorb net investor order flow A_t. Then, market-clearing requires:

$$Z_t + \omega_t = A_t \tag{11}$$

where ω_t is exogenous order imbalance or noise demand. The equilibrium price p_t adjusts to satisfy Equation 11. Three regimes emerge as shown in Figure 5:

- If $A_t > n\chi\phi_t$: price must drop below $1-\phi_t$ to induce redemptive arbitrage.
- If $A_t < 0$: price must rise above 1 to trigger minting arbitrage.

- If $A_t \in [0, n\chi\phi_t]$: price floats within the no-arbitrage zone $[1-\phi_t, 1]$.

2.2.5 Dynamic Implications of Liquidity Fee

The liquidity fee ϕ_t governs arbitrage activation on the downside but not on the upside (Figure 3). This asymmetry implies:

- Higher ϕ_t widens the lower-bound dead zone, reducing arbitrage stabilization after negative shocks.
- Lower ϕ_t tightens price anchoring near the peg but increases vulnerability to liquidity runs by investors.

Hence, ϕ_t acts as a policy instrument balancing price stability and run deterrence.[10] Arbitrage only enforces the peg asymmetrically, and its strength is state-contingent.

2.3 Issuer's Optimization Problem

The issuer of the stablecoin operates under multiple constraints and objectives. At each time t, it selects three control variables: the liquidity fee $\phi_t \in [0, \bar{\phi}]$, the number of arbitrageurs admitted $n_t \in \mathbb{N}$, and the number of new tokens minted $m_t \geq 0$. These choices jointly determine the system's liquidity dynamics, investor behavior, and arbitrage responsiveness. The issuer aims to ensure short-term price stability, deter speculative runs, and sustain long-term solvency.

[10] This result is consistent with SEC (2023) who states (in relation withe the MMF),

> "*As a result, it (liquidity fee) may dampen any first-mover advantage, thus reducing the incentive to redeem early, the resulting fund outflows, and dilution resulting from these outflows.*"

The liquidity fee ϕ_t is charged upon redemption and directly discourages withdrawals while generating fee revenue. The arbitrage capacity n_t governs the depth and responsiveness of the secondary market, as arbitrageurs help anchor the stablecoin price to its fundamental value. The minting quantity m_t expands supply and provides revenue, but increases exposure to future redemptions. Internally, the issuer may also target a liquidity ratio L_t^{target} through its portfolio policy, though this is not a direct control.

The system evolves through a set of endogenous state variables: the actual liquidity ratio L_t, the fundamental backing θ_t, the belief (μ_t, σ_t^2) over this fundamental, a public signal z_t, and the total outstanding stablecoin supply S_t. These variables are updated recursively as market outcomes unfold and investor behavior materializes.

Let the issuer's value function be denoted $V_t(L_t, \mu_t, \sigma_t^2, z_t, S_t)$. The dynamic program is given by:

$$V_t(L_t, \mu_t, \sigma_t^2, z_t, S_t) = \max_{\phi_t, n_t, m_t} \Big\{ \underbrace{\phi_t \cdot A_t(\phi_t, \theta_t)}_{\text{redemption fee revenue}} + \underbrace{P_t^{\text{mint}} \cdot m_t}_{\text{minting revenue}} - \underbrace{C(n_t)}_{\text{cost of supporting arbitrageurs}} - \underbrace{\gamma \cdot \max\{0, L_{\min} - L_{t+1}\}}_{\text{liquidity violation penalty}} + \beta\, \mathbb{E}_t \left[V_{t+1}(L_{t+1}, \mu_{t+1}, \sigma_{t+1}^2, z_{t+1}, S_{t+1}) \right] \Big\} \tag{12}$$

subject to the system's transition dynamics.[11] First, the liquidity ratio evolves as:

$$L_{t+1} = \frac{\text{liquid reserves at } t + \text{cash inflows} - \text{redemptions}}{S_{t+1}} \tag{13}$$

[11] Kindly note that, in this context, $A_t(\phi_t, \theta_t)$ denotes the volume of redemptions executed by arbitrageurs, conditional on the prevailing liquidity fee ϕ_t and market signal θ_t.

where the next period's supply is:

$$S_{t+1} = S_t + m_t - A_t(\phi_t, n_t) \tag{14}$$

Redemptions $A_t(\phi_t, n_t)$ are determined by investor behavior, which responds to the perceived fragility of the system. Higher fees reduce withdrawal incentives, while stronger arbitrage capacity stabilizes price expectations and mitigates run dynamics. We denote this investor best-response function implicitly:

$$A_t = \mathcal{A}(\phi_t, n_t; \mu_t, \sigma_t^2, z_t) \tag{15}$$

The minting price P_t^{mint} reflects the expected per-unit reserve backing of newly issued stablecoins. It satisfies:

$$P_t^{\text{mint}} = \mathbb{E}_t[\text{per-token liquidation value of reserve assets}] \leq 1 \tag{16}$$

The arbitrage cost function $C(n_t)$ captures increasing resource needs or operational complexity associated with managing more arbitrage participants. At the same time, a larger n_t enhances secondary market depth and reduces price volatility:

$$\text{Price volatility at } t = f(n_t, \chi) \tag{17}$$

where χ parameterizes arbitrageurs' inventory risk tolerance. Price volatility, in turn, feeds back into investor beliefs via the convenience yield (Gorton and Pennacchi (1990)) and perceived liquidity of the system, amplifying or dampening redemption incentives.

The recursive problem thus integrates three critical channels of issuer influence: (i) liquidity fees that alter redemption behavior and raise defensive cap-

ital, (*ii*) arbitrage support that stabilizes secondary markets, and (*iii*) minting policy that raises revenue but dilutes liquidity coverage. Optimal policy trades off revenue generation against liquidity risk and investor fragility.

2.3.1 Closed-Form Characterization of the Issuer's Problem

To derive tractable insights into optimal policy design, we consider a simplified two-period environment, $t = 0, 1$, in which a stablecoin issuer selects the liquidity fee $\phi \in [0, \bar{\phi}]$, the number of arbitrageurs $n \in \mathbb{N}$, and the quantity of new stablecoins minted $m \geq 0$. Investor redemptions $A(\phi, n)$ are taken to be a deterministic and differentiable function, decreasing in ϕ and increasing in fragility, proxied here by inverse market depth n.

The issuer enters period 0 with an initial liquidity ratio L_0 and outstanding supply S. Minting increases total supply, while redemptions shrink it. At time 1, the liquidity ratio is given by:

$$L_1 = \frac{L_0 S + \phi A(\phi, n) - A(\phi, n)}{S + m - A(\phi, n)} \tag{18}$$

A penalty $\gamma(L_{\min} - L_1)$ is imposed whenever liquidity falls below the regulatory threshold $L_{\min}$. Revenues in period 0 include withdrawal fees and proceeds from minting, offset by a cost κn associated with supporting n arbitrageurs. The terminal value at $t = 1$ reflects the liquidity penalty. The issuer's total objective becomes:

$$V_0 = \phi A(\phi, n) + p_m m - \kappa n - \gamma \max\{0, L_{\min} - L_1\} \tag{19}$$

where $p_m \leq 1$ denotes the per-unit backing value of newly issued tokens.

To analyze optimal policy, we assume a linear redemption function:

$$A(\phi, n) = \alpha - \beta\phi + \delta n \tag{20}$$

capturing the demand elasticity to liquidity fees and arbitrage efficiency.

We first consider the optimal arbitrageur capacity n^*. Substituting $A(\phi, n)$ into the objective and differentiating yields:

$$\frac{\partial V_0}{\partial n} = \phi\delta - \kappa - \gamma \cdot \frac{\partial}{\partial n} \max\{0, L_{\min} - L_1\} \tag{21}$$

The interior solution satisfies $\partial V_0/\partial n = 0$ if the liquidity constraint binds; otherwise, if $L_1 \geq L_{\min}$, the penalty term drops out and the optimal n^* balances the marginal fee revenue gain from reduced redemptions against arbitrage support costs.

The optimal liquidity fee ϕ^* satisfies:

$$\frac{\partial V_0}{\partial \phi} = \alpha - 2\beta\phi + \delta n - \gamma \cdot \frac{\partial}{\partial \phi} \max\{0, L_{\min} - L_1\} \tag{22}$$

When $L_1 \geq L_{\min}$, the solution is explicit:

$$\phi^* = \frac{\alpha + \delta n}{2\beta} \tag{23}$$

This characterizes the optimal fee as a function of demand elasticity and market depth. However, if the resulting ϕ^* leads to a violation of the liquidity threshold, the issuer must raise ϕ until the penalty is avoided.

The optimal minting decision m^* follows similarly. The marginal value of

minting is:

$$\frac{\partial V_0}{\partial m} = p_m - \gamma \cdot \frac{\partial}{\partial m} \max\{0, L_{\min} - L_1\} \tag{24}$$

As minting increases the denominator in L_1, it mechanically lowers the liquidity ratio. Hence, if liquidity is near the threshold, minting is optimally constrained; otherwise, the issuer maximizes revenue by setting m as high as market acceptance and backing allow.

Taken together, the optimal policy rule balances three interdependent margins: the fee ϕ that generates revenue and deters redemptions; the arbitrage capacity n, which stabilizes the price at the cost κ; and the minting amount m, which boosts immediate proceeds but reduces the liquidity ratio. The issuer acts strategically to navigate this trade-off, ensuring that liquidity remains compliant while extracting maximum feasible value from operations in the face of potential run dynamics.

3 Comparative Statics

3.1 Redemption Threshold

Combining equations A8 and A9, the closed-form expression for the investor redemption cutoff is given by:

$$\begin{aligned} x_t^*(z_t) = \theta_0 - \sigma_\theta \cdot \Phi^{-1}\left(1 - \frac{1 - c_i - \gamma \cdot \max\{0, L_{\min} - L_t\}}{\eta}\right) + \\ \sigma_x \cdot \Phi^{-1}\left(\theta_0 - \sigma_\theta \cdot \Phi^{-1}\left(1 - \frac{1 - c_i - \gamma \cdot \max\{0, L_{\min} - L_t\}}{\eta}\right)\right) \end{aligned} \tag{25}$$

Letting

$$\kappa_t \equiv \frac{1 - c_i - \gamma \cdot \max\{0, L_{\min} - L_t\}}{\eta},$$

we rewrite the cutoff as:

$$x_t^*(z_t) = \theta_0 - \sigma_\theta \cdot \Phi^{-1}(1-\kappa_t) + \sigma_x \cdot \Phi^{-1}\left(\theta_0 - \sigma_\theta \cdot \Phi^{-1}(1-\kappa_t)\right).$$

Next, consider the effect of the liquidity ratio L_t. When $L_t < L_{\min}$, the penalty term is active and the threshold is increasing in L_t:

$$\frac{\partial x_t^*}{\partial L_t} = \left(\frac{\gamma\sigma_\theta}{\eta} + \frac{\gamma\sigma_x\sigma_\theta}{\eta} \cdot \phi'\left(\theta_0 - \sigma_\theta \cdot \Phi^{-1}(1-\kappa_t)\right)\right) \cdot \phi'\left(\Phi^{-1}(1-\kappa_t)\right). \quad (26)$$

If $L_t \geq L_{\min}$, then the penalty is zero and $\frac{\partial x_t^*}{\partial L_t} = 0$.

3.2 Comparative Statics of Arbitrage Supply

The arbitrage supply function arises from the optimization behavior of symmetric arbitrageurs who simultaneously operate in the primary market (where redemptions occur at a fixed price of 1 minus any applicable fee ϕ_t) and the secondary market (where tokens trade at the market price p_t). Each arbitrageur faces a quadratic inventory cost parameterized by $\chi > 0$ and is required to unwind positions within the same period. The aggregate arbitrage supply Z_t is characterized by a piecewise-linear response in Equation 10 as:

$$Z_t = \begin{cases} n\chi(1-\phi_t-p_t), & \text{if } p_t < 1-\phi_t \\ -n\chi(p_t-1), & \text{if } p_t > 1 \\ 0, & \text{if } p_t \in [1-\phi_t, 1] \end{cases}$$

This formulation highlights the existence of a no-arbitrage band between $[1-\phi_t, 1]$, within which arbitrage is not profitable. Outside this region, arbi-

trageurs engage in primary market redemptions (when $p_t < 1 - \phi_t$) or minting (when $p_t > 1$), exerting price pressure to restore parity between market and redemption values.

The impact of the liquidity fee ϕ_t on arbitrage supply is asymmetric. An increase in ϕ_t widens the no-arbitrage band from below, tightening arbitrage activity when prices fall, while leaving the upper boundary ($p_t = 1$) unchanged. This has two key implications. First, the sensitivity of arbitrage supply to secondary market dislocations becomes weaker for downward deviations in p_t, thereby dampening immediate corrective flows. Second, it introduces a convexity in the arbitrage supply function that endogenously reduces the responsiveness of the primary market to small adverse shocks.

Formally, within the active arbitrage region $p_t < 1 - \phi_t$, the partial derivative of arbitrage supply with respect to the liquidity fee is negative:

$$\frac{\partial Z_t}{\partial \phi_t} = -n\chi < 0. \tag{27}$$

This comparative static reveals a stabilizing ex ante role for liquidity fees: by disincentivizing arbitrage inflows in mild stress episodes, they delay full primary market pass-through, allowing prices to absorb transitory volatility without triggering redemptions. However, in crisis scenarios when p_t sharply diverges from unity, arbitrage reactivates aggressively, preserving the discipline of the peg. This nonlinear arbitrage response is a central feature of our model's transmission mechanism.

Our framework connects with the literature on limited arbitrage in financial markets, particularly the insight that arbitrage is not frictionless, risk-

less, or infinitely elastic. In reality, professional arbitrage requires capital, incurs risk, and often operates under balance sheet and strategic constraints (Shleifer and Vishny, 1997). These frictions are central to our mechanism: we show that stablecoin arbitrage behaves nonlinearly, remaining dormant during mild dislocations—due to liquidity fees and expected volatility—but reactivating aggressively during larger price divergences. In this respect, our model also aligns with the equilibrium limits-to-arbitrage literature (Gromb and Vayanos, 2002), which emphasizes how financial constraints can distort optimal arbitrage activity.

Liquidity Ratio and Arbitrage Supply: Indirect Channel

While the arbitrage supply function in Equation 10 is not explicitly a function of the liquidity ratio L_t, the liquidity buffer plays a critical role in determining the expected redemption dynamics and market price p_t. Specifically, L_t influences investor redemption behavior through the penalty term $\gamma \cdot \max\{0, L_{\min} - L_t\}$, which enters the investor's utility function and therefore shifts the cutoff threshold for strategic runs.

A lower liquidity ratio L_t increases the perceived fragility of the issuer and raises the likelihood of early redemptions by informed investors. As more redemptions are anticipated, secondary market participants expect higher redemption volumes, which places downward pressure on the market price p_t. If p_t drops below the arbitrage trigger $1 - \phi_t$, this activates arbitrageurs in the primary market:

$$\text{If } L_t \downarrow \quad \Rightarrow \quad A_t \uparrow \quad \Rightarrow \quad p_t \downarrow \quad \Rightarrow \quad Z_t \uparrow \text{ (when } p_t < 1 - \phi_t).$$

Thus, although $\partial Z_t/\partial L_t$ is undefined in a direct sense, we can identify an *indirect effect* via the endogenous equilibrium path of p_t. Differentiating implicitly, we obtain:

$$\frac{\partial Z_t}{\partial L_t} = \frac{\partial Z_t}{\partial p_t} \cdot \frac{\partial p_t}{\partial A_t} \cdot \frac{\partial A_t}{\partial x_t^*(z_t)} \cdot \frac{\partial x_t^*(z_t)}{\partial L_t}. \tag{28}$$

The sign of this expression depends on the cutoff's sensitivity to L_t, which from our earlier expression satisfies:

$$\frac{\partial x_t^*(z_t)}{\partial L_t} = \frac{\gamma}{\eta} \cdot \mathbb{1}\{L_t < L_{\min}\} \cdot \frac{\phi(\cdot)}{f_\theta(\cdot)} > 0. \tag{29}$$

Therefore, a decline in L_t raises $x_t^*(z_t)$, meaning more investors redeem, raising A_t, depressing p_t, and ultimately triggering greater arbitrage activity in the primary market when $p_t < 1 - \phi_t$. This mechanism establishes the complementarity between the liquidity ratio and arbitrage supply: tighter ex-ante liquidity buffers suppress early redemptions, which stabilizes prices and delays the onset of arbitrage pressure.

3.3 Optimal Issuer Policy

3.3.1 Liquidity Fee and Minting

The liquidity constraint $L_1 \geq L_{\min}$ plays a central role in shaping the issuer's optimal policy. In the unconstrained region, the liquidity fee

$$\phi^* = \frac{\alpha + \delta n}{2\beta}$$

maximizes fee revenue net of redemption sensitivity. However, since higher ϕ reduces redemptions $A(\phi, n)$, it simultaneously increases the post-redemption liquidity ratio L_1. When $L_1(\phi^*) < L_{\min}$, the issuer must increase ϕ until the

liquidity buffer is restored, thereby imposing a lower bound $\phi \geq \phi^{\min}$ to ensure compliance.

Similarly, the optimal minting quantity m^* must account for the negative impact on L_1:

$$\frac{\partial L_1}{\partial m} = -\frac{L_0 S + \phi A - A}{(S + m - A)^2} < 0.$$

If the liquidity ratio is sufficiently high, minting is unconstrained. Otherwise, the issuer faces a trade-off between raising new funds and preserving regulatory liquidity standards. The binding constraint introduces an implicit upper bound on m that tightens as L_1 approaches $L_{\min}$.

In summary, the optimal policy must internalize the endogeneity of the liquidity ratio, which links investor behavior, fee policy, arbitrage capacity, and minting incentives through a dynamic solvency constraint.

3.3.2 Optimal Arbitrageur Capacity

We now characterize the issuer's optimal choice of arbitrage capacity n^* under two regimes: one with diminishing returns to arbitrage efficiency and another where the liquidity constraint binds.

Case I: Explicit Solution under Diminishing Returns: Suppose the arbitrage supply exhibits diminishing returns, so redemptions take the form:

$$A(\phi, n) = \alpha - \beta\phi + \delta n^{\xi}, \quad \text{with } 0 < \xi < 1.$$

For tractability, assume $\xi = \frac{1}{2}$, so that:

$$A(\phi, n) = \alpha - \beta\phi + \delta\sqrt{n}.$$

If the liquidity constraint is slack ($L_1 \geq L_{\min}$), the issuer's problem becomes:

$$\max_n \quad V_0 = \phi A(\phi, n) + p_m m - \kappa n.$$

Substituting $A(\phi, n)$ and differentiating:

$$\frac{\partial V_0}{\partial n} = \phi \cdot \frac{\delta}{2\sqrt{n}} - \kappa.$$

Setting the first-order condition $\partial V_0 / \partial n = 0$, we obtain the explicit solution:

$$n^* = \left(\frac{\delta \phi}{2\kappa} \right)^2.$$

Case II: Implicit Solution under Binding Liquidity Constraint: Now suppose that redemptions follow a linear form:

$$A(\phi, n) = \alpha - \beta\phi + \delta n,$$

and the liquidity constraint binds ($L_1 < L_{\min}$), so the issuer's problem becomes:

$$V_0 = \phi A(\phi, n) + p_m m - \kappa n - \gamma \cdot (L_{\min} - L_1).$$

The liquidity ratio at $t = 1$ is:

$$L_1 = \frac{L_0 S + \phi A(\phi, n) - A(\phi, n)}{S + m - A(\phi, n)}.$$

Letting $A_n := \frac{\partial A}{\partial n} = \delta$, the first-order condition is:

$$\frac{\partial V_0}{\partial n} = \phi\delta - \kappa + \gamma \cdot \frac{\partial L_1}{\partial A} \cdot \delta = 0,$$

where

$$\frac{\partial L_1}{\partial A} = \frac{(S+m-A)(\phi-1)+(L_0 S+\phi A-A)}{(S+m-A)^2}.$$

This yields an implicit solution for n^*, which can be solved numerically given parameter values.

Taken together, these expressions describe how the optimal number of arbitrageurs balances the tradeoff between redemption stabilization and arbitrage support costs, depending on whether the issuer faces regulatory liquidity constraints.

4 *Ex-Ante* Response to Liquidity Regulation

This subsection formally explains how a liquidity fee ϕ_t, applied exclusively in the primary market to arbitrageurs, can influence the behavior of secondary market investors, even if those investors do not directly pay the fee. The channel operates through the effect of the fee on arbitrageurs' willingness to transact, which in turn shapes equilibrium prices observed by investors. We develop the logic step-by-step within a forward-looking market structure.

4.1 Arbitrageur Behavior

I model the arbitrageur's expected profit from purchasing stablecoins in the secondary market and redeeming them in the primary market at face value, subject to liquidity fees and redemption frictions. Let p_t denote the stablecoin price in the secondary market at time t, and let $\phi_t \in [0,1]$ represent the liquidity fee applied by the issuer in the primary market. The arbitrageur redeems at a gross face value of \$1, but the net redemption value is reduced by ϕ_t and by an expected liquidation discount $\kappa_t \in [0,1]$, which captures fire-sale losses

or delays in accessing reserve assets. Redemption also carries a probability $\lambda_t \in [0, 1]$ that the issuer remains solvent and honors withdrawals.

The arbitrageur's expected profit per unit at time t is given by:

$$\Pi_t = \lambda_t \cdot (1 - \phi_t - \kappa_t) - p_t \tag{30}$$

Arbitrageurs enter the trade only when $\Pi_t > 0$, so the arbitrage constraint imposes an upper bound on the secondary market price:

$$p_t < \lambda_t \cdot (1 - \phi_t - \kappa_t) \tag{31}$$

This inequality defines the maximum price at which arbitrageurs are willing to purchase stablecoins in the secondary market, thereby enforcing a soft price cap driven by expected redemption economics.

We now consider a dynamic environment in which regulatory changes are pre-announced. Suppose that at time $t = T$, a new policy is enacted that imposes a liquidity fee $\phi_T > 0$ and a binding minimum liquidity requirement $L_{\min} > 0$. Prior to enforcement, at $t = T - 1$, the fee is absent ($\phi_{T-1} = 0$), and the issuer is not yet constrained by the new rules.

At time $T - 1$, the arbitrageur's expected profit is:

$$\Pi_{T-1} = \lambda_{T-1} \cdot (1 - \kappa_{T-1}) - p_{T-1} \tag{32}$$

At time T, under the new regulatory regime, the arbitrageur expects:

$$\Pi_T = \lambda_T \cdot (1 - \phi_T - \kappa_T) - p_T \tag{33}$$

The intertemporal arbitrage condition compares these values. If $\Pi_T > \Pi_{T-1}$, the arbitrageur prefers to delay participation until the fee is active. This yields the condition:

$$\lambda_T(1 - \phi_T - \kappa_T) - p_T > \lambda_{T-1}(1 - \kappa_{T-1}) - p_{T-1} \tag{34}$$

This inequality characterizes the arbitrageur's forward-looking decision. The future imposition of a fee $\phi_T > 0$ lowers expected profits. However, this may be offset by an improvement in the issuer's solvency outlook, such that $\lambda_T > \lambda_{T-1}$, and a reduction in expected fire-sale losses, such that $\kappa_T < \kappa_{T-1}$. Both effects arise endogenously from the issuer's compliance with a binding liquidity requirement $L_{\min}$, which improves the reserve buffer and reduces redemption risk.

Hence, arbitrageurs face an intertemporal tradeoff: delay offers the prospect of redeeming into a more stable system with lower liquidation frictions, but at the cost of paying the liquidity fee. The net preference depends on the relative magnitudes of ϕ_T, $\Delta\kappa = \kappa_T - \kappa_{T-1}$, and $\Delta\lambda = \lambda_T - \lambda_{T-1}$. Anticipatory behavior by arbitrageurs, especially in thin markets, can shift secondary market prices in advance of the policy change, even when arbitrageurs themselves are the only parties subject to the fee.

4.2 Investors' Response

Although investors do not interact directly with the primary market, their payoff is shaped endogenously by the arbitrageur's willingness to enforce price parity. Investors transact solely in the secondary market, and their proceeds from selling are determined by the market-clearing price p_t, which is anchored by arbitrage activity.

Because arbitrageurs redeem in the primary market at face value \$1 but incur a liquidity fee ϕ_t, a redemption haircut κ_t, and a risk of issuer failure captured by $\lambda_t \in [0, 1]$, their reservation price in the secondary market is:

$$p_t = \lambda_t(1 - \phi_t - \kappa_t) \tag{35}$$

Thus, the investor's selling payoff is indirectly but strictly decreasing in ϕ_t, increasing in expected solvency λ_t, and decreasing in liquidation friction κ_t. Any deterioration in these parameters leads to a decline in the price arbitrageurs are willing to pay, and hence a decline in investor utility, despite investors never facing ϕ_t directly.

Now consider the dynamic implications of a pre-announced policy change. Suppose that at some future date T, the regulator imposes a binding liquidity fee $\phi_T > 0$ on all redemptions. Prior to that date, in periods $t < T$, there is no fee ($\phi_t = 0$). Under this regime, arbitrageurs in earlier periods are willing to pay:

$$p_{t<T} = \lambda_t(1 - \kappa_t) \tag{36}$$

However, once the fee becomes active, the arbitrage price ceiling drops to:

$$p_T = \lambda_T(1 - \phi_T - \kappa_T) \tag{37}$$

Even under a constant solvency belief ($\lambda_T = \lambda_{T-1}$) and unchanged liquidation losses ($\kappa_T = \kappa_{T-1}$), the imposition of $\phi_T > 0$ causes a discrete drop in the investor's expected payoff. Formally, the price change is given by:

$$\Delta p = p_{T-1} - p_T = \lambda_T(\phi_T + \Delta\kappa) \tag{38}$$

where $\Delta\kappa = \kappa_T - \kappa_{T-1} \geq 0$ captures the endogenous increase in expected losses under fee-enforced stress. This shift in arbitrage pricing directly translates into a run incentive for investors.

If investors expect that the secondary market price will decline at time T due to the anticipated fee, they strictly prefer to sell in period $T-1$. That is, they compare:

$$U_{T-1}^{\text{sell}} = p_{T-1} > p_T = U_T^{\text{sell}} \tag{39}$$

This inequality implies that a forward-looking investor will liquidate holdings preemptively. When this behavior is generalized across the investor base, it results in a form of endogenous run: a price-decline-driven liquidation wave triggered not by current fundamentals, but by anticipated future policy.

The welfare implications are critical. While the liquidity fee is intended to discourage run behavior once imposed, its mere announcement can generate instability in advance. If arbitrageur balance sheets are insufficient to absorb investor outflows in $T-1$, the resulting order imbalance leads to a discon-

tinuous drop in prices and potentially a disorderly unwind of positions. The secondary market becomes illiquid not because of poor fundamentals, but because the arbitrage boundary itself is mechanically shifted by the policy. In this sense, the liquidity fee has the paradoxical potential to promote *ex-ante* instability even as it aims to support *ex-post* solvency.

A close parallel exists between the anticipatory dynamics in my stablecoin model and investor behavior during the implementation of key MMF reforms. When the SEC mandated a transition from constant NAV (CNAV) to floating NAV (FNAV) for prime institutional MMFs in 2015–2017, roughly $1 trillion in assets exited prime funds in favor of government MMFs (Allen and Winters (2020); Cipriani and La Spada (2021)) and offshore alternatives (Fricke et al. (2024)). Notably, these reallocations occurred before the reform went into effect, reflecting forward-looking responses by institutional investors seeking to avoid uncertain liquidity or pricing regimes. Similarly, in my setting, arbitrageurs and investors internalize the future imposition of a liquidity fee and adjust their behavior in advance, driving price effects and market imbalances prior to fee activation. The model captures this anticipatory unwind by formally characterizing how intertemporal incentives shift the arbitrage boundary and compress liquidity provision. These predictions are empirically validated in the context of the SEC's 2024 liquidity fee provision, which saw significant fund outflows in the months leading up to the compliance deadline (Keshav (2024)). Together, these findings underscore a general principle: when *ex-post* stabilization tools are telegraphed in advance, they can distort *ex-ante* behavior and catalyze market fragility—even in the absence of immediate fundamental shocks.

4.3 Issuer's Response

We now turn to the issuer's forward-looking decision problem when a liquidity fee $\phi_T > 0$ and/or a minimum liquidity requirement $L_{\min} > 0$ is announced to take effect at a known future date T. While investors are not charged the fee directly, their behavior adjusts in anticipation, as demonstrated previously. The issuer must account for these anticipatory dynamics, particularly the potential for preemptive investor redemptions at $t = T - 1$, and determine its optimal policy path in periods $t < T$.

The issuer collects revenue from several sources. These include liquidity fees $\phi_T \cdot R_T$ applied to redemptions via arbitrage at time T, residual returns from reserve assets not liquidated in a run, and potentially retained seigniorage or dividends.[12] At the same time, the issuer faces risks of fire-sale costs, NAV violations, and franchise value destruction when investor redemptions are concentrated and market liquidity is thin.

Let R_t denote redemptions via arbitrageurs at time t, ϕ_t the liquidity fee set at time t, θ_t the fundamental solvency state, and L_t the liquidity ratio. Let $\kappa(R_t, L_t) \in [0, 1]$ denote the expected per-unit loss from liquidation, increasing in R_t and decreasing in L_t, capturing the endogenous fire-sale discount. The issuer's continuation value at time t is denoted V_t.

The dynamic optimization problem of the issuer for any period $t < T$ is given by:

$$V_t = \max_{\phi_t,\, L_t,\, R_t} \{(1 - \kappa(R_t, L_t))R_t + \Pi_t + \beta \cdot \mathbb{E}_t \left[V_{t+1}(\theta_{t+1})\right]\} \tag{40}$$

[12] See, Ma et al. (2023) for dividend provision.

where Π_t captures other period-specific returns (such as investment income or retained surplus), and $\beta \in (0, 1)$ is the discount factor.

The key complexity arises at $t = T - 1$, the final period before fee enforcement. The issuer anticipates that arbitrageur redemptions R_T will decline once the fee $\phi_T > 0$ becomes binding, while redemptions may spike at $T - 1$ as investors exit in anticipation. This creates a discontinuity in redemption behavior:

$$\begin{aligned} R_{T-1} \uparrow \quad & \text{(preemptive selling)} \\ R_T \downarrow \quad & \text{(post-fee retrenchment)} \end{aligned} \tag{41}$$

These shifts also feed into liquidation costs, with $\kappa(R_{T-1}, L_{T-1})$ potentially rising due to volume-induced pressure, amplifying losses from fire-sales.

The issuer's *ex-ante* valuation at $T - 1$ thus becomes:

$$\mathbb{E}[V_{T-1}] = (1 - \kappa(R_{T-1}, L_{T-1}))R_{T-1} + \beta \cdot \mathbb{E}[(1 - \kappa(R_T, L_T))R_T + \cdots] \tag{42}$$

While the fee ϕ_T suppresses redemptions at T, its very announcement causes front-loaded withdrawals in $T - 1$, which may trigger large fire-sale costs and reduce the issuer's expected value. Unless these anticipatory outflows are offset by a sufficiently high liquidity buffer L_{T-1}, or unless the issuer pre-commits to a smoothing policy to anchor arbitrageur expectations, the introduction of a liquidity fee may reduce net value rather than enhance it.

This result underscores an intertemporal policy tradeoff. Liquidity fees, when applied credibly and proactively, can mitigate run dynamics under stress.

However, if poorly timed or anticipated by sophisticated arbitrageurs, they can accelerate redemptions into the pre-enforcement window, raising fragility *ex-ante*. In equilibrium, the issuer must solve not only for optimal fees and liquidity ratios, but also for the timing and credibility of transition policies in order to minimize run risk and preserve solvency.

Given that the issuer anticipates redemptions intensifying in advance of the compliance date for a liquidity fee, regulatory authorities must permit issuers to accumulate sufficient liquidity buffers *ex-ante* to absorb such anticipated outflows from both the primary and secondary markets. This principle has been operationalized in the context of US MMF regulation: Under the SEC's 2023 reform, mandatory liquidity ratio thresholds became effective in April 2024, several months before the effective date in October 2024 for the liquidity fee provision (SEC (2023)). Thus, the *ex-ante* redemptions from MMFs occurred without triggering market instability (Keshav (2024)).

5 Effect of Stale Pricing

Stale pricing in fixed income markets arises when fund net asset values (NAVs) fail to reflect real-time secondary market valuations, typically due to illiquidity, delayed mark-to-market accounting, or reliance on matrix pricing. This disconnect creates arbitrage opportunities and distorts investor incentives, especially during periods of stress (Choi et al. (2022)). A similar dynamic is present in stablecoin markets, where primary market redemption values are fixed (often at $1) while secondary market prices fluctuate. When arbitrageurs face frictions or capacity constraints, deviations between the stablecoin's secondary price and its fixed primary NAV can persist—mirroring the stale NAV

phenomenon in bond funds. These pricing gaps weaken peg discipline and can amplify fragility, particularly when arbitrage supply is endogenous or constrained.

5.1 Investor Run Threshold with Stale Pricing

In this section, we model investor redemption behavior under informational frictions arising from stale NAV signals. Investors observe a delayed public signal

$$z_t = \theta_{t-\Delta} + \eta_t,$$

where θ_t denotes the latent fundamental value of the issuer's reserves, $\Delta \geq 0$ represents the lag in information dissemination (e.g., due to infrequent portfolio disclosure), and $\eta_t \sim \mathcal{N}(0, \sigma_z^2)$ is Gaussian noise. A run occurs if the investor expects that aggregate redemptions A_t will exceed the actual reserve value θ_t. Since investors form beliefs based on stale and noisy information, the perceived run condition becomes:

$$\mathbb{P}(A_t > \theta_t) = c + \tau(L_t),$$

where c denotes a baseline withdrawal friction (e.g., reputational cost) and $\tau(L_t)$ is an increasing function capturing the penalty associated with low liquidity buffers.

We assume that $\theta_t \sim \mathcal{N}(\hat{\theta}_t, \sigma_0^2)$, where $\hat{\theta}_t = \mathbb{E}_t[\theta_t \mid z_t]$ is the belief based only on the stale public signal. Then the perceived probability of shortfall is:

$$\Phi\left(\frac{A_t - \hat{\theta}_t}{\sigma_0}\right) = c + \tau(L_t),$$

where $\Phi(\cdot)$ is the standard normal CDF. This expression can be inverted to solve for the threshold level of redemptions A_t that makes investors indifferent between staying and redeeming:

$$A_t = \hat{\theta}_t + \sigma_0 \cdot \Phi^{-1}\left(c + \tau(L_t)\right).$$

This characterization highlights how stale pricing dynamics shape the timing and severity of redemption waves. When Δ is large, $\hat{\theta}_t$ reacts sluggishly to deteriorating fundamentals, delaying investor withdrawals until insolvency is more likely. However, once triggered, runs become more severe because redemptions overshoot updated reserve values.

5.2 Stale Pricing and Its Distortion of Arbitrage Incentives

We consider a setting in which the issuer redeems shares at a fixed primary market value of \$1 per unit, i.e., $\text{NAV}_t = \$1$ by design. However, the true market value of the underlying assets, denoted v_t, evolves dynamically and may not be directly observed. The secondary market price p_t reflects contemporaneous information about v_t, incorporating both real-time asset valuations and expectations of redemption frictions. In contrast, the primary market redemption price remains mechanically pegged, and thus fails to internalize valuation changes stemming from asset illiquidity or credit impairment. This creates a wedge between the true market value and the nominal redemption value, especially in periods of stress or incomplete information.

Let ϕ_t denote the liquidity fee imposed on primary market redemptions. The actual arbitrage profit from purchasing one unit at p_t and redeeming in

the primary market is given by

$$\pi_t^{\text{true}} = 1 - \phi_t - p_t.$$

Under full information and rational expectations, arbitrageurs would exploit this spread, supplying liquidity when the secondary market price falls below $1-\phi_t$, thereby stabilizing prices. However, when arbitrageurs form expectations based on stale or internal NAV estimates—potentially reflecting lagged asset marks, delayed disclosure, or internal risk adjustments—they may understate the actual redemption value. Suppose the arbitrageur's perceived redemption value is $\text{NAV}_t < 1$, then the perceived arbitrage profit becomes

$$\hat{\pi}_t = \text{NAV}_t - p_t,$$

which understates π_t^{true} when $\text{NAV}_t < 1 - \phi_t$. Let arbitrage supply be linear in perceived profit:

$$Z_t = n \cdot \chi \cdot \hat{\pi}_t = n \cdot \chi(\text{NAV}_t - p_t).$$

This yields a distorted supply response:

$$Z_t^{\text{stale}} = n \cdot \chi(\text{NAV}_t - p_t) < n \cdot \chi(1 - \phi_t - p_t) = Z_t^{\text{true}}.$$

Arbitrage capital misallocates due to the misperception of redemption value, muting price pressures that would otherwise force $p_t \to 1 - \phi_t$. This distortion is persistent and nontrivial: it is not due to physical frictions or execution lags but rather to belief-based miscalibration caused by information asymmetry or anchoring to internal valuation models.

The resulting mispricing has significant implications for market stability. First, price volatility increases as arbitrageurs fail to fully exploit profitable deviations, reducing market resilience in the face of shocks. Second, strategic investors observing persistent discounts and weakened arbitrage support may lower their redemption thresholds, amplifying the likelihood of a run via coordination failures. Finally, although liquidity fees are intended to reduce fragility by discouraging redemptions, their efficacy is compromised when arbitrageurs fail to enforce pricing discipline. In such an environment, the presence of a fixed NAV can become a source of instability rather than a stabilizing anchor, particularly when investor beliefs about underlying asset values diverge from observable prices.

5.3 Issuer's Dynamic Bellman Problem with Stale Pricing

We model the issuer's optimization problem in a setting where it faces uncertainty about the true value of reserve assets due to stale information and must respond dynamically to redemptions, liquidity constraints, and arbitrage costs. Let θ_t denote the latent fundamental value of the issuer's reserves at time t, which is not directly observed. Instead, the issuer observes a stale public signal:

$$z_t = \theta_{t-\Delta} + \eta_t \tag{43}$$

where $\Delta \geq 0$ captures the degree of staleness and η_t is noise. This delay reflects the fact that redemption activity and valuation signals may be based on outdated asset prices or lagged portfolio disclosures.

Let the state vector at time t be $s_t = (\hat{\theta}_t, A_t, L_t)$, where $\hat{\theta}_t$ is the issuer's

posterior belief about θ_t, A_t denotes redemptions, and L_t is the liquidity ratio. The issuer's belief is updated via:

$$\hat{\theta}_t = \mathbb{E}_t[\theta_t \mid z_t] = \rho^{\Delta} z_t + (1 - \rho^{\Delta})\mu \tag{44}$$

where μ is the unconditional mean of θ_t and $\rho \in (0, 1)$ captures the autocorrelation of the fundamental process. The higher the delay Δ, the less responsive the posterior $\hat{\theta}_t$ is to current observations.

At each time t, the issuer selects a liquidity fee ϕ_t, post-redemption liquidity ratio L_t, and arbitrage support capacity n_t to maximize its dynamic value function:

$$V_t(s_t) = \max_{\phi_t, L_t, n_t} \Big\{ \Pi(\hat{\theta}_t, \phi_t, L_t, A_t) + \beta\, \mathbb{E}_t[V_{t+1}(s_{t+1})] \Big\} \tag{45}$$

The instantaneous profit function Π captures three components:

$$\Pi(\hat{\theta}_t, \phi_t, L_t, A_t) = (1 - \phi_t) \cdot \min(A_t, \hat{\theta}_t) - \kappa \cdot \mathbb{1}\{L_t < L_{\min}\} - \psi(n_t) \tag{46}$$

The first term reflects net redemption revenue, where the issuer pays out one unit per redeemed token but retains ϕ_t in fees. However, the issuer may only be able to honor redemptions up to the perceived reserve value $\hat{\theta}_t$, leading to a truncation of outflows. The second term imposes a penalty κ when post-redemption liquidity falls below a regulatory minimum $L_{\min}$. The third term $\psi(n_t)$ represents the cost of supporting n_t arbitrageurs, assumed to be convex in n_t.

Post-redemption liquidity is updated according to Equation 4:

$$L_{t+1} = L_t + R_t - \lambda A_t,$$

where L_0 is the pre-redemption liquidity ratio and $\lambda \in (0, 1)$ captures the degree to which redemptions draw down liquid buffers.

This recursive formulation captures the issuer's intertemporal tradeoff: charging a higher fee ϕ_t may reduce redemptions and preserve liquidity, but may also weaken arbitrage activity and depress future demand. Similarly, greater arbitrage support n_t stabilizes prices and enhances market confidence but comes at a resource cost. The added friction of stale pricing compounds these challenges, as current policy decisions are made under distorted signals about the true state of solvency.

6 Conclusion

This paper develops a dynamic model of stablecoin fragility in which three key agents, investors, arbitrageurs, and the issuer, interact in markets of liquidity mismatch, coordination risk, and segmented markets. Investors receive noisy signals about the issuer's solvency and must decide whether and when to redeem, taking into account the expected actions of others. Arbitrageurs operate across primary and secondary markets, enforcing the price peg but facing balance sheet constraints and inventory frictions. The issuer, in turn, selects policy instruments, namely the liquidity fee applied to redemptions in the primary market, the number of contracted arbitrageurs, and the volume of new minted stablecoins, subject to a liquidity management constraint. The model embeds strategic complementarities among agents and explicitly

solves for run thresholds, arbitrage bounds, and the issuer's optimal policy path under both dynamic and two-period environments. This unified framework provides a tractable yet rich foundation for analyzing the economics of stablecoin stabilization.

A central insight of the analysis is that liquidity fees and liquidity buffers are jointly effective in mitigating run risk. A sufficiently large liquidity buffer reduces the expected dilution cost of late redemptions, thereby lowering the investor's incentive to run. In contrast, a liquidity fee discourages early redemptions by imposing a cost on arbitrage-driven withdrawals in the primary market while simultaneously replenishing reserves. However, the presence of one tool crowds out the marginal benefit of the other, generating a non-linear trade-off in the issuer's optimization problem. The optimal policy thus requires a calibrated combination of liquidity management and pricing incentives, rather than reliance on a single instrument.

The model highlights a novel *ex-ante* channel: when liquidity fees or binding reserve requirements are announced in advance, forward-looking arbitrageurs and investors adjust their behavior before the policy takes effect. Arbitrageurs revise their reservation prices downward in expectation of future redemption frictions, and investors anticipate a lower secondary market price path. This shift in expectations can trigger preemptive redemptions, even when the fundamentals remain unchanged. The issuer must internalize this anticipatory behavior when designing transition paths, as poorly timed or noncredible announcements may inadvertently induce runs and amplify fragility in the pre-implementation phase.

When secondary market prices persistently deviate from the issuer's net asset value (NAV), a phenomenon called stale pricing, arbitrators face distorted incentives. Even when redemption is profitable in expectation, the inability to accurately observe NAV in real time can delay arbitrage activity and weaken peg enforcement. This friction becomes more severe when balance sheet constraints or information asymmetries are present. The model demonstrates how stale pricing not only undermines the efficacy of liquidity fees, but also contributes to miscoordination among investors, further heightening redemption pressure. Addressing pricing transparency is therefore essential to safeguarding stablecoin stability.

The results have direct implications for the optimal design of stablecoin regulatory frameworks. Effective stabilization requires a coordinated deployment of liquidity fees, arbitrage capacity, and reserve buffers. However, poorly timed or insufficiently credible policy announcements can backfire, triggering anticipatory runs and price instability. Policymakers must weigh not only the static deterrent effect of liquidity fees but also their dynamic consequences on investor and arbitrageur expectations. Welfare is maximized when policies are implemented preemptively, calibrated jointly, and supported by mechanisms that mitigate stale pricing and preserve transparency.

Declaration of Generative AI and AI-assisted Technologies in the Writing Process

The authors did not use any generative AI or related tools during the preparation of this work. The author assumes full responsibility for the content of the publication.

Declaration of Funding Sources

This research did not receive specific grants from funding agencies in the public, commercial or non-profit sectors.

References

[1] Abadi, J. and Brunnermeier, M. (2018). Blockchain economics. Technical report, National Bureau of Economic Research.

[2] Abreu, D. and Brunnermeier, M. K. (2003). Bubbles and crashes. *Econometrica*, 71(1):173–204.

[3] Allen, F. and Gale, D. (1998). Optimal financial crises. *Journal of Finance*, 53:1245–1284.

[4] Allen, F. and Gale, D. (2004). Financial intermediaries and markets. *Econometrica*, 72:1023–1061.

[5] Allen, K. D. and Winters, D. (2020). Crisis regulations: The unexpected consequences of floating nav for money market funds. *Journal of Banking & Finance*, 117:105851.

[6] Anadu, K., Azar, P., Cipriani, M., Eisenbach, T. M., Huang, C., Landoni, M., La Spada, G., Macchiavelli, M., Malfroy-Camine, A., and Wang, J. C. (2023). Runs and flights to safety: Are stablecoins the new money market funds? *FRB of Boston Supervisory Research & Analysis Unit Working Paper No. SRA*, pages 23–02.

[7] Bernardo, A. and Welch, I. (2004). Liquidity and financial market runs. *Quarterly Journal of Economics*, 119:135–158.

[8] Brunnermeier, M. K. and Oehmke, M. (2013). Bubbles, financial crises, and systemic risk. In *Handbook of the Economics of Finance*, volume 2, pages 1221–1288.

[9] Bryant, J. (1980). A model of reserves, bank runs, and deposit insurance. *Journal of Banking & Finance*, 4(4):335–344.

[10] Chari, V. V. and Jagannathan, R. (1988). Banking panics, information, and rational expectations equilibrium. *The Journal of Finance*, 43(3):749–761.

[11] Chernenko, S. and Sunderam, A. (2014). Frictions in shadow banking: Evidence from the lending behavior of money market mutual funds. *The Review of Financial Studies*, 27(6):1717–1750.

[12] Choi, J., Kronlund, M., and Oh, J. Y. J. (2022). Sitting bucks: Stale pricing in fixed income funds. *Journal of Financial Economics*, 145:296–317.

[13] Cipriani, M. and La Spada, G. (2021). Investors' appetite for money-like assets: The mmf industry after the 2014 regulatory reform. *Journal of Financial Economics*, 140(1):250–269.

[14] Cong, L. W., Li, Y., and Wang, N. (2021). Tokenomics: Dynamic adoption and valuation. *The Review of Financial Studies*, 34:1105–1155.

[15] Diamond, D. W. and Dybvig, P. H. (1983). Bank runs, deposit insurance, and liquidity. *Journal of Political Economy*, 91(3):401–419.

[16] D'Avernas, A., Maurin, V., and Vandeweyer, Q. (2022). Can stablecoins be stable? Working Paper.

[17] Farhi, E., Golosov, M., and Tsyvinski, A. (2009). A theory of liquidity and regulation of financial intermediation. *Review of Economic Studies*, 76:973–992.

[18] Fricke, D., Greppmair, S., and Paludkiewicz, K. (2024). You can't always get what you want (where you want it): Cross-border effects of the us money market fund reform. *Journal of International Economics*, 147:103846.

[19] Gallagher, E. A., Schmidt, L. D., Timmermann, A., and Wermers, R. (2020). Investor information acquisition and money market fund risk rebalancing during the 2011–2012 eurozone crisis. *The Review of Financial Studies*, 33(4):1445–1483.

[20] Goldstein, I. and Pauzner, A. (2005). Demand deposit contracts and the probability of bank runs. *Journal of Finance*, 60:1293–1328.

[21] Gorton, G., Klee, E. C., Ross, C., Ross, S., and Vardoulakis, A. P. (2023). Leverage and stablecoin pegs. Working Paper.

[22] Gorton, G. and Pennacchi, G. (1990). Financial intermediaries and liquidity creation. *Journal of Finance*, 45:49–71.

[23] Gorton, G., Ross, C., and Ross, S. (2022). Making money. Working Paper.

[24] Gorton, G. and Zhang, J. (2021). Taming wildcat stablecoins. Working Paper.

[25] Gromb, D. and Vayanos, D. (2002). Equilibrium and welfare in markets with financially constrained arbitrageurs. *Journal of Financial Economics*, 66:361–407.

[26] Gryglewicz, S., Mayer, S., and Morellec, E. (2021). Optimal financing with tokens. *Journal of Financial Economics*, 142(3):1038–1067.

[27] Hanson, S. G., Scharfstein, D. S., and Sunderam, A. (2015). An evaluation of money market fund reform proposals. *IMF Economic Review*, 63(4):984–1023.

[28] Harvey, C., Ramachandran, A., and Santoro, J. (2021). *DeFi and the Future of Finance*. John Wiley & Sons.

[29] Hu, A. S., Parlour, C. A., and Rajan, U. (2019). Cryptocurrencies: Stylized facts on a new investible instrument. *Financial Management*, 48(4):1049–1068.

[30] Jacklin, C. (1987). Demand deposits, trading restrictions and risk sharing. In Prescott, E. D. and Wallace, N., editors, *Contractual Arrangements for Intertemporal Trade*. University of Minnesota Press, Minnesota.

[31] Jacklin, C. J. and Bhattacharya, S. (1988). Distinguishing panics and information-based bank runs: Welfare and policy implications. *Journal of political economy*, 96(3):568–592.

[32] John, K., Kogan, L., and Saleh, F. (2022). Smart contracts and decentralized finance. Working Paper.

[33] Kacperczyk, M. and Schnabl, P. (2010). When safe proved risky: commercial paper during the financial crisis of 2007–2009. *Journal of Economic Perspectives*, 24(1):29–50.

[34] Kacperczyk, M. and Schnabl, P. (2013). How safe are money market funds? *The Quarterly Journal of Economics*, 128(3):1073–1122.

[35] Keshav, V. (2024). Liquidity fees in open-end mutual funds: Theory and evidence. (24/17). Available at SSRN.

[36] Keshav, V. (2025). Redemption incentives, regulatory buffers, and cross-border spillovers in money market funds. SSRN Working Paper.

[37] Keshav, V. and Saha, P. (2023). Does bank sponsorship matter during banking crisis: Evidence from the us money market funds. *Available at SSRN*.

[38] Keshav, V. and Winters, D. (2025a). European money market fund investors demand liquidity. *European Financial Management (Forthcoming)*.

[39] Keshav, V. and Winters, D. (2025b). From bunds to borrowers: Sovereign risk and the reallocation behavior of money market funds. SSRN Working Paper.

[40] Koont, N., Ma, Y., Pastor, L., and Zeng, Y. (2022). Steering a ship in illiquid waters: Active management of passive funds. Working Paper.

[41] Li, J. and Mann, W. (2018). Digital tokens and platform building. Unpublished Manuscript.

[42] Li, L., Li, Y., Macchiavelli, M., and Zhou, X. (2021). Liquidity restrictions, runs, and central bank interventions: Evidence from money market funds. *The Review of Financial Studies*, 34(11):5402–5437.

[43] Li, Y. and Mayer, S. (2021). Money creation in decentralized finance: A dynamic model of stablecoin and crypto shadow banking. Working Paper.

[44] Liu, J., Makarov, I., and Schoar, A. (2023). Anatomy of a run: The terra luna crash. Working Paper.

[45] Liu, Y. and Tsyvinski, A. (2021). Risks and returns of cryptocurrency. *The Review of Financial Studies*, 34(6):2689–2727.

[46] Ma, Y., Zeng, Y., and Zhang, A. L. (2023). Stablecoin runs and the centralization of arbitrage. SSRN Working Paper. Available at SSRN: magentahttps://ssrn.com/abstract=4398546 or magentahttp://dx.doi.org/10.2139/ssrn.4398546.

[47] Makarov, I. and Schoar, A. (2021). Blockchain analysis of the bitcoin market. Technical report, National Bureau of Economic Research.

[48] Makarov, I. and Schoar, A. (2022). Cryptocurrencies and decentralized finance (defi). *Brookings Papers on Economic Activity*, Spring 2022:1–71.

[49] Morris, S. and Shin, H. (1998). Unique equilibrium in a model of self-fulfilling currency attacks. *American Economic Review*, 88:587–597.

[50] Morris, S. and Shin, H. (2003). Global games: Theory and applications. In Dewatripont, M., Hansen, L. P., and Turnovsky, S. J., editors, *Advances in Economics and Econometrics*, volume 3, pages 56–114. Cambridge University Press. Proceedings of the Eighth Congress of the Econometric Society.

[51] Morris, S. and Shin, H. S. (2004). Coordination risk and the price of debt. *European Economic Review*, 48(1):133–153.

[52] Parlatore, C. (2016). Fragility in money market funds: Sponsor support and regulation. *Journal of Financial Economics*, 121(3):595–623.

[53] Schmidt, L., Timmermann, A., and Wermers, R. (2016). Runs on money market mutual funds. *American Economic Review*, 106(9):2625–2657.

[54] SEC (2014). Money market fund reform; amendments to form pf.

[55] SEC (2023). 17 cfr parts 270, 274, and 279 [release nos. 33-11211; 34-97876; ia-6344; ic-34959; file no. s7-22-21]. Accessed via Federal Register or SEC official records.

[56] SEC (July 2023). Money market fund statistics. *Division of Investment Management Analytics Office*.

[57] Shleifer, A. and Vishny, R. (1997). The limits of arbitrage. *Journal of Finance*, 52:35–55.

[58] Sockin, M. and Xiong, W. (2023a). Decentralization through tokenization. *Journal of Finance*, 78:247–299.

[59] Sockin, M. and Xiong, W. (2023b). A model of cryptocurrencies. *Management Science*. Forthcoming.

[60] Uhlig, H. (2022). A lunatic stablecoin crash. Working Paper.

[61] U.S. Congress (2025). Guiding and establishing national innovation for u.s. stablecoins of 2025 (genius act of 2025). Introduced in U.S. Senate. Introduced on February 4, 2025.

Figures

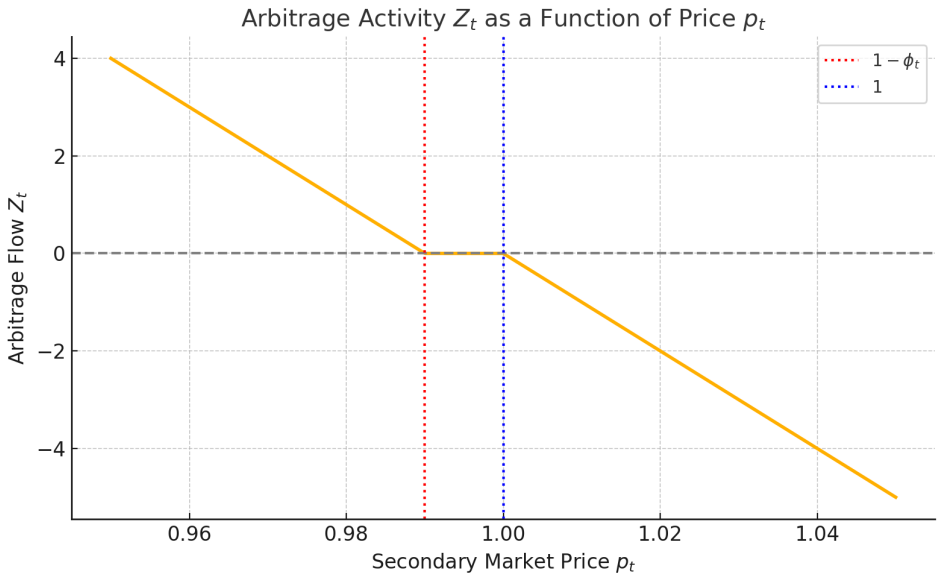


Figure 1: Arbitrage Supply vs. p_t

The above figure plots relationship between secondary market price (p_t) and Arbitrage Supply (Z_t). Source: Author's calculations.

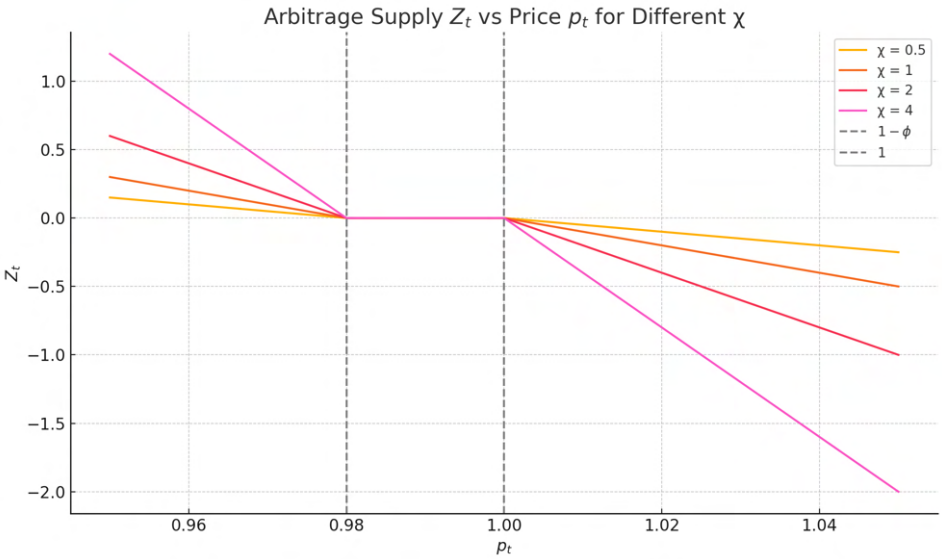


Figure 2: Arbitrage Supply vs. χ

The above figure plots relationship between secondary market price (p_t) and Arbitrage Supply (Z_t) for different χ. Source: Author's calculations.

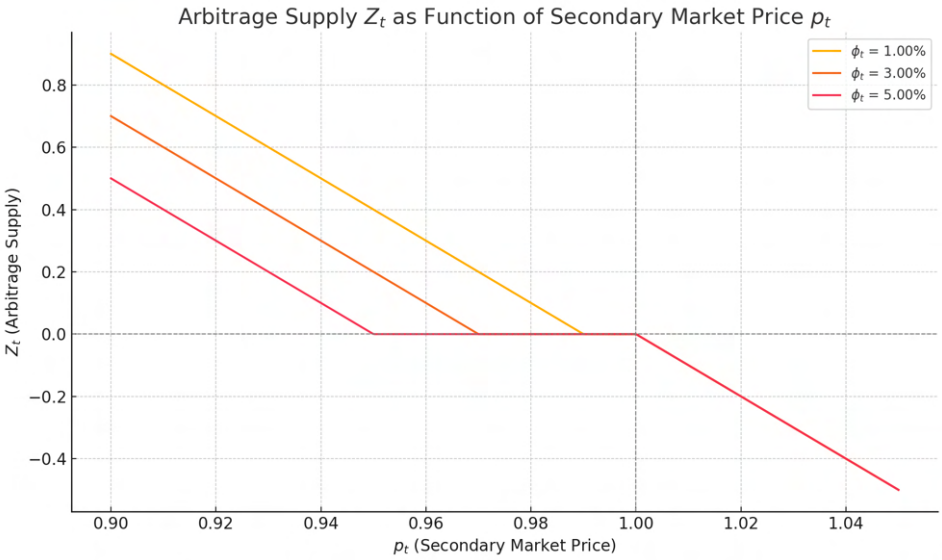


Figure 3: Arbitrage Supply vs. ϕ

The above figure plots relationship between secondary market price (p_t) and Arbitrage Supply (Z_t) for different liquidity fees ϕ. Source: Author's calculations.

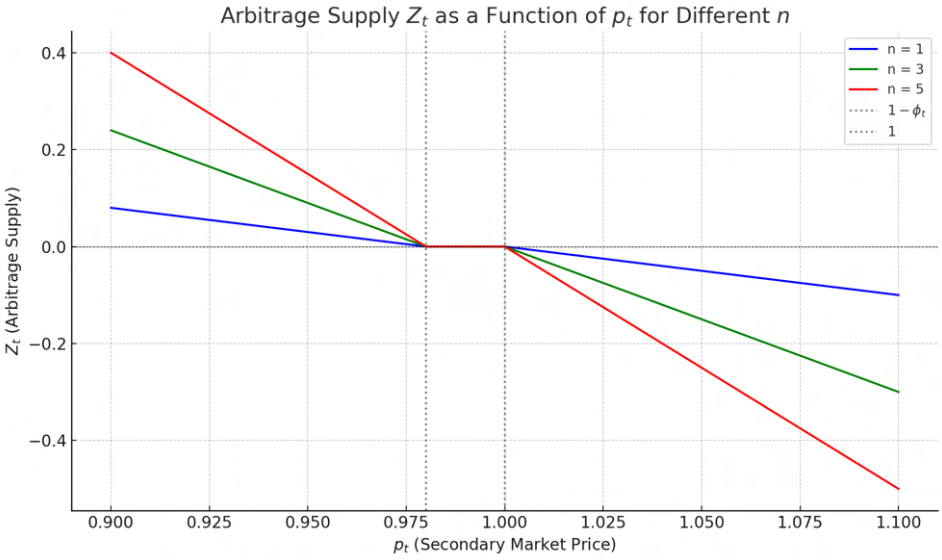


Figure 4: Arbitrage Supply vs. n

The above figure plots relationship between secondary market price (p_t) and Arbitrage Supply (Z_t) for different n. Source: Author's calculations.

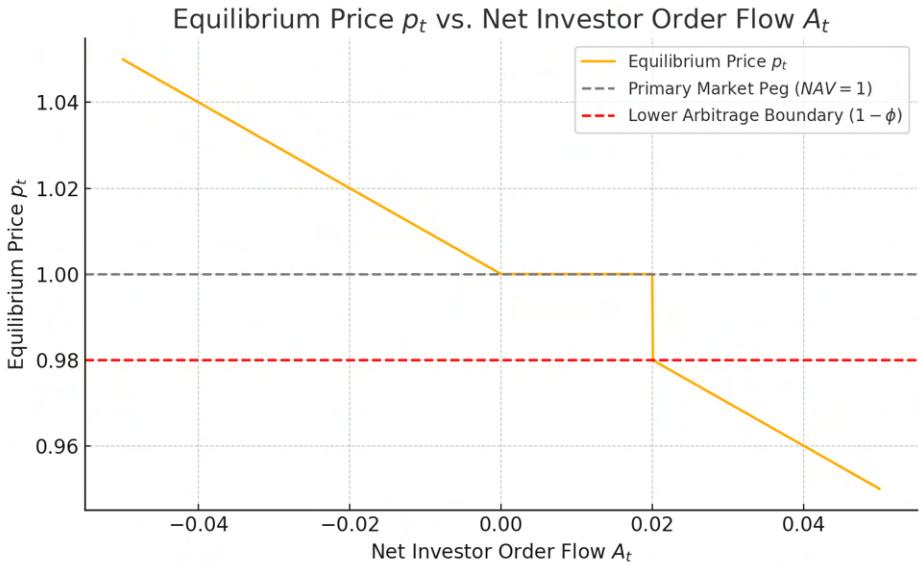


Figure 5: Dynamics of Price in the Secondary Market

The above figure plots relationship between secondary market price (p_t) and Net investor flow (A_t). Source: Author's calculations.

A1 Mathematical Appendix

A1.1 Derivation of the Cutoff Signal $x_t^*(z_t)$

Under simplifying assumptions, we derive an explicit formula for the cutoff signal $x_t^*(z_t)$, at which an investor is indifferent between redeeming and staying invested.

We assume,

1. Fund survival is driven by the condition $A_t \leq \theta_t$, where A_t is the fraction of redeeming investors and θ_t is a latent threshold.

2. Private signals x_i are normally distributed around θ_t:

$$x_i \sim \mathcal{N}(\theta_t, \sigma_x^2)$$

3. Investors redeem if their private signal falls below the cutoff:

$$x_{i,t} \leq x_t^*$$

A1.2 Redemption Share and Survival Belief

The fraction of investors redeeming is:

$$A_t = \Phi\left(\frac{x_t^* - \theta_t}{\sigma_x}\right) \tag{A1}$$

Investors expect a default if $A_t > \theta_t$. Hence, the survival probability is:

$$P_{\text{surv}}(x_t^*, \theta_t) = \mathbb{P}\left(\Phi\left(\frac{x_t^* - \theta_t}{\sigma_x}\right) \leq \theta_t\right) \tag{A2}$$

Step 1: Invert the Redemption Function

Define the inverse:

$$x_t^* = \theta_t + \sigma_x \Phi^{-1}(A_t) \tag{A3}$$

Survival occurs when $A_t \leq \theta_t$. Substituting:

$$\Phi\left(\frac{x_t^* - \theta_t}{\sigma_x}\right) \leq \theta_t \quad \Rightarrow \quad \frac{x_t^* - \theta_t}{\sigma_x} \leq \Phi^{-1}(\theta_t) \quad \Rightarrow \quad x_t^* \leq \theta_t + \sigma_x \Phi^{-1}(\theta_t)$$

Define the implied fundamental threshold $\theta^\dagger$ satisfying:

$$x_t^* = \theta^\dagger + \sigma_x \Phi^{-1}(\theta^\dagger) \tag{A4}$$

Then the survival probability becomes:

$$P_{\text{surv}}(x_t^*) = \mathbb{P}(\theta_t \geq \theta^\dagger) \tag{A5}$$

Step 2: Indifference Condition

Let the investor redeem if:

$$1 - c_i - \gamma \cdot \max\{0, L_{\min} - L_t\} \geq \sum_{s=t+1}^{T} \beta^{s-t} \cdot P_{\text{surv}}(x_t^*) \cdot \eta \tag{A6}$$

Under a one-period horizon ($T = t + 1$) and discount factor $\beta \approx 1$, this simplifies to:

$$1 - c_i - \gamma \cdot \max\{0, L_{\min} - L_t\} = P(\theta_t \geq \theta^\dagger) \cdot \eta \tag{A7}$$

Step 3: Solving for $\theta^\dagger$ and x_t^*

Define:

$$\tau = 1 - c_i - \gamma \cdot \max\{0, L_{\min} - L_t\}$$

Then:

$$\Phi\left(\frac{\theta_0 - \theta^\dagger}{\sigma_\theta}\right) = 1 - \frac{\tau}{\eta} \quad \Rightarrow \quad \theta^\dagger = \theta_0 - \sigma_\theta \cdot \Phi^{-1}\left(1 - \frac{\tau}{\eta}\right) \tag{A8}$$

Substitute back into Equation A4:

$$x_t^* = \theta^\dagger + \sigma_x \cdot \Phi^{-1}(\theta^\dagger) \tag{A9}$$

A1.3 Summary Formula

Combining Equations A8 and Equation A9, the closed-form expression for the cutoff becomes:

$$\begin{aligned} x_t^*(z_t) = \theta_0 - \sigma_\theta \cdot \Phi^{-1}\left(1 - \frac{1 - c_i - \gamma \cdot \max\{0, L_{\min} - L_t\}}{\eta}\right) \\ + \sigma_x \cdot \Phi^{-1}\left(\theta_0 - \sigma_\theta \cdot \Phi^{-1}\left(1 - \frac{1 - c_i - \gamma \cdot \max\{0, L_{\min} - L_t\}}{\eta}\right)\right) \end{aligned} \tag{A10}$$

This expression can be evaluated numerically for given parameters, and it characterizes the marginal investor's switching threshold as a function of fees, liquidity, signal precision, and beliefs.

Gluon W: A Cryptocurrency Stabilization Protocol

Joachim Zahnentferner
The Stable Order

Luca D'Angelo
zenGate Global
The Stable Order

Mohammad Shaheer
Carnegie Mellon University in Qatar
The Stable Order

Giselle Reis
Carnegie Mellon University in Qatar

Abstract

This paper introduces Gluon W, a novel stablecoin protocol inspired by nuclear physics and named after the particle responsible for the stability of matter in the universe. The key idea in Gluon W is to split (as in nuclear fission) an existing volatile asset into its stable and unstable components. These components can be merged back (as in nuclear fusion) into the original asset or transmuted into each other (as in nuclear beta decays). Various stability theorems are proven and their proofs are formally verified using the interactive proof assistant *Rocq*.

2012 ACM Subject Classification Applied computing → Economics

Keywords and phrases Blockchain, cryptocurrency, stablecoin, formal methods, interactive theorem prover

Acknowledgements We are grateful to the Sui Foundation and Cardano's Catalyst for grants that partially supported this research. We would like to thank: Kii, Luca D'Angelo, Sangeet, Hammerman and nkz for the implementation on Ergo; Krasavice Blasen for the Python Simulation and the audit of the Ergo implementation; Michael for testing the Ergo implementation. The main responsibilities of the authors were: Zahnentferner – invention and description of Gluon and supervision of the work on proving the stability theorems; Luca D'Angelo – proofs of the stability theorems; Mohammad Shaheer – formal verification of the proofs in Rocq; Giselle Reis – supervision and mentoring of the formal verification.

1 Introduction

For cryptocurrencies to actually be proper currencies, they need to perform well the functions of money. They must be capable of serving as a medium of exchange, unit of account and store of value. All these functions require stability of their value relative to the goods and services for which one might use them. However, traditional cryptocurrencies, such as BTC, have been plagued by volatility, since their monetary policies are rigid and unable to cope with fluctuations in demand. To address this volatility problem, the concept of *stablecoin* has arisen. Stablecoins aim to have a stable value in relation to another asset, typically (but not necessarily) a fiat currency, such as the USD.

The most popular stablecoins by market cap are fiat-backed: they are backed by roughly the same asset as the asset that their price is pegged to. For example, USD stablecoins (such as USDT and USDC) are backed by USD in a bank account or low-risk USD-denominated bonds. Some, but not necessarily all, holders of such stablecoins have a right to redeem their stablecoins for actual USD in their bank accounts. Such fiat-backed stablecoins, however, need to be centrally managed by entities whose reserves are opaque and whose business

models rely on the interest rate of the bonds in which they invest the reserves and thus depend on the very inflation of the supply of fiat currencies that was a major motivating factor for the invention and pursuit of cryptocurrencies. Furthermore, because their profit increases almost linearly with the amount of reserves controlled by these entities but their costs do not, there is a tendency for the entities issuing fiat-backed stablecoins with large market caps to outcompete entities issuing fiat-backed stablecoins with small market caps. This leads to further centralization of the control of vast amounts of fiat currency reserves. This, and the financial risks that it entails, should be concerning not only to those who are philosophically aligned with the decentralization principles of cryptocurrencies, but also for regulators who care about financial and economic stability.

In the past decade, although they are currently not as popular as centralized fiat-backed stablecoins, many decentralized stablecoin protocols have been proposed, implemented, deployed and used. They can be can be classified in a few different categories [9], such as: seigniorage-based [5] (e.g. Terra), crypto-collaterized (e.g. DAI [1]) and crypto-backed (e.g. Djed [10, 11]). While seigniorage-based stablecoin protocols have proven to be fatally flawed with the collapse of Terra, crypto-collateralized and crypto-backed stablecoins have proven to be able to maintain their stability despite severe shocks in the price of their collateral and reserve assets.

Crypto-backed stablecoins, for instance, function similarly to fiat-backed stablecoins in the sense that the stablecoin holder also has a right of redemption. But the stablecoins can be redeemed not for the fiat currency for which the stablecoin is pegged to, but for an amount of a cryptocurrency held by a smart contract as a reserve asset. Since cryptocurrencies are volatile, the reserve ratio is kept at levels significantly higher than 100%, so that full-value redemption remains possible even after significant drops in the price of the reserve asset.

The Gluon W stablecoin protocol belongs to an entirely new category of stablecoin protocols. Unlike crypto-backed stablecoins, which issue a new currency that is backed by a cryptocurrency, Gluon W allows users to take the existing cryptocurrency and split it into two sub-assets: one of them is stable w.r.t. a target peg asset and the other one is even more volatile than the base cryptocurrency.

This splitting mechanism is inspired by the nuclear physics reaction of *fission*. Taking this physics analogy further, Gluon W allows users to merge the two sub-assets back together to obtain the original cryptocurrency, similarly to nuclear *fusion*; and one sub-asset may be transmuted into the other, similarly to nuclear *beta decays* of protons to neutrons and neutrons to protons.

The Gluon W protocol can also be seen as a dual of liquidity pools. Whereas in a liquidity pool, one deposits two assets into a pool and obtains an LP token that represents a share of the pool, in Gluon W one deposits the base cryptocurrency and receives two (sub-)assets that represent a share of the pool. Redeeming the LP token gives the user his/her an amount of the two assets back. Likewise, conversely, by fusioning the two (sub-)assets the user gets an amount of base cryptocurrency back.

As in the case of liquidity pools, which have mechanisms to discourage large swaps between the two assets, Gluon W has mechanisms to discourage large beta decays of one sub-asset into the other. Unlike liquidity pools, though, the amounts of base cryptocurrency represented by each sub-asset depend also on an external oracle.

This paper has the following structure. In Section 2, the Gluon W protocol is precisely described. In Section 3, various theorems about Gluon W's stability are mathematically proven. In Section 4, the formal verification of the proofs of these theorems using the interactive proof assistant Rocq [4] is discussed. In Section 5, a comparison with the Djed

Stablecoin Protocol is made. In Section 6, current implementations of the protocol are described.

2 Protocol Description

2.1 Definitions

The essence of Gluon W is that, analogously to how an atom's nucleus is composed of protons and neutrons (known collectively as nucleons), a token, denoted by ☯, is composed of two sub-assets (named collectively as *tokeons*):

- ○: *stable tokeons* or *neutrons*, whose price is kept stable relative to a target price (e.g. a fiat currency or commodity).
- ●: *volatile tokeons* or *protons*, whose price is more volatile than ☯'s.

The protocol defines the rules of an autonomous *reactor* capable of four reactions:

1. **Fission** The user sends ☯ to the reactor and receives neutrons and protons.
2. **Fusion** The user sends neutrons and protons to the reactor and receives ☯.
3. **Beta decay** β^+ **(● ⇒ ○):** The user sends protons to the reactor and receives neutrons.
4. **Beta decay** β^- **(○ ⇒ ●):** The user sends neutrons to the reactor and receives protons.

The reactor's state is a triple $(R, S_\circ, S_\bullet)$ where:

- R is the number of ☯ that is currently fissioned, also called the *reserve*,
- $S_\circ$ is the number of neutrons in circulation,
- $S_\bullet$ is the number of protons in circulation.

In order to determine the quantities involved in each reaction, the protocol is parametrized by the following values, which are set at the time of deployment and never changed.

- q^*: Upper bound for fusion ratio.
  Constraint: $0 < q^* < 1$
- $\Phi_\downarrow$: Intuitively denotes the intrinsic cost for a *fission*.
  Constraint: $0 < \Phi_\downarrow < 1$
- $\Phi_\uparrow$: Intuitively denotes the intrinsic cost for a *fusion*.
  Constraint: $0 < \Phi_\uparrow < 1$
- Φ_β^x ($x \in \{0, 1\}$): y-intercept and slope of the linear function used to determine the cost of beta decays.
  Constraints: $0 < \Phi_\beta^x \leq 1$.
- T: A fixed length of time. The cost of a beta decay takes into account the volume of beta decays within a time window of this length.

Besides the constant parameters, the protocol also depends on a variable *target price* for neutrons, denoted $P_\circ^*$ and assumed to be given by an independent external oracle.

The *fusion ratio* determines what fraction of the reserve is backing neutrons. The *neutron price* is that amount of reserve per neutron, and the *proton price* is the amount of reserve per proton. These three important notions are defined by the following equations.

- **Fusion ratio**

$$q(R, S_\circ) = min\left(q^*, \frac{S_\circ P_\circ^*}{R}\right)$$

- **Neutron price**

$$P_\circ(R, S_\circ) = \frac{q(R, S_\circ)R}{S_\circ}$$

- **Proton price**

$$P_\bullet(R, S_\circ, S_\bullet) = \frac{(1 - q(R, S_\circ))R}{S_\bullet}$$

2.2 Beta decay fees

At a given time τ, the beta decay fees for β^+ ($\bullet \Rightarrow \circ$) and β^- ($\circ \Rightarrow \bullet$) are, respectively:

$$\varphi_{\beta^+}(\tau) = \min\left\{1, \Phi^0_\beta + \Phi^1_\beta \frac{\max\{\Delta(\tau), 0\}}{R}\right\}$$
$$\varphi_{\beta^-}(\tau) = \min\left\{1, \Phi^0_\beta + \Phi^1_\beta \frac{\max\{-\Delta(\tau), 0\}}{R}\right\}$$

$\Delta(\tau)$ is defined as:

$$\Delta(\tau) = \sum_{r \in \chi_+(\tau)} \nu(r) - \sum_{r \in \chi_-(\tau)} \nu(r)$$

Where $\chi_+(\tau)$ and $\chi_-(\tau)$ are the sets of beta decays β^+ and β^-, respectively, between times $(\tau - T)$ and τ, and $\nu(r)$ is the volume in ◕ of a beta decay reaction r. The volume of a β^+ reaction r^+ is defined as $\nu(r^+) = P_\bullet n_\bullet$, where $P_\bullet$ is the current proton price and $n_\bullet$ is the amount of protons being converted to neutrons. Analogously, the volume of a β^- reaction $r^-$is $\nu(r^-) = P_\circ n_\circ$.

In other words, $\Delta(\tau)$ is the balance of beta decay reactions in the time window up to τ.

Note in particular that, if the balance is positive, that is, there was a bigger volume of β^+ decays than β^- decays, then the fee φ_{β^-} is Φ^0_β, while the fee φ_{β^+} for the dual reaction is higher. The use of *max* ensures that the fee is never negative.

2.3 Reactions

The inputs (tokens or tokeons that the users provides) and outputs (tokens or tokeons that the user receives) of Gluon W's reactions are defined in Tables 1, 2 and 3. These three tables are mathematically equivalent. One can be transformed into the others by expanding definitions and performing algebraic simplifications. The three tables are shown here for convenience, since some of these equations are easier to implement than others, depending on the implementation approach taken, and also for the sake of providing additional insight into the protocol's behavior. For instance, Tables 2 and 3 make it clear that fission and fusion do not depend on the target price.

2.4 Example Calculation

For example, suppose the current reactor state is: $R = 1$, $S_\circ = 2$, and $S_\bullet = 1$. And suppose that the fission fee is 0.1. If a user sends 10 ◕ to the reactor, the user receives 18 neutrons and 9 protons, and the new reactor state will be $R = 11$, $S_\circ = 20$, and $S_\bullet = 10$.

Table 1 Gluon W's Reactions (v1)

Reaction	Inputs	Outputs
Fission	m ☯	$\frac{m(1-q(R,S_\circ))(1-\Phi_\downarrow)}{P_\bullet(R,S_\circ,S_\bullet)}$ • $\frac{m\ q(R,S_\circ)(1-\Phi_\downarrow)}{P_\circ(R,S_\circ)}$ ○
Fusion	$\frac{m(1-q(R,S_\circ))}{P_\bullet(R,S_\circ,S_\bullet)}$ • $\frac{m\ q(R,S_\circ)}{P_\circ(R,S_\circ)}$ ○	$m(1-\Phi_\uparrow)$ ☯
β^+	$\frac{m}{P_\bullet(R,S_\circ,S_\bullet)}$ •	$\frac{m(1-\varphi_{\beta+}(\tau))}{P_\circ(R,S_\circ)}$ ○
β^-	$\frac{m}{P_\circ(R,S_\circ)}$ ○	$\frac{m(1-\varphi_{\beta-}(\tau))}{P_\bullet(R,S_\circ,S_\bullet)}$ •

Table 2 Gluon W's Reactions (v2)

Reaction	Inputs	Outputs
Fission	m ☯	$m(1-\Phi_\downarrow)\frac{S_\bullet}{R}$ • $m(1-\Phi_\downarrow)\frac{S_\circ}{R}$ ○
Fusion	$m\frac{S_\bullet}{R}$ • $m\frac{S_\circ}{R}$ ○	$m(1-\Phi_\uparrow)$ ☯
β^+	m •	$m(1-\varphi_{\beta+}(\tau))\frac{(1-q(R,S_\circ))}{q(R,S_\circ)}\frac{S_\circ}{S_\bullet}$ ○
β^-	m ○	$m(1-\varphi_{\beta-}(\tau))\frac{q(R,S_\circ)}{(1-q(R,S_\circ))}\frac{S_\bullet}{S_\circ}$ •

Table 3 Gluon W's Reactions (v3)

Reaction	Inputs	Outputs
Fission	m ☯	$m(1-\Phi_\downarrow)\frac{S_\bullet}{R}$ • $m(1-\Phi_\downarrow)\frac{S_\circ}{R}$ ○
Fusion	$m\frac{S_\bullet}{R(1-\Phi_\uparrow)}$ • $m\frac{S_\circ}{R(1-\Phi_\uparrow)}$ ○	m ☯
β^+	m •	$m(1-\varphi_{\beta+}(\tau))\frac{(1-q(R,S_\circ))}{q(R,S_\circ)}\frac{S_\circ}{S_\bullet}$ ○
β^-	m ○	$m(1-\varphi_{\beta-}(\tau))\frac{q(R,S_\circ)}{(1-q(R,S_\circ))}\frac{S_\bullet}{S_\circ}$ •

3 Stability Properties

Gluon W enjoys various stability properties, as proved in this section. These properties are stated in terms of the auxiliary concepts defined below.

- **Liabilities**

$$L(R, S_\circ) = S_\circ P_\circ(R, S_\circ) = q(R, S_\circ)R$$

- **Reserve Ratio**:

$$r(R, S_\circ) = \frac{R}{L(R, S_\circ)} = \frac{1}{q(R, S_\circ)}$$

- **Equity**:

$$E(R, S_\circ) = R - L(R, S_\circ) = (1 - q(R, S_\circ))R$$

Note the intuitive nature of these auxiliary concepts. The liabilities are the portion of the reserve backing neutrons, while the equity is the portion of the reserve backing protons, and the reserve ratio indicates by what factor the reserve is larger than the liabilities.

In the proofs, we often omit the arguments of functions for brevity. For instance, we write q instead of $q(R, S_\circ)$.

The first stability property satisfied by Gluon W is that, in the normal reserve ratio range[1], users have no incentive to trade stablecoins outside the peg range in a secondary market. This is shown in Theorems 1 and 2.

▶ **Theorem 1** (Peg Maintenance - Upper Bound). *Assume that the neutron target price $P^*_\circ$ remains constant between the consecutive actions of fission and β^+ ($\bullet \Rightarrow \circ$), where the number of neutrons obtained is N.*

If a user u wants to sell a neutron in the secondary market for a price P such that $P > (1 + \Phi(\tau))P^*_\circ$, where $\Phi(\tau)$ is the effective fee for the operations of fission and then β^+, then there is no rational user u^* who would buy from u.

Furthermore, if $r(R, S_\circ) > \frac{1}{q^*}$, $|\Delta(\tau)| \ll R + m$, $P_\circ N \ll R$, then the effective fee is $\Phi(\tau) \approx \frac{r}{(1-\Phi_\downarrow)(1+(r-1)(1-\Phi^0_\beta))} - 1$.

Proof. Assume, for the sake of contradiction, that such a rational u^* exists. Then u^* would have two options: to buy a neutron from u for P; or to buy a neutron directly from the bank for P' where $P' = (1 + \Phi(\tau))P_\circ$. Note that the second option is available because the action of buying stable tokeons from the bank always holds. By definition of $P_\circ$, $P' < (1 + \Phi(\tau))P^*_\circ$. Therefore, buying directly from the bank for P' would have been less costly than buying from u for P, and would have been the preferred option for a rational user. Hence u^* is irrational.

Now for the effective fee. We define purchasing N neutrons as first performing a fission and then a β^+ using the protons from the fission. Thus, we must derive an equation for the following question: how many m ☯ do we need in order to purchase N neutrons? Let $P^\downarrow_\circ$ and $P^\downarrow_\bullet$ denote the neutron price and proton price after the fission, respectively.

The fission reaction of m ☯ results in (see Table 3):

[1] In practice, there may be factors that may restrict interaction of the user with the reactor, such as blockchain congestion and high blockchain transaction fees. Such factors would have an effect on the ability of the reactor to maintain the peg range in secondary markets.

$$\frac{m(1-q)(1-\Phi_{\downarrow})}{P_{\bullet}} \quad \bullet \text{ (protons)}$$
$$\frac{mq(1-\Phi_{\downarrow})}{P_{\circ}} \quad \circ \text{ (neutrons)}$$

The β^+ reaction transforms (see Table 1):

$$\frac{\tilde{m}}{P_{\bullet}^{\downarrow}} \bullet \quad \text{into} \quad \frac{\tilde{m}(1-\varphi_{\beta^+}(\tau))}{\tilde{P_{\circ}^{\downarrow}}} \circ$$

We can determine $\tilde{m}$ from the amount of protons resulting from the fission, assuming all of them will be used in β^+:

$$\frac{\tilde{m}}{P_{\bullet}^{\downarrow}} = \frac{m(1-q)(1-\Phi_{\downarrow})}{P_{\bullet}}$$
$$\tilde{m} = \frac{m(1-q)(1-\Phi_{\downarrow})P_{\bullet}^{\downarrow}}{P_{\bullet}}$$

After both reactions, we will have in total N neutrons:

$$N = \frac{mq(1-\Phi_{\downarrow})}{P_{\circ}} + \frac{\tilde{m}(1-\varphi_{\beta^+}(\tau))}{P_{\circ}^{\downarrow}}$$
$$N = \frac{mq(1-\Phi_{\downarrow})}{P_{\circ}} + \frac{m(1-q)(1-\Phi_{\downarrow})P_{\bullet}^{\downarrow}(1-\varphi_{\beta^+}(\tau))}{P_{\bullet}P_{\circ}^{\downarrow}}$$

And now we can write m as a function of N:

$$m = \frac{P_{\bullet}P_{\circ}^{\downarrow}P_{\circ}N}{q(1-\Phi_{\downarrow})P_{\bullet}P_{\circ}^{\downarrow} + (1-q)(1-\Phi_{\downarrow})(1-\varphi_{\beta^+}(\tau))P_{\bullet}^{\downarrow}P_{\circ}}$$

Thus, by setting $N = 1$ we obtain the equation for calculating how many ☯ are required for purchasing one neutron from the bank.

We can obtain the effective fee from the following:

$$m = (1+\Phi(\tau))NP_{\circ}$$
$$\Phi(\tau) = \frac{P_{\bullet}P_{\circ}^{\downarrow}}{q(1-\Phi_{\downarrow})P_{\bullet}P_{\circ}^{\downarrow} + (1-q)(1-\Phi_{\downarrow})(1-\varphi_{\beta^+}(\tau))P_{\bullet}^{\downarrow}P_{\circ}} - 1$$

Since we assume that the neutron target price remains constant between operations and that $r > \frac{1}{q^*}$ so that the peg is maintained, we have that $P_{\circ}^{\downarrow} = P_{\circ} = P_{\circ}^{*}$. Since we also assume that $|\Delta(\tau)| \ll R + m$ then the beta-decay fee will be constant so that $\varphi_{\beta^+}(\tau) \approx \Phi_{\beta}^{0}$. Similarly, since we assume that $P_{\circ}N \ll R$ then $r^{\downarrow} = \alpha r \approx r$, where $\alpha = \frac{R+m}{R+(1-\Phi_{\downarrow})m}$. By applying these assumptions we obtain the following simplified effective fee:

$$\begin{aligned}
\Phi(\tau) &= \frac{\frac{(1-q)R}{S_\bullet}}{q(1-\Phi_\downarrow)(1-q)\frac{R}{S_\bullet} + (1-q)(1-\Phi_\downarrow)(1-\Phi^0_\beta)\frac{(1-q^\downarrow)R\alpha}{S_\bullet}} - 1 \\
&= \frac{1}{(1-\Phi_\downarrow)(q+\alpha(1-q^\downarrow)(1-\Phi^0_\beta))} - 1 \\
&= \frac{1}{(1-\Phi_\downarrow)(q+\alpha(1-\frac{q}{\alpha})(1-\Phi^0_\beta))} - 1 \\
&= \frac{1}{(1-\Phi_\downarrow)(q+q(\alpha r-1)(1-\Phi^0_\beta))} - 1 \\
&\approx \frac{r}{(1-\Phi_\downarrow)(1+(r-1)(1-\Phi^0_\beta))} - 1
\end{aligned}$$

◀

▶ **Theorem 2** (Peg Maintenance - Lower Bound). *Assume that the neutron target price $P^*_\circ$ remains constant between the consecutive actions of β^- and fusion.*

If $r(R, S_\circ) > \frac{1}{q^*}$, $|\Delta(\tau)| \ll R$, and a user u wants to buy a neutron in the secondary market for a price P such that $P < (1-\Phi(\tau))P^*_\circ$, where $\Phi(\tau)$ is the effective fee for selling a neutron after the operations of partial β^- and fusion, then there is no rational user u^* who would sell to u.

Furthermore, the effective fee will be $\Phi(\tau) = 1 - \frac{(1-\Phi_\uparrow)(1-\Phi^0_\beta)}{(1-q\Phi^0_\beta)}$.

Proof. Assume, for the sake of contradiction, that such a rational u^* exists. Then u^* would have had two options: to sell a neutron to u for P; or to sell a neutron directly to the bank for P' where $P' = (1-\Phi(\tau))P_\circ$. By definition of $P_\circ$ and the fact that $r(R, S_\circ) > \frac{1}{q^*}$, $P' = (1-\Phi(\tau))P^*_\circ$. Therefore, selling directly to the bank for P' would have been more profitable than selling to u for P, and would have been the preferred option for a rational user. Hence u^* is irrational.

We define selling N neutrons in order to receive m ☯ as first performing a partial β^-, to convert the necessary portion γN of our initial neutrons into protons, and then performing a fusion with these new protons and the remaining $(1-\gamma)N$ neutrons. Thus, we must derive an equation for the following question: given N neutrons, how many m ☯ do we get? Using Table 3, we have the following:

$$\begin{aligned}
\frac{\tilde{m}q^-}{P^-_\circ}\circ + \frac{\tilde{m}(1-q^-)}{P^-_\bullet}\bullet &= \tilde{m}(1-\Phi_\uparrow)☯ = m☯ \\
\frac{\gamma m_{\beta^-}}{P_\circ}\circ &= \frac{\gamma m_{\beta^-}(1-\varphi_{\beta^-}(\tau))}{P_\bullet}\bullet = \beta^-(\gamma N)\bullet
\end{aligned}$$

q^-, $P^-_\circ$, and $P^-_\bullet$ are the fusion ratio, neutron price, and proton price after the β^- reaction, respectively. To actually obtain the equation for m and γ we need to expand the terms in the equation according to their definition and use the simplified form. Accordingly, we now obtain the following results:

$$\begin{aligned}
(1-\gamma)N = \frac{\tilde{m}S^-_\circ}{R} \implies \tilde{m} &= \frac{(1-\gamma)NR}{S^-_\circ} \\
\implies m = (1-\Phi_\uparrow)\tilde{m} &= \frac{(1-\Phi_\uparrow)(1-\gamma)NR}{S_\circ - \gamma N}
\end{aligned}$$

We can now obtain the effective fee by doing the following:

$$
\begin{aligned}
m &= \frac{(1-\Phi_{\uparrow})(1-\gamma)NR}{(S_{\circ}-\gamma N)} \\
&= \frac{(1-\Phi_{\uparrow})(1-\gamma)NR}{(1-\frac{\gamma N}{S_{\circ}})S_{\circ}}\frac{q}{q} \\
&= \frac{(1-\Phi_{\uparrow})(1-\gamma)NP_{\circ}}{q(1-\frac{\gamma N}{S_{\circ}})} \\
&= (1-\Phi(\tau))NP_{\circ} \\
&\implies \Phi(\tau) = 1 - \frac{(1-\Phi_{\uparrow})(1-\gamma)}{q(1-\frac{\gamma N}{S_{\circ}})}
\end{aligned}
$$

We must now derive equations for γ from the following:

$$
\begin{aligned}
\beta^{-}(\gamma N) &= \frac{\gamma N(1-\varphi_{\beta^{-}}(\tau))qS_{\bullet}}{(1-q)S_{\circ}} \\
\beta^{-}(\gamma N) &= \frac{\tilde{m}S_{\bullet}^{-}}{R} = \frac{(1-\gamma)NS_{\bullet}^{-}}{S_{\circ}^{-}} \\
S_{\bullet}^{-} &= S_{\bullet} + \beta^{-}(\gamma N) \\
S_{\circ}^{-} &= S_{\circ} - \gamma N
\end{aligned}
$$

Since $|\Delta(\tau)| \ll R$ then $\varphi_{\beta^{-}}(\tau) \approx \Phi_{\beta}^{0}$

$$
\begin{aligned}
\beta^{-}(\gamma N) &= \gamma N\beta^{-}(1) \\
\gamma N\beta^{-}(1) &= \frac{(1-\gamma)N(S_{\bullet}+\gamma N\beta^{-}(1))}{(S_{\circ}-\gamma N)} \\
\gamma S_{\circ}\beta^{-}(1) - \gamma^{2}N\beta^{-}(1) &= S_{\bullet} + \gamma N\beta^{-}(1) - \gamma S_{\bullet} - \gamma^{2}N\beta^{-}(1) \\
\gamma S_{\circ}\beta^{-}(1) - \gamma N\beta^{-}(1) + \gamma S_{\bullet} &= S_{\bullet} \\
\gamma &= \frac{S_{\bullet}}{S_{\circ}\beta^{-}(1) - N\beta^{-}(1) + S_{\bullet}} \\
\gamma &= \frac{S_{\bullet}}{\beta^{-}(S_{\circ}-N) + S_{\bullet}}
\end{aligned}
$$

We can then insert the derived equation for γ into the equation for $\Phi(\tau)$, where once we expand and simplify terms, we obtain the following equation for the effective-fee that is independent of the amount of neutrons:

$$
\Phi(\tau) = 1 - \frac{(1-\Phi_{\uparrow})(1-\Phi_{\beta}^{0})}{(1-q\Phi_{\beta}^{0})}
$$

◀

The second stability property is that stablecoins remain pegged despite market crashes up to a magnitude that depends on the reserve ratio.

▶ **Theorem 3** (Peg Robustness during Market Crashes). *If $S_\circ > 0$, $r > \frac{1}{q^*}$ is the current reserve ratio, x and x' are the exchange rates before and after the crash, then the bank can tolerate a ☯ price crash of $\frac{r-\frac{1}{q^*}}{r}$ and the new stablecoin price would still be $P_\circ^*$.*

Proof. By the definition of $P_\circ$, the peg is maintained as long as $P_\circ^* \leq \frac{q^* R}{S_\circ}$. By the definition of $P_\circ^*$, reserve ratio and liabilities, this inequation can be simplified to $x' \leq q^* r x$. Since x and x' are the exchange rates of 1 unit of pegged currency in BCs, the exchange rates of 1 BC in pegged currency are $y = \frac{1}{x}$ and $y' = \frac{1}{x'}$. The inequation can then be rewritten as $y' \geq \frac{y}{q^* r}$. Therefore, $\frac{y-y'}{y} \leq \frac{r-\frac{1}{q^*}}{r}$. ◀

Another crucial property is that the reactor never becomes insolvent, which is a condition defined by having negative equity.

▶ **Theorem 4** (No Insolvency). *In all reactor states and for any exchange rate, $E(R, S_\circ) \geq 0$.*

Proof. Since $q \leq q^* < 1 \Rightarrow E(R, S_\circ) = (1-q)R \geq 0$. ◀

The following theorem shows that, under the conditions of the theorem, the equity per proton increases after every operation due to collected fees. Note that this is one of the main incentives to hold protons.

▶ **Theorem 5** (Monotonically Increasing Equity per Reservecoin). *Assuming that the neutron target price, $P_\circ^*$, remains constant between actions, $R, S_\circ, S_\bullet, > 0$, and that in the depeg state a user does not transmute neutrons to protons, for every action a, $\frac{E(R^a, S_\circ^a)}{S_\bullet^a} \geq \frac{E(R, S_\circ)}{S_\bullet}$, where R^a, $S_\circ^a$, $S_\bullet^a > 0$ are respectively, the reserve, number of stable tokeons, and number of volatile tokeons after action a.*

Proof. See Appendix A for the proof. ◀

4 Formal Verification

To obtain stronger formal guarantees and uncover potential issues in the informal proofs, we formalized the Gluon W protocol using the Rocq theorem prover. Rocq is an interactive theorem prover based on the Calculus of Inductive Constructions. We chose Rocq over other theorem provers such as Isabelle [3] or Lean [2] due to its ease of use, our familiarity with it, and the availability of useful libraries.

4.1 Design

As with any formal verification endeavor, it was critical to choose the right representation. Since the informal proofs involved substantial algebraic manipulation, we had to decide how to represent numbers in a way that aligned closely with the protocol definition while easing the formalization. We initially experimented with the Rocq standard library for floats [6] and rational numbers [7], but ultimately decided against this approach, as we would have to manually prove many basic algebraic facts and inequalities to make the libraries useful. Although using the float library might make sense when formalizing a protocol implementation in a language like Solidity, it proved inefficient for theorem proving. In the end, we chose to use the Rocq standard library for real numbers [8]. It offers the most expressive numerical representation, encompassing both rational and irrational numbers, and includes a rich set of lemmas that simplify algebraic manipulations and facilitate formalization.

Defined below are some of the relevant datatypes we used for the formalization. In the formalization, we make use of the word *Coin* instead of *Token*, or *Tokeon*.

```
Definition StableCoins : Type := R.
Definition VolatileCoins : Type := R.
Definition BaseCoins : Type := R.
Definition ExchangeRate : Type := R.
Definition TimePeriod : Type := nat.
Definition Timestamp : Type := nat.
Definition QStar : Type := {x : R | 0 < x < 1}.
Definition FissionFee : Type := {x : R | 0 < x < 1}.
Definition FusionFee : Type := {x : R | 0 < x < 1}.
Definition BetaDecayFee0 : Type := {x : R | 0 < x}.
Definition BetaDecayFee1 : Type := {x : R | 0 < x}.

Record ReactorState :=
{
    baseCoins : BaseCoins;
    stableCoins : StableCoins;
    volatileCoins : VolatileCoins;
}.

Record StableCoinState :=
{
    reactorState : ReactorState;
    exchangeRate : ExchangeRate;
}.

Inductive Reaction : Type :=
| FissionReaction (baseCoins : BaseCoins)
| FusionReaction (baseCoins : BaseCoins)
| BetaDecayPosReaction (volatileCoins : VolatileCoins)
| BetaDecayNegReaction (stableCoins : StableCoins).

Definition Trace : Type := list (StableCoinState * Timestamp * Reaction).

Record State :=
{
    stableCoinState : StableCoinState;
    reactions : Trace;
}.

Inductive Action : Type :=
| BuyStableCoin
| SellStableCoin.
```

The protocol parameters q^*, $\Phi_\downarrow$, $\Phi_\uparrow$, Φ^0_β, Φ^1_β, and T are expressed as `QStar`, `FissionFee`, `FusionFee`, `BetaDecayFee0`, `BetaDecayFee1`, and `TimePeriod` respectively. Since most protocol parameters are bounded real numbers, we define them using Rocq's *Sigma Type*, which embeds the bounds as part of the type. This allows us to conveniently use the bounds in proofs via the `destruct` tactic. The record `ReactorState` keeps track of the number of `baseCoins`, `stableCoins`, and `volatileCoins` at any point in time and these fields mirror the variables R, $S_\circ$ and $S_\bullet$ defined above.

The record `StableCoinState` keeps track of the `reactorState`, and the `exchangeRate`. The field `exchangeRate` corresponds to $P^*_\circ$ defined above. The record `State` is composed of the `stableCoinState` and the `trace`. The `trace` is a history of all the reactions that occurred ordered from the most recent to least recent. Every entry in the list is a tuple of the `StableCoinState` before the reaction, the `Timestamp` when the reaction took place and the `Reaction` itself. The `trace` is useful for calculating $\Delta(\tau)$ in order to obtain the $\varphi_{\beta^+}(\tau)$ and $\varphi_{\beta^-}(\tau)$. We could have defined one record called the `State` that keeps track of everything but we adopted a composite structure to keep things organized and more importantly so the type `Trace` can refer to the `StableCoinState`.

The Gluon W protocol assumes that R, $S_\circ$, $S_\bullet$, and $P^*_\circ$ are always positive. These assumptions are formalized as `Axiom`s and used throughout the proofs. While we initially encoded these facts in the types `BaseCoins`, `VolatileCoins`, `StableCoins`, and `ExchangeRate` using *Sigma Types*, this approach complicated function definitions by requiring explicit proofs for each return value. The Gluon W protocol definitions are formalized as functions analogous to their definitions above. In the formalization, we also define functions like `execute` that take in a `state : State`, `reaction : Reaction`, and `timestamp : Timestamp` as input and returns a new `State` as output as a result of executing the reaction. This function proves to be useful in the formalization of several theorems. We also define a function called `is_valid_state` that takes in a `state : State` as input and returns a `Prop` expressing the correctness of the record `State`.

4.2 Formalization of Theorems

The process of formalizing the theorems proved to be extremely fruitful. Apart from the obvious benefit of obtaining stronger guarantees, it helped us recognize issues in the informal proofs and the protocol itself and make adjustments accordingly. While working on the formal proofs, we made sure to replicate the informal proofs as closely as possible while striving for simplicity and clarity.

4.2.1 Theorem 1

The formal statement of the theorem is stated as follows:

```
Theorem peg_maintenance_upper_bound :
forall
    (timestamp : nat)
    (m : R)
    (s1 s2 : State)
    (offer : Offer)
    (betaDecayPosFeeVal : R)
    (gamma : R)
    (k rr fissFee b0 effFee primaryMarketOffer
    secondaryMarketOffer e0 e1 : R),
(k > 0) → (m > 0) →
(s2 = execute (s1) (FissionEvent (m) (timestamp)) →
(e0 = target_price (s1.(stableCoinState))) →
(e1 = target_price (s2.(stableCoinState))) →
(betaDecayPosFeeVal = k * extract_value (betaDecayFee0)) →
rr = reserve_ratio (s1.(stableCoinState)) →
fissFee = extract_value (fissionFee) →
b0 = extract_value (betaDecayFee0) →
effFee = get_effective_fee (timestamp) (s1) (s2) (gamma)
(betaDecayPosFeeVal) (BuyStableCoin) →
offer.(action) = SellStableCoin →
primaryMarketOffer = get_primary_market_offer (timestamp) (s1)
(s2) (gamma) (betaDecayPosFeeVal) (BuyStableCoin) →
secondaryMarketOffer = offer.(value) →
(/extract_value (qStar)) < rr →
((1 + effFee) * target_price (s1.(stableCoinState))) <
(offer.(value)) →
(effFee >= 0) →
stablecoin_price (s1.(stableCoinState)) <> 0 →
betaDecayPosFeeVal < 1 →
e0 = e1 →
get_rational_choice (BuyStableCoin) (primaryMarketOffer)
(secondaryMarketOffer) = Primary
∧
effFee < ((rr) / ((1 - fissFee) * (1 + ((rr - 1) * (1 - (k * b0)))))) - 1.
```

The formal proof for this theorem involved two major goals. To prove the first goal that no rational user u^* would buy a stable tokeon from the secondary market for a price higher than what he/she is getting in the primary market, we made use of a proof by contradiction mirroring the informal proof. We defined a function called `get_rational_choice` that returns the rational choice based on whether $u*$ is either buying or selling a stable tokeon.

The second part of the informal proof derives an expression for m which gives us the number of base coins required to obtain N stable tokeons. The proof then uses this expression to obtain an expression for $\Phi(\tau)$ and performs algebraic simplifications based on the assumptions.

The formal proof reflects a similar structure and we defined functions

`base_coins_for_n_stable_coins`, and `get_effective_fee` that reflect m, and $\Phi(\tau)$ respectively. We use the function `get_effective_fee` to obtain the $\Phi(\tau)$ for buying a stable tokeon and perform algebraic manipulations to derive an upper-bound. Rather than deriving the value m from the protocol reactions, we make use of an elegant shortcut and prove the correctness of the function `base_coins_for_n_stable_coins`. We did so by defining another function called `stablecoins_from_m_basecoins` that returns the stable tokeons obtained from performing *Fission* on the base coins and then *Beta Decay +* on the volatile tokeons through the reactor equations. We proved that calling `stablecoins_from_m_basecoins` on the output of `base_coins_for_n_stable_coins` gives us the same stable tokeons we required initially.

Formalizing this theorem revealed several subtle issues. First, we identified missing terms in the denominator when computing the value of m. In the original proof of Theorem 1, we incorrectly assumed that the price of the volatile token remains unchanged before and after the fission reaction—an assumption invalidated by Theorem 5, which shows that each reaction increases the equity per volatile token, i.e., its price. When deriving the value m in the informal proof we incorrectly assumed that the final expression is completely independent of m. This assumption is wrong as $\varphi_{\beta^+}(\tau)$ depends on $\Delta(\tau)$ that takes into account the volume of the beta decay reaction that is yet to happen. This makes the $\varphi_{\beta^+}(\tau)$ depend on m as m ultimately determines how many volatile tokeons we get that would influence the value $\Delta\tau$. Obtaining an expression for m that is independent of m proved to be hard so we had to make assumptions about the value of the value of $\varphi_{\beta^+}(\tau)$. In the informal proof we assume that $|\Delta(\tau)| << R + m$ which makes $\varphi_{\beta^+}(\tau) \approx \Phi^0_\beta$ In the formalization we assume that $\varphi_{\beta^+}(\tau) = k * \Phi^0_\beta$ where $k \in R$ and $k > 0$. The informal proof of Theorem 1 differs slightly from the formalized proof when it comes to the effective fee. In the informal proof, an approximation is derived for the effective whereas in the formalization an upper bound is derived as its tricky to formalize the notion of approximately equal. By removing the term α since $\alpha \approx 1$, the informal proof does derive an upper bound but the actual value is very close to the upper bound hence the $\approx$ symbol is used.

4.2.2 Theorem 2

The formal statement of the theorem is stated as follows:

```
Theorem peg_maintenance_lower_bound :
forall
    (timestamp : nat)
    (s1 s2 : State)
    (offer : Offer)
    (primaryMarketOffer secondaryMarketOffer sC e0 e1 b0 rr k
    betaDecayNegFeeVal effFee gamma q fusFee s_0 p_s_0 : R),
(s2 = execute (s1) (BetaDecayNegEvent (sC)) (timestamp)) →
(q = fusion_ratio (s1.(stableCoinState))) →
(fusFee = extract_value (fusionFee)) →
```

```
(s_0 = get_stablecoins (s1.(stableCoinState).(reactorState))) →
(s_0 > 1) →
(p_s_0 = stablecoin_price (s1.(stableCoinState))) →
(e0 = target_price (s1.(stableCoinState))) →
(e1 = target_price (s2.(stableCoinState))) →
(e0 = e1) →
(b0 = extract_value (betaDecayFee0)) →
(rr = reserve_ratio (s1.(stableCoinState))) →
(rr > /extract_value (qStar)) →
(k > 0) →
(betaDecayNegFeeVal = k * b0) →
(betaDecayNegFeeVal < 1) →
(effFee = get_effective_fee (timestamp) (s1) (s2) (gamma)
(betaDecayNegFeeVal) (SellStableCoin)) →
(offer.(action) = BuyStableCoin) →
(primaryMarketOffer = get_primary_market_offer (timestamp) (s1)
(s2) (gamma)
(betaDecayNegFeeVal) (SellStableCoin)) →
(secondaryMarketOffer = offer.(value)) →
(((1 - effFee) * target_price (s1.(stableCoinState))) >
(secondaryMarketOffer)) →
(gamma = ((1 - q) * s_0) / (((s_0 - 1) * (1 - betaDecayNegFeeVal) * q) +
((1 - q) * s_0))) →
(get_rational_choice (SellStableCoin) (primaryMarketOffer)
(secondaryMarketOffer) = Primary)
∧
(effFee = 1 - (((1 - fusFee) * (1 - betaDecayNegFeeVal))
/ (1 - (q * betaDecayNegFeeVal)))).
```

The formalization of Theorem 2 is similar to the formalization of Theorem 1 and the lessons learned from the formalization of Theorem 1 streamlined the process. The first part of the theorem statement that says no rational user u^* would sell a stable tokeon in the secondary market for a price lower than what he/she is getting in the primary market is proved using a proof by contradiction. For deriving the value of the effective fee, we make use of the function `get_effective_fee` to obtain an expression for the effective fee and then perform algebraic manipulations based on the theorem assumptions to derive the final value. In this formal proof, we derive an exact value for the effective fee as opposed to an upper bound derived in the formal proof of Theorem 1.

4.2.3 Theorems 3 and 4

The formal statement of the theorems is stated as follows:

```
Theorem peg_robustness_during_market_crashes :
forall
    (s1 s2 : State),
get_stablecoins (s1.(stableCoinState).(reactorState)) > 0 ∧
reserve_ratio (s1.(stableCoinState)) > (1 / extract_value (qStar)) ∧
target_price (s1.(stableCoinState)) > 0 ∧
target_price (s2.(stableCoinState)) > 0 ∧
(get_stablecoins (s1.(stableCoinState).(reactorState)) /
get_basecoins (s1) (stableCoinState).(reactorState))) =
(get_stablecoins (s2.(stableCoinState) (reactorState)) /
get_basecoins (s2.(stableCoinState).(reactorState))) ∧
base_coin_price_crash (s1) (s2) <= (reserve_ratio (s1.(stableCoinState)) -
(1 / extract_value (qStar))) / (reserve_ratio (s1.(stableCoinState))) →
stablecoin_price (s2.(stableCoinState)) =
target_price (s2.(stableCoinState)).
```

```
Theorem insolvency :
    forall (s : State),
equity (s.(stableCoinState)) >= 0.
```

The formal proofs of both these theorems were relatively straightforward and followed the same argument as the informal proofs.

4.2.4 Theorem 5

The formal statement of the theorem is stated as follows:

```
Theorem insolvency :
forall
    (s1 s2 : State)
    (e : Event)
    (t : Timestamp),
is_valid_state (s1) ∧
is_valid_state (s2) ∧
s2 = execute (s1) (e) (t) ∧
target_price (s1.(stableCoinState)) =
target_price (s2.(stableCoinState)) ∧
(is_depeg (s1.(stableCoinState)) → forall x, e <> BetaDecayNegEvent x) →
reservecoin_price (s2.(stableCoinState)) >=
reservecoin_price (s1.(stableCoinState)).
```

Formalizing this theorem took significant amount of time primarily because of the sheer number of cases to consider. In the first version of the informal proof, we were not considering the possibility that the value of the fusion ratio q might move from a pegged to a de-pegged state or vice-versa after performing the reaction. Working on the formal proof, uncovered this wrong assumption and we had to revise the proof to account for all possibilities which greatly increased the number of cases to consider. Since we had to account for all 4 reactions and each reaction having 4 sub-cases we had about 16 cases to consider. While working on one of these cases (the user performs a β^- reaction while the fusion ratio q is in a de-pegged state) we realized the theorem statement does not hold. To account for this, we had to modify the theorem statement.

4.3 General Remarks

Rocq's `Search` tactic proved to be quite useful when searching for relevant theorems and lemmas to apply. Most of the formal proofs are written in backward reasoning style where we apply theorems/lemmas whose conclusion is the goal we are trying to prove and then proceed to prove that the assumptions of the applied theorem/lemma also hold. We made extensive use of tactics like `lra`, `nra`, and `lia` to resolve easy goals. We also defined a `Module` called `HelperLemmas` where we proved various algebraic inequalities/equalities related to real numbers. These helper lemmas proved to be quite useful throughout the formalization.

5 Related Work

The conception of Gluon W took into account lessons learned from Djed [10, 11]. The main improvements in comparison to Djed are:

- In Djed users are forbidden to redeem reservecoins when the reserve ratio is below a threshold. Although this is beneficial to stablecoin holders, it is very undesirable for potential reservecoin buyers, since they may remain unable to redeem their reservecoins for an unbounded amount of time. In Gluon W, users are never forbidden to perform any of the four reactions. In particular, users can always redeem their protons by first transmutting part of them to neutrons and then fusioning their protons and neutrons back to base coins.

- In Djed, the minting and burning fees are fixed. Therefore, Djed can only tolerate oracle delays or manipulations up to a certain limit. To have a sufficiently high tolerance, the fees need to be high. But high fees are unattractive to users. In Gluon W, on the other hand, the beta decay fees depend on the volume of recent transactions and on the volume of the current transaction. This allows the fee to be small for small transactions in times of low volume, but increase in times of high volume to protect against large transactions that would attempt to exploit oracle delays or manipulations.

However, to achieve these improvements, some stability properties that were provable for Djed are not provable for Gluon W. They are:

- No Bank Runs for Stablecoins: this property is not provable for Gluon W because, due to the variable fee, neutron holders who redeeem their neutrons earlier would enjoy a lower fee. Therefore, it is rational to run to be early. Nevertheless, like Djed, apart from the fees, Gluon W still ensures that all neutron holders are treated equally.
- No Reserve Drainage: although in Gluon W it is still impossible to outrightly drain the reserve, it is possible, under certain conditions, to shift part of the portion of reserves backing neutrons (or protons) to the portion backing protons (or neutrons, respectively). This means that Gluon W may behave counterintuitively with respect to how much of reserves are backing neutrons and how much of reserves are backing protons when the reserve ratio is low.
- Bounded Dilution: this property required Djed to forbid minting of reservecoins when the reserve ratio was above a threshold. In Gluon W, a deliberate choice was made, for the sake of improved user experience, not to prohibit any action.

6 Implementations

The Gluon W protocol has been implemented in Ergoscript and deployed on the Ergo blockchain. It can be used in `https://gluon.gold`. This is the first ever decentralized gold-pegged stablecoin: the price of neutrons in this deployment is pegged to the price of 1g of gold.

The source code of the smart contracts can be inspected in `https://github.com/StabilityNexus/Gluon-Ergo-Contracts`. The source code of the web UI is available at `https://github.com/StabilityNexus/Gluon-Ergo-UI`. A typescript SDK to interact with the smart contracts can be used as described in `https://github.com/StabilityNexus/Gluon-Ergo-SDK`. A simulator of the protocol, written in Python, is available in `https://github.com/StabilityNexus/Gluon-Simulation`. And the formalization of the stability theorems and their proofs can be found in `https://github.com/StabilityNexus/Gluon-Formalization-Coq`.

7 Conclusions and Future Work

The Gluon W protocol is the first of a family of protocols whose main characteristic is the ability to split (*fission*) a volatile asset into a stable sub-asset and an even more volatile sub-asset and merge (*fusion*) them back. It is only the second stablecoin protocol, after Djed, to have stability theorems proven and formally verified. Besides its theoretical foundations, Gluon W is currently being battle-tested in practice, as the basis for the first decentralized gold-pegged stablecoin. In our current and future work, we are exploring variations of this protocol, particularly by looking at different options for Equation 1 and alternative approaches for beta

decays. The formalization could be improved as well. For the formal proof of Theorem 2, we assume that the expression for γ equal the one described in the informal Proof 2 rather than deriving this value from the equations of the reactions. The next step would be to define a function called `gamma` that returns the expression γ and prove its correctness similar to how we proved the correctness of the function `base_coins_for_n_stable_coins` in the formalization of Theorem 1. There are opportunities for reducing the redundancies in the proof scripts as well. While working on the formal proofs, we noticed patterns in the type of algebraic manipulations performed. The next step would be to identify these patterns and if they occur repeatedly, abstract them away either as helper lemmas or new tactics using `Ltac`.

References

1 The Maker Protocol: MakerDAO's Multi-Collateral Dai (MCD) System. [Online], 2017. https://makerdao.com/en/whitepaper/.

2 Leonardo de Moura and Sebastian Ullrich. The lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28: 28th International Conference on Automated Deduction, Virtual Event, July 12–15, 2021, Proceedings*, page 625–635, Berlin, Heidelberg, 2021. Springer-Verlag. doi:10.1007/978-3-030-79876-5_37.

3 Tobias Nipkow, Lawrence C. Paulson, and Markus Wenzel. *Isabelle/HOL – A Proof Assistant for Higher-Order Logic*, volume 2283 of *LNCS*. Springer, 2002.

4 Rocq. Rocq Theorem Prover. https://rocq-prover.org/, Retrieved in July/2025.

5 Robert Sams. A Note on Cryptocurrency Stabilisation: Seigniorage Shares. [Online], 2014. https://blog.bitmex.com/wp-content/uploads/2018/06/A-Note-on-Cryptocurrency-Stabilisation-Seigniorage-Shares.pdf.

6 The Rocq Development Team. *Rocq Standard Library: Module Stdlib.Floats.Floats*. Inria Rocquencourt, 2025. Provides machine-level floating-point types and operations. URL: https://rocq-prover.org/doc/V9.0.0/stdlib/Stdlib.Floats.Floats.html.

7 The Rocq Development Team. *Rocq Standard Library: Module Stdlib.QArith.QArith*. Inria Rocquencourt, 2025. Provides rational number arithmetic on the type `Q` (fractions of integers). URL: https://rocq-prover.org/doc/V9.0.0/stdlib/Stdlib.QArith.QArith.html.

8 The Rocq Development Team. *Rocq Standard Library: Module Stdlib.Reals.Reals*. Inria Rocquencourt, 2025. Provides the real number axiomatization and core analysis theorems on the type `R`. URL: https://rocq-prover.org/doc/V9.0.0/stdlib/Stdlib.Reals.Reals.html.

9 Bruno Woltzenlogel Paleo. *Stablecoin*, pages 1–5. Springer Berlin Heidelberg, Berlin, Heidelberg, 2019. doi:10.1007/978-3-642-27739-9_1671-1.

10 Joachim Zahnentferner, Dmytro Kaidalov, Jean-Frédéric Etienne, and Javier Díaz. Djed: A formally verified crypto-backed pegged algorithmic stablecoin. Cryptology ePrint Archive, Report 2021/1069, 2021. https://eprint.iacr.org/2021/1069.

11 Joachim Zahnentferner, Dmytro Kaidalov, Jean-Frédéric Etienne, and Javier Díaz. Djed: A formally verified crypto-backed pegged autonomous stablecoin. In *IEEE International Conference on Blockchain and Cryptocurrency*, 2023. https://icbc2023.ieee-icbc.org/program/icbc-2023-final-program.

A Proof of Theorem 5

▶ **Theorem 5** (Monotonically Increasing Equity per Reservecoin). *Assuming that the neutron target price, $P^*_\circ$, remains constant between actions, $R, S_\circ, S_\bullet, > 0$, and that in the depeg state a user does not transmute neutrons to protons, for every action a, $\frac{E(R^a, S^a_\circ)}{S^a_\bullet} \geq \frac{E(R, S_\circ)}{S_\bullet}$, where R^a, $S^a_\circ$, $S^a_\bullet > 0$ are respectively, the reserve, number of stable tokeons, and number of volatile tokeons after action a.*

Proof. Let $A = \{F^\downarrow, F^\uparrow, \beta^+, \beta^-\}$ be the set of four fundamental actions in the Gluon protocol: fission, fusion, beta-decay plus, and beta-decay minus, respectively. Since we assume that the neutron target price remains constant between actions, for each possible value of the fusion ratio, q, before and after the action, we show what the equity per volatile tokeon is when applying the corresponding action on an arbitrary reactor state $|R, S_\circ, S_\bullet\rangle \in \mathbb{R}^3_{>0}$.

- $a = F^\downarrow$
  - $F^\downarrow |R, S_\circ, S_\bullet\rangle = |R + F^\downarrow_{◕}, S_\circ + F^\downarrow_\circ, S_\bullet + F^\downarrow_\bullet\rangle$
  -

$$P^\downarrow_\bullet = \frac{E^\downarrow}{S^\downarrow_\bullet} \tag{1}$$
$$= \frac{(1-q^\downarrow)R^\downarrow}{S^\downarrow_\bullet} \tag{2}$$
$$= \frac{(1-q^\downarrow)(R + F^\downarrow_{◕})}{S_\bullet + F^\downarrow_\bullet} \tag{3}$$
$$= \frac{(1-q^\downarrow)(R+m)}{S_\bullet + \frac{(1-\Phi_\downarrow)mS_\bullet}{R}} \tag{4}$$
$$= (1-q^\downarrow)\frac{R}{S_\bullet}\frac{(1+\frac{m}{R})}{(1+\frac{(1-\Phi_\downarrow)m}{R})} \tag{5}$$
$$= \frac{(1-q^\downarrow)R\alpha}{S_\bullet}, \alpha > 1 \tag{6}$$

  - Case 1: $q = q^*, q^\downarrow = q^*$

$$= \frac{(1-q^*)R\alpha}{S_\bullet} \tag{7}$$
$$> \frac{E}{S_\bullet} \tag{8}$$
$$= P_\bullet \tag{9}$$

    Since $q^* < \frac{P^*_\circ S_\circ}{R\alpha} < \frac{P^*_\circ S_\circ}{R}$, this case is possible if $P^*_\circ > \frac{q^* R}{S_\circ}$.
  - Case 2: $q = q^*, q^\downarrow = \frac{P^*_\circ S^\downarrow_\circ}{R^\downarrow} = \frac{P^*_\circ S_\circ}{R\alpha} < q^*$

$$= \frac{(1-\frac{P^*_\circ S_\circ}{R\alpha})R\alpha}{S_\bullet} \tag{10}$$
$$> \frac{(1-q^*)R\alpha}{S_\bullet} \tag{11}$$
$$> \frac{E}{S_\bullet} \tag{12}$$
$$= P_\bullet \tag{13}$$

Since $\frac{P_\circ^* S_\circ}{R\alpha} < q^* < \frac{P_\circ^* S_\circ}{R}$, this case is possible if $\alpha > \frac{P_\circ^* S_\circ}{q^* R} > 1$.

- Case 3: $q = \frac{P_\circ^* S_\circ}{R}, q^\downarrow = \frac{P_\circ^* S_\circ^\downarrow}{R^\downarrow} = \frac{P_\circ^* S_\circ}{R\alpha} < q$

$$= \frac{(1 - \frac{P_\circ^* S_\circ}{R\alpha})R\alpha}{S_\bullet} \tag{14}$$
$$= \frac{(1 - \frac{q}{\alpha})R\alpha}{S_\bullet} \tag{15}$$
$$> \frac{(1 - q)R\alpha}{S_\bullet} \tag{16}$$
$$> \frac{E}{S_\bullet} \tag{17}$$
$$= P_\bullet \tag{18}$$

This case is possible by definition since $\alpha > 1$.

- Case 4: $q = \frac{P_\circ^* S_\circ}{R}, q^\downarrow = q^* < \frac{P_\circ^* S_\circ^\uparrow}{R^\uparrow} = \frac{P_\circ^* S_\circ}{R\alpha} < q$
  Since we assume the neutron target price remains constant between actions, this case is not possible.

- $a = F^\uparrow$
  - $F^\uparrow |R, S_\circ, S_\bullet\rangle = |R - F_◕^\uparrow, S_\circ - F_\circ^\uparrow, S_\bullet - F_\bullet^\uparrow\rangle$
  -

$$P_\bullet^\uparrow = \frac{E^\uparrow}{S_\bullet^\uparrow} \tag{19}$$
$$= \frac{(1 - q^\uparrow)R^\uparrow}{S_\bullet^\uparrow} \tag{20}$$
$$= \frac{(1 - q^\uparrow)(R - F_◕^\uparrow)}{S_\bullet - F_\bullet^\uparrow} \tag{21}$$
$$= \frac{(1 - q^\uparrow)(R - (1 - \Phi_\uparrow)m)}{S_\bullet - \frac{mS_\bullet}{R}} \tag{22}$$
$$= (1 - q^\uparrow)\frac{R}{S_\bullet}\frac{(1 - \frac{(1-\Phi_\uparrow)m}{R})}{(1 - \frac{m}{R})} \tag{23}$$
$$= \frac{(1 - q^\uparrow)R\alpha}{S_\bullet}, \alpha > 1 \tag{24}$$

- Case 1: $q = q^*, q^\uparrow = q^*$

$$= \frac{(1 - q^*)R\alpha}{S_\bullet} \tag{25}$$
$$> \frac{E}{S_\bullet} \tag{26}$$
$$= P_\bullet \tag{27}$$

Since $q^* < \frac{P_\circ^* S_\circ}{R\alpha} < \frac{P_\circ^* S_\circ}{R}$, this case is possible if $P_\circ^* > \frac{q^* R}{S_\circ}$.

- Case 2: $q = q^*, q^\uparrow = \frac{P^*_\circ S^\uparrow_\circ}{R^\uparrow} = \frac{P^*_\circ S_\circ}{R\alpha} < q^*$

$$= \frac{(1 - \frac{P^*_\circ S_\circ}{R\alpha})R\alpha}{S_\bullet} \tag{28}$$
$$> \frac{(1 - q^*)R\alpha}{S_\bullet} \tag{29}$$
$$> \frac{E}{S_\bullet} \tag{30}$$
$$= P_\bullet \tag{31}$$

  Since $\frac{P^*_\circ S_\circ}{R\alpha} < q^* < \frac{P^*_\circ S_\circ}{R}$, this case is possible if $\alpha > \frac{P^*_\circ S_\circ}{q^* R} > 1$.

- Case 3: $q = \frac{P^*_\circ S_\circ}{R}, q^\uparrow = \frac{P^*_\circ S^\uparrow_\circ}{R^\uparrow} = \frac{P^*_\circ S_\circ}{R\alpha} < q$

$$= \frac{(1 - \frac{P^*_\circ S_\circ}{R\alpha})R\alpha}{S_\bullet} \tag{32}$$
$$= \frac{(1 - \frac{q}{\alpha})R\alpha}{S_\bullet} \tag{33}$$
$$> \frac{(1 - q)R\alpha}{S_\bullet} \tag{34}$$
$$> \frac{E}{S_\bullet} \tag{35}$$
$$= P_\bullet \tag{36}$$

  This case is possible by definition since $\alpha > 1$.

- Case 4: $q = \frac{P^*_\circ S_\circ}{R}, q^\uparrow = q^* < \frac{P^*_\circ S^\uparrow_\circ}{R^\uparrow} = \frac{P^*_\circ S_\circ}{R\alpha} < q$
  Since we assume the neutron target price remains constant between actions, this case is not possible.

- $a = \beta^+$
  - $\beta^+ \left| R, S_\circ, S_\bullet \right\rangle = \left| R, S_\circ + \beta^+_\circ, S_\bullet - \beta^+_\bullet \right\rangle$
  -

$$P^+_\bullet = \frac{E^+}{S^+_\bullet} \tag{37}$$
$$= \frac{(1 - q^+)R^+}{S^+_\bullet} \tag{38}$$
$$= \frac{(1 - q^+)R}{S_\bullet - \beta^+_\bullet} \tag{39}$$
$$= \frac{(1 - q^+)R}{S_\bullet - m} \tag{40}$$
$$= (1 - q^+)\frac{R}{S_\bullet \left(1 - \frac{m}{S_\bullet}\right)} \tag{41}$$
$$= \frac{(1 - q^+)R\alpha}{S_\bullet}, \alpha > 1 \tag{42}$$

- Case 1: $q = q^*, q^+ = q^*$

$$= \frac{(1-q^*)R\alpha}{S_\bullet} \tag{43}$$
$$> \frac{E}{S_\bullet} \tag{44}$$
$$= P_\bullet \tag{45}$$

  Since $q^* < \frac{P_\circ^* S_\circ}{R} < \frac{P_\circ^* S_\circ \tilde{\alpha}}{R}$, this case is possible if $P_\circ^* > \frac{q^* R}{S_\circ}$, where $\tilde{\alpha} = (1 + \frac{m(1-\varphi_{\beta^+}(\tau))(1-q)}{qS_\bullet}) > 1$.
- Case 2: $q = q^*, q^+ = \frac{P_\circ^* S_\circ^+}{R^+} = \frac{P_\circ^* S_\circ \tilde{\alpha}}{R} < q^*$
  Since we assume the neutron target price remains constant between actions, this case is not possible.
- Case 3: $q = \frac{P_\circ^* S_\circ}{R}, q^+ = \frac{P_\circ^* S_\circ^+}{R^+} = \frac{P_\circ^* S_\circ \tilde{\alpha}}{R} < q^*$

$$= \frac{(1 - \frac{P_\circ^* S_\circ \tilde{\alpha}}{R})R\alpha}{S_\bullet} \tag{46}$$
$$= \frac{(1 - q\tilde{\alpha})R}{S_\bullet} \tag{47}$$
$$= \frac{(1 - q(1 + \frac{m(1-\varphi_{\beta^+}(\tau))(1-q)}{qS_\bullet}))R\alpha}{S_\bullet} \tag{48}$$
$$= \frac{((1-q) - \frac{m(1\varphi_{\beta^+}(\tau))(1-q)}{S_\bullet})R\alpha}{S_\bullet} \tag{49}$$
$$= \frac{(1-q)R}{S_\bullet} \frac{(1 - \frac{m(1-\varphi_{\beta^+}(\tau))}{S_\bullet})}{(1 - \frac{m}{S_\bullet})} \tag{50}$$
$$> \frac{E}{S_\bullet} \tag{51}$$
$$= P_\bullet \tag{52}$$

  Since $\frac{P_\circ^* S_\circ}{R} < \frac{P_\circ^* S_\circ \tilde{\alpha}}{R} < q^*$, this case is possible if $1 < \tilde{\alpha} < \frac{q^* R}{P_\circ^* S_\circ}$.
- Case 4: $q = \frac{P_\circ^* S_\circ}{R}, q^+ = q^* < \frac{P_\circ^* S_\circ^+}{R^+} = \frac{P_\circ^* S_\circ \tilde{\alpha}}{R}$

$$= \frac{(1-q^*)R\alpha}{S_\bullet} \tag{53}$$
$$> (1 - \frac{P_\circ^* S_\circ \tilde{\alpha}}{R})\frac{R\alpha}{S_\bullet} \tag{54}$$
$$= (1 - q(1 + \frac{m(1-\varphi_{\beta^+}(\tau))(1-q)}{qS_\bullet}))\frac{R\alpha}{S_\bullet} \tag{55}$$
$$= (1-q)\frac{R}{S_\bullet} \frac{(1 - \frac{m(1-\varphi_{\beta^+}(\tau))}{S_\bullet})}{(1 - \frac{m}{S_\bullet})} \tag{56}$$
$$> \frac{E}{S_\bullet} \tag{57}$$
$$= P_\bullet \tag{58}$$

  Since $\frac{P_\circ^* S_\circ}{R} < \frac{P_\circ^* S_\circ \tilde{\alpha}}{R} < q^*$, this case is possible if $1 < \frac{q^* R}{P_\circ^* S_\circ} < \tilde{\alpha}$.
- $a = \beta^-$

- $\beta^- |R, S_\circ, S_\bullet\rangle = |R, S_\circ - \beta_\circ^-, S_\bullet + \beta_\bullet^-\rangle$
-

$$P_\bullet^- = \frac{E^-}{S_\bullet^-} \tag{59}$$
$$= \frac{(1-q^-)R^-}{S_\bullet^-} \tag{60}$$
$$= \frac{(1-q^-)R}{S_\bullet + \beta_\bullet^-} \tag{61}$$
$$= \frac{(1-q^-)R}{S_\bullet + \frac{m(1-\varphi_{\beta^-}(\tau))qS_\bullet}{(1-q)S_\circ}} \tag{62}$$
$$= \frac{(1-q^-)R}{S_\bullet \left(1 + \frac{m(1-\varphi_{\beta^-}(\tau))q}{(1-q)S_\circ}\right)} \tag{63}$$
$$= \frac{(1-q^-)R\alpha}{S_\bullet}, \alpha < 1 \tag{64}$$

- Case 1: $q = q^*, q^- = q^*$
  By assumption, we do not consider this case since it would result in a decrease in the equity per volatile tokeon.
- Case 2: $q = q^*, q^- = \frac{P_\circ^* S_\circ^-}{R^-} = \frac{P_\circ^* S_\circ \tilde{\alpha}}{R} < q^*$
  By assumption, we do not consider this case since it would result in a decrease in the equity per volatile tokeon, where $\tilde{\alpha} = (1 - \frac{m}{S_\circ}) < 1$.
- Case 3: $q = \frac{P_\circ^* S_\circ}{R}, q^- = \frac{P_\circ^* S_\circ^-}{R^-} = \frac{P_\circ^* S_\circ \tilde{\alpha}}{R} < q$

$$= \frac{(1 - \frac{P_\circ^* S_\circ \tilde{\alpha}}{R})R\alpha}{S_\bullet} \tag{65}$$
$$= \frac{(1 - q\tilde{\alpha})R\alpha}{S_\bullet} \tag{66}$$
$$= \frac{(1 - q(1 - \frac{m}{S_\circ}))R}{(1 + \frac{m(1-\varphi_{\beta^-}(\tau))q}{(1-q)S_\circ})S_\bullet} \tag{67}$$
$$= \frac{(1-q)R}{S_\bullet} \frac{(1 - q(1 - \frac{m}{S_\circ}))}{(1 - q(1 - \frac{m(1-\varphi_{\beta^-}(\tau))}{S_\circ}))} \tag{68}$$
$$> \frac{E}{S_\bullet} \tag{69}$$
$$= P_\bullet \tag{70}$$

  This case is possible by definition since $\tilde{\alpha} < 1$.
- Case 4: $q = \frac{P_\circ^* S_\circ}{R}, q^- = q^* < \frac{P_\circ^* S_\circ^-}{R^-} = \frac{P_\circ^* S_\circ \tilde{\alpha}}{R}$
  Since we assume the neutron target price remains constant between actions, this case is not possible.

◀

Djed Shu: A Stablecoin Protocol with Two Oracles – Protocol Description –

Zahnentferner
The Stable Order

Yogesh Agrawal
AOSSIE – The Australian Open Source Software Innovation and Education

Abstract

This paper describes Djed Shu, a new version of the Djed stablecoin protocol. Like previous versions, Djed Shu is an autonomous crypto-backed stablecoin protocol. Djed Shu differs from and improves upon previous versions of Djed by using two oracle prices. This makes it more resilient and robust against oracle manipulations and delays.

2012 ACM Subject Classification Theory of computation → Design and analysis of algorithms

Keywords and phrases blockchain, cryptocurrency, stablecoin

Digital Object Identifier 10.4230/LIPIcs.STABILITY.2025.XX

1 Introduction

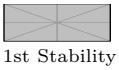
In the narrowest sense, a *stablecoin* is a cryptocurrency that has its price pegged to a fiat currency (e.g. USD) and is fully backed by reserves denominated in the same fiat currency. More broadly, a *stablecoin* can be defined as a digital asset that has mechanisms to maintain a low deviation of its price from a target price [2].

This paper describes Djed Shu, a new version of the Djed stablecoin protocol. Like previous versions, Djed Shu is an autonomous crypto-backed stablecoin protocol. It behaves like a central bank that, using its reserves, buys and sells its currency (the stablecoin) in order to keep it pegged within a target range. It is autonomous in the sense that it acts according to transparent and well-defined rules, that are executed by smart contracts independent of external influences. And it is crypto-backed in the sense that the bank keeps a cryptocurrency as its reserve asset. Besides stablecoins, the protocol also trades reservecoins in order to capitalize itself and maintain a reserve ratio significantly greater than 100% to absorb shocks in the price of the volatile cryptocurrency. Djed Shu differs from and improves upon previous versions of Djed by using two oracle prices. This makes it more resilient and robust against oracle manipulations and delays.

Djed Shu is defined in Section 2. The protocol is designed to be as intuitive and minimalistic as possible, while still being stable, safe and secure. Current implementations of Djed Shu are briefly discussed in Section 3. Section **??** discusses related work.

2 The Djed Shu Protocol

The Djed protocol's main purpose is to maintain the price of internally issued stablecoins (SCs) stable relative to a target price. The target price can be, for instance, the price of USD or EUR, or any other currency, asset or index. To achieve this, Djed maintains a reserve of basecoins (BCs), which is another cryptocurrency, such as BTC or ETH. The protocol issues SCs to users in exchange for BCs, which are then stored in the reserve. The amount of BCs a user has to pay for one stablecoin is defined by the exchange rate between the BC and the target asset, which is provided by an oracle (e.g., if a user buys 1 SC pegged to USD, the


© Zahnentferner and Yogesh Agrawal;
licensed under Creative Commons License CC-BY 4.0



user pays an amount of BCs worth 1 USD). Similarly, the SCs can be sold back to Djed in exchange for a corresponding amount of BCs according to the current exchange rate.

As the price of BCs can be volatile, the protocol aims to have abundant reserves to preserve the peg to the target price even if the price of BCs drops significantly relatively to the target asset. Additional BCs are acquired to the reserve through issuance of reservecoins (RCs). Purchase of RCs is attractive for users looking for greater returns. Not only RC holders indirectly benefit from the trading fees that accumulate in the reserve, but they are also exposed to a leverage effect: ceteris paribus, when the price of BCs grows, the price of RCs grows even more. Furthermore, the lower the reserve ratio, the stronger this leverage effect. Thus, buying reservecoins is more attractive precisely when the reserve ratio is lower. As buying reservecoins increases the reserve ratio, this varying leverage effect serves as an important mechanism for the sustainability of the protocol. On the other hand, the leverage effect also brings greater risks: ceteris paribus, when the price of BCs fall, the price of RCs fall even more. Thus, RC holders take the risk of BC price depreciation.

Djed's state is a triple (R, N_{SC}, N_{RC}) where:

- R is the number of basecoins (BCs) in Djed's reserve,
- N_{SC} is the number of stablecoins (SCs) in circulation,
- N_{RC} is the number of reservecoins (RCs) in circulation.

The protocol defines 4 operations to interact with the users:

1. The user **Buys SCs** from Djed by sending an amount of BCs to Djed. Djed then stores the provided BCs in the reserve and mints new SCs to the user.
2. The user **Sells SCs** to Djed. Djed then burns the provided SCs and returns to the user an amount of BCs from the reserve.
3. The user **Buys RCs** from Djed by sending an amount of BCs to Djed. Djed then stores the provided BCs in the reserve and mints new RCs to the user.
4. The user **Sells RCs** to Djed. Djed then burns the provided RCs and returns to the user an amount of BCs from the reserve.

Djed Shu uses an oracle system with two prices to enhance stability and protect reserves during times of high price volatility. By using two prices, Djed Shu acknowledges the fact that there is always some uncertainty in the price of one unit of the peg currency in BCs. The two prices could be the maximum and minimum prices over a specified time window, or the latest price and a moving average price, the maximum and minimum latest prices over a set of markets, the prices one standard deviation deviation above and below an average price over a period of time... What matters for Djed Shu is that the oracle provides two prices, denoted $X^{\uparrow}$ (the higher of the two prices) and $X^{\downarrow}$ (the lower of the two prices), with $0 < X^{\downarrow} \leq X^{\uparrow}$.

The *target price* of an SC in BCs now depends on the operation being performed. For operations requiring conservative pricing (user buying SCs or selling RCs), the target price is $P^t_{SC} = X^{\uparrow}$. For operations allowing liberal pricing (user selling SCs or buying RCs), the target price is $P^t_{SC} = X^{\downarrow}$. Djed Shu always chooses the price that maximizes preservation of its reserve.

Because Djed Shu's reserve may be insufficient to buy back all stablecoins for the target price, Djed Shu sets the actual *price* P_{SC} of SCs according to the following equation, where the target price P^t_{SC} depends on the specific operation:

$$P_{SC} = \begin{cases} P^t_{SC} & \text{if } N_{SC} = 0 \\ min(P^t_{SC}, \frac{R}{N_{SC}}) & \text{otherwise.} \end{cases} \quad (1)$$

For clarity, we define operation-specific SC prices:

$$P_{SC}^{\uparrow} = min(X^{\uparrow}, \frac{R}{N_{SC}}) \quad \text{(when user is buying SCs or selling RCs)} \tag{2}$$

$$P_{SC}^{\downarrow} = min(X^{\downarrow}, \frac{R}{N_{SC}}) \quad \text{(when user is selling SCs or buying RCs)} \tag{3}$$

The portion of Djed Shu's reserve that would need to be used to buy back all stablecoins is known as its *liabilities*. Since the protocol buys back stablecoins using the lower price, liabilities are calculated as:

$$L(N_{SC}) = N_{SC} P_{SC}^{\downarrow} \tag{4}$$

The *reserve ratio* is calculated as:

$$r(R, N_{SC}) = \frac{R}{L(N_{SC})} = \frac{R}{N_{SC} P_{SC}^{\downarrow}} \tag{5}$$

Djed Shu maintains safety by having a *minimum reserve ratio* r_{min} and disallowing users from buying SCs or selling back RCs if, after these actions, $r(R, N_{SC}) < r_{min}$. To prevent dilution for the RC holders, Djed Shu also has a *maximum reserve ratio* r_{max} and disallows users from buying more RCs if, before or after the purchase, $r(R, N_{SC}) > r_{max}$; unless $N_{SC} < N_{SC}^{*}$, where N_{SC}^{*} is the *threshold number of stablecoins* parameter.

The reserve surplus is Djed Shu's *equity*. Since equity calculations depend on which SC price is used for valuation, we define:

$$E^{\downarrow}(R, N_{SC}) = R - N_{SC} P_{SC}^{\uparrow} \tag{6}$$

$$E^{\uparrow}(R, N_{SC}) = R - N_{SC} P_{SC}^{\downarrow} \tag{7}$$

Djed Shu's equity is shared equally among RC holders. We define both a *buying price* and a *selling price* for RCs:

$$P_{RC}^{b}(R, N_{SC}, N_{RC}) = \begin{cases} max\left(\frac{E^{\uparrow}(R, N_{SC})}{N_{RC}}, P_{RC}^{min}\right) & \text{if } N_{RC} > 0 \\ P_{RC}^{init} & \text{otherwise} \end{cases} \tag{8}$$

$$P_{RC}^{s}(R, N_{SC}, N_{RC}) = \frac{E^{\downarrow}(R, N_{SC})}{N_{RC}} \tag{9}$$

where P_{RC}^{min} is a parameter of Djed Shu.

Djed Shu's operations are summarized in Table 1. And Djed Shu's 6 parameters (r_{min}, r_{max}, fee, N_{SC}^{*}, P_{RC}^{min}, P_{RC}^{init}) are assumed to be set to values that satisfy the following constraints: $r_{min} > 1 + fee$; $r_{max} \geq r_{min}$; $0 < fee \leq 1$; $N_{SC}^{*} > 0$; $P_{RC}^{min} > 0$; $P_{RC}^{init} > P_{RC}^{min}$.

Table 1 Djed Shu's User Actions

Action	Price	User Sends	User Receives	Condition
Buy SCs	$X^{\uparrow}$	$n(1+fee)P_{SC}^{\uparrow}$	n SCs	$r(R, N_{SC}) \geq r_{min}$
Sell SCs	$X^{\downarrow}$	n SCs	$n(1-fee)P_{SC}^{\downarrow}$	
Buy RCs	$X^{\downarrow}$	$n(1+fee)P_{RC}^{b}(R, N_{SC}, N_{RC})$	n RCs	$r(R, N_{SC}) \leq r_{max}$ or $N_{SC} < N_{SC}^{*}$
Sell RCs	$X^{\uparrow}$	n RCs	$n(1-fee)P_{RC}^{s}(R, N_{SC}, N_{RC})$	$r(R, N_{SC}) \geq r_{min}$

3 Implementations

Djed Shu has the following implementations:

- **Djed Shu in Solidity for EVM-Compatible Blockchains:** Djed Shu has been implemented in Solidity for EVM-Compatible blockchains, using the blockchain's native currency as the basecoin. Since most oracles in EVM-compatible chains provide only a single latest price, this implementation also includes an *oracle converter* (implemented in the `ShuOracleConverter` contract), which transforms any single-price oracle into a time-windowed min/max oracle system. The converter maintains a 24-hour sliding window of hourly price averages $\bar{P}_h$ for hour h, computed as $\bar{P}_h = \frac{1}{n_h} \sum_{i=1}^{n_h} P_{h,i}$, where $P_{h,i}$ is the i-th price reading in hour h and n_h is the number of updates in that hour. Given the 24-hour window of prices $\{\bar{P}_0, \bar{P}_1, \ldots, \bar{P}_{23}\}$, the converter computes $X^{\downarrow} = \min_{h \in [0,23]} \bar{P}_h$ and $X^{\uparrow} = \max_{h \in [0,23]} \bar{P}_h$, ensuring that $X^{\downarrow} \leq X^{\uparrow}$ always holds. The source code of this implementation can be found in https://github.com/DjedAlliance/Djed-Solidity.
- **Zephyr:** Zephyr (https://zephyrprotocol.com/) is a fork of Monero (https://www.getmonero.org/) where Djed Shu has been implemented natively in the blockchain's ledger layer. This results in a blockchain that combines the privacy features of Monero with the stability of Djed. In Zephyr, the oracle's two prices are the latest price and a moving average.

4 Discussion

Previous versions of Djed [3, 4], which rely on oracles that provide a single price, are sensitive to oracle delays and price manipulations. The protection against such problems is done solely through the fee for the buy and sell operations. For instance, if the fee is 2%, then the bank can tolerate a discrepancy of up to approximately 2% between the price provided by the oracle and the real price. In case of a greater discrepancy, due to oracle delays or price manipulations, users can arbitrage against the Djed bank because, essentially, Djed would be selling stablecoins or reservecoins for less than they are worth elsewhere or buying stablecoins or reservecoins for more than they are worth elsewhere. This would lead to a reduction in Djed's reserve. Thus, to protect against such issues, previous versions of Djed needed high fees, which resulted in a large (and fixed!) spread between the effective buy and sell prices for stablecoins and reservecoins.

Djed Shu, on the other hand, thanks to using oracles with two prices, has a dynamic and variable spread. During times of high volatility, the two prices will diverge and the spread will be larger. During times of low volatility, the spread will be smaller. This allows Djed Shu to have lower base fees and provides robust protection against various oracle attacks.

With an oracle where the two prices are the minimum and maximum in a period of time or the latest price and a moving average, for example, price manipulations would have to be sustained for a long period of time to affect the one price that would allow an attacker to profit from the price manipulation. This rules out so-called flash loan price manipulation attacks that occur in a short period of time. The asymmetric pricing mechanism inherently protects against sandwich attacks since SC purchases use $X^{\uparrow}$ and sales use $X^{\downarrow}$, preventing attackers from profiting through artificial price movements.

The oracle converter has also been designed to prevent attacks. For example, gas-based DoS attacks are prevented through selective min/max recalculation: full array scans are only executed when the sliding window moves past current extreme indices. The converter also maintains state consistency through safe initialization (all array elements initialized to current price), bounded array access using modular arithmetic, and monotonic update counters ensuring proper hourly averaging.

We conjecture that all stability theorems proven for previous versions of Djed hold for Djed Shu as well, since the theorem staments and proofs just need to be adapted to use one of the two prices, depending on the case. But adapting these proofs remain for future work.

Another direction for future work is the combination of Djed Shu and the Gluon Stablecoin Protocol [1]. Current versions of Gluon rely on a single-price oracle, but tackle the sensitivity to oracles by making the fees depend on recent transaction volume. A combination of both protocols could provide even greater robustness against oracle delays and price manipulations.

References

1 Bruno Woltzenlogel Paleo, Luca D'Angelo, Mohammad Shaheer, and Giselle Reis. Gluon w: A cryptocurrency stabilization protocol. Cryptology ePrint Archive, Paper 2025/1372, 2025. URL: https://eprint.iacr.org/2025/1372.

2 Bruno Woltzenlogel Paleo. *Stablecoin*, pages 1–5. Springer Berlin Heidelberg, Berlin, Heidelberg, 2019. doi:10.1007/978-3-642-27739-9_1671-1.

3 Joachim Zahnentferner, Dmytro Kaidalov, Jean-Frédéric Etienne, and Javier Díaz. Djed: A formally verified crypto-backed pegged algorithmic stablecoin. Cryptology ePrint Archive, Report 2021/1069, 2021. https://eprint.iacr.org/2021/1069.

4 Joachim Zahnentferner, Dmytro Kaidalov, Jean-Frédéric Etienne, and Javier Díaz. Djed: A formally verified crypto-backed pegged autonomous stablecoin. In *IEEE International Conference on Blockchain and Cryptocurrency*, 2023. https://icbc2023.ieee-icbc.org/program/icbc-2023-final-program.

Dexy: Simple Stablecoin Design Based on Algorithmic Central Bank

Alexander Chepurnoy
Ergo Platform

Amitabh Saxena
Ergo Platform

Luca D'Angelo
zenGate Global
The Stable Order

Abstract

In this paper, we consider a new stablecoin protocol design called Dexy. The protocol is maintaining the peg with two assets only (basecoin and stablecoin) and two functional components, namely, a reference market (in the form of a liquidity pool where anyone can trade basecoins against stablecoins) and an algorithmic central bank, which is responsible for new stablecoin issuance and for maintaining stablecoin pricing corresponding to a peg in the reference market.

2012 ACM Subject Classification Applied computing → Economics

Keywords and phrases Blockchain, cryptocurrency, stablecoin

Acknowledgements Authors would like to thank Ile for his inspiring forum posts, Bruno for discussions and the extract to future idea, and 4EYESLabs for the audit of all the ErgoScript contracts.

1 Introduction

Algorithmic stablecoins are a natural extension of cryptocurrencies, trying to solve problems with volatility of their prices by pegging the stablecoin price to an asset whose price is considered to be more or less stable over long periods of time (e.g. gold).

Having an asset with a stable value can be useful in many scenarios, for example:

- Securing fundraising, a project can be sure that funds collected during fundraising will have stable value in the mid- and long-term.
- Doing business with predictable results, for example, a shop can be sure that funds collected from sales will be about the same value when the shop is ordering goods from warehouses (otherwise, the shop may go bankrupt if its margin is not that big to cover exchange rate fluctuations).
- Shorting, when cryptocurrency prices are high, it is desirable for investors to rebalance their portfolio by increasing exposure to fiat currencies (or real-world commodities). However, as fiat currencies and centralized exchanges impose significant risks, it would be better to buy fiat and commodity substitues in form of stablecoins on decentralized exchanges.
- Lending and other decentralized finance applications, stability of collateral value is critical for many applications.

Algorithmic stablecoins are different from centralized stablecoins, such as USDT and USDC, for which there is a trusted party that holds the peg. In case of an algorithmic stablecoin, the pegging is done via rebasement of total supply (e.g. [4]), or via imitating a trusted party which holds reserves and doing market interventions when it is needed for getting exchange rate back to the peg. Imititating the trusted party is usually done by allowing anyone on the blockchain to create over-collateralized financial instruments, such as

collateralized debt positions (e.g. [3]), zero-coupon bonds (as in the Yield protocol), reserve asset (e.g. [11, 12, 8]), or by issuing stabilizing financial instruments in case of depeg (e.g. [5]).

In this work, we present Dexy, a stablecoin protocol where an algorithmic central bank performs interventions in case of a depeg, presented explicitly as a contract with a few predefined rules. The bank tries to stabilize the stablecoin value in the secondary market, using a liquidity pool as a reference market, by providing stablecoin liquidity, when the stablecoin price is over the peg, or injecting basecoins from its reserves, when the stablecoin price is under the peg. In extreme cases, when bank reserves are depleted and the stablecoin price is still under the peg, its value is restored by extracting stablecoins from the liquidity pool (to inject back when the stablecoin price goes above the peg again). In some aspects, Dexy could resemble Fei or Gyroscope [7].

The rest of the paper is organized as follows. In Section 2, the Dexy design in general is explained and notation used in other sections is introduced. Section 3 defines the worst-case scenario for bank reserves (and so, for stablecoin stability). Then the Dexy bank rules are detailed in Section 4. Stability of the rules are discussed in Section 5. The trust assumptions that the Dexy protocol is based on are stated in Section 6. Section 7 describes concrete Dexy implementation, the gold-pegged DexyGold stablecoin and its tokenomics.

2 Dexy Design in General

Unlike popular algorithmic stablecoins based on two tokens - in addition to basecoins - such as Djed and Gluon, Dexy is based on one token - the stablecoin - but with two protocols. The first protocol is a reference market implemented on-chain, such as an automated market maker (e.g. [10]), where trading of Dexy stablecoins and basecoins (e.g. ERG) occurs. The second protocol handles situations when the reference market price deviates too much from the target price of the pegged asset, as reported by a trusted oracle, and is implemented as an algorithmic central bank that is doing interventions in order to readjust the market price so it converges back to the target price of the oracle. The central bank can also mint new Dexy tokens by selling them for basecoins. The bank is using reserves in basecoins it is buying - i.e. deposited from users purchasing stablecoins directly from the bank - for the interventions.

As a simple solution for the *reference market*, a constant-product Automated Market Maker (CP-AMM) liquidity pool is used, similar to ErgoDex [1] and UniSwap [6]. The pool has basecoins on one side and Dexy on another. For CP-AMM pool, multiplication of basecoins and Dexy amounts before and after a trade should be preserved, so $e * u = e' * u'$, where e and u are amounts of basecoins and Dexy in the pool before the trade, and e' and u' are amounts of basecoins and Dexy in the pool after the trade, respectively. As for any CP-AMM pool, it is possible to put liquidity into the pool, and remove liquidity from it, however, with certain limitations that will be explained.

The bank has two basic operations. It can mint new stablecoin tokens per request, using the trusted oracle's price, by accepting basecoins in its reserves. It also can intervene into markets by providing basecoins from the reserves when needed - when the price in the pool $\frac{e}{u}$ is significantly different from the price on external markets p reported by the oracle.

2.1 Notation

We now introduce notation that will be used in further sections:

- T - period before intervention starts. After one intervention, the bank can start another one only after T time units has passed.
- p - basecoin price reported by the oracle at the moment (e.g. 20 USD per ERG, note, however, that the price provided by the oracle will usually be in units of $\frac{\text{basecoin}}{\text{stablecoin}}$, representing the price of the stablecoin in units of basecoins).
- s - price when the bank should intervene in case of a sudden basecoin price crash. For example, we can assume that $s = \frac{p}{4}$ (so if p is 20 USD per ERG, then s is 5 USD per ERG, means the bank needs to have enough reserves to save the markets when the price is suddenly crashing from 20 to 5 USD per ERG).
- R - ratio between p and s, $R = \frac{p}{s}$.
- r - ratio between p, and price in the pool, which is $\frac{u}{e}$, thus $r = \frac{p}{\frac{u}{e}} = \frac{p*e}{u}$.
- e - amount of basecoin in the liquidity pool.
- u - amount of stablecoin in the liquidity pool.
- O - amount of stablecoin in circulation that is outside the liquidity pool. The amount O is not known for the Dexy protocol, but the bank can easily store how many stablecoin tokens were minted and then get O by deducting u from it.
- E - amount of basecoins in the bank.

3 Worst Case Scenario and Bank Reserves

Possible failure scenarios for such a design, and how to put restrictions and design rules for the system to ensure stability of the stablecoin token price, will now be considered.

The bank intervenes when there is a sudden drop in the basecoin price provided by the oracle and enough time has passed for the markets to have stabilize themselves, with no interventions, but have failed to do so. In our case, the bank is doing interventions based on the stablecoin price in the liquidity pool in comparison with the oracle price. So, the bank's intervention is about injecting its basecoin reserves into the pool, thereby decreasing the basecoin price to match the oracle price.

Consider an example where the basecoins are ERG, and assume that the oracle price crashes from p to s sharply and stands there, and before the crash there were e of ERG and u of stablecoin in the liquidity pool, with pool's price being p. The worst case then is when no liqudity is put into the pool during the period T. With large enough T and large enough R, this assumption is not very realistic: some traders will buy ERGs with their stablecoins anyway, price is usually failing with swings where traders could mint stablecoins thus increasing ERG bank reserves, etc. However, it would be reasonable to consider worst-case scenario, then in the real world Dexy will be even more durable than in theory.

In this case, the bank must intervene after T units of time, as the price differs significantly, and restore the price in the pool, so set it to s. We denote amounts of basecoins and stablecoins in the pool after the intervention as e' and u', respectively. Then:

- $e * u = e' * u'$.
- The bank must inject E_1 basecoins into the pool, thus after the trade there will be $e' = e + E_1$ basecoins in the pool.
- The new basecoin price in the pool will be $\frac{u'}{e'} = s$, thus $u' = s * (e + E_1)$.
- So, isolating for the amount that must be injected, we obtain $E_1 = \sqrt{\frac{e*u}{s}} - e$ basecoins that must be added into the pool from the bank.

So by injecting E_1 basecoins, the bank recovers the price. However, this is not enough, as now there are O stablecoin units which can be injected into the pool from outside. Again, it is not realistic to assume that all the O stablecoin would be injected, as some of them are simply lost, some would be kept to buy cheap basecoins at the bottom (stablecoins are often used as a bet for a basecoin price decline), etc. However, we need to assume the worst-case scenario again. We also unrealistically assume that all the O tokens are being sold in very small batches that do not significantly affecting the pool price, where after each batch, a seller of a new batch is waiting for a bank intervention to happen (so for T units of time), and sells only after the intervention. In this case, all the O tokens are being sold at price close to s, so the bank should have about $E_2 = \frac{O}{s}$ basecoins in reserve to buy the tokens back.

Summing E_1 and E_2, we obtain the basecoin reserves the bank should have to be ready for the worst-case scenario: $E_w = E_1 + E_2 = \sqrt{\frac{e*u}{s}} - e + \frac{O}{s}$.

This scenario is the worst-case for the bank (and only the bank) because it is buying all of the external O *stablecoin* at the worst possible price s, and also sets the new price by burning its own reserves only.

4 Bank and Pool Rules

Based on needed reserves for worst-case scenario estimation, we can consider minting rules accordingly. Similarly to SigUSD [2] (a Djed instantiation on the Ergo blockchain), we can, for example, target for security in case of a 4x price drop, so to consider $R = \frac{p}{s} = 4$. We can also allow minting of stablecoins while there are enough, so not less than E_w, basecoins in reserves. However, in this case, the stablecoin would be non-mintable most of the time, and only during moments of basecoin price going up significantly will it be possible to mint Dexy. As worst-case scenario is based on unrealistic assumptions, unlikely a realistic protocol can be built on top of it.

Thus, we leave worst-case scenario for UI, so dapps implemented to interact with the Dexy protocol may show, for example, collateralization for the worst-case scenario. Having on-chain data analysis, more precise estimations of reserve quality can be made (by considering stablecoins locked in DeFi protocols, stablecoins likely forgotten in wallets, etc).

Instead, we always allow for cautious minting. That is, we allow minting when oracle price is above pool's price (so the bank provides liquidity for arbitrage when the stablecoin price is above the peg), or we allow to mint a little bit (per some time period) when liquidity pool is in good shape. In details, we have two following minting operations, with minting price being the oracle's price p:

- Arbitrage mint: if price reported by the oracle is higher than in the pool, i.e. $p > \frac{u}{e}$, we allow to print enough stablecoin tokens to bring the price to p. That is, the bank allows to mint up to $\delta_u = \sqrt{p * e * u} - u$ stablecoin by accepting up to $\delta_e = \frac{\delta_u}{p}$ basecoins into reserves.
  To instantiate the rule, we can allow for arbitrage minting if the price p is more than $\frac{u}{e}$ by at least 1% for time period T_{arb} (e.g. 1 hour), with the bank charging 0.5% fee for the operation. After arbitrage mint, it is not possible to do another one within 30 minutes, in order to prevent aggressive liquidity minting via chained transactions, etc.
- Free mint: we allow to mint up to $\frac{u}{100}$ stablecoins within time period T_{free}.
  To instantiate the rule, we propose to allow for free mint if $0.98 < r < 1.02$. Minting fee in this case is also 0.5%. We propose to set T_{free} to 12 hours, then bank reserves can grow by 2% of LP volume per day when the peg is okay.

In addition to minting actions, which increase the bank reserves, we define the following action which decreases the bank reserves, but functions as the first stabilization mechanism:

- Intervention: if the price reported by the oracle is lower than the pool price by a significant enough margin, i.e. $r = \frac{p*e}{u}$ is less than or equal to some constant that is set as a protocol parameter, then the bank will provide enough basecoins to restore the pool price to the oracle price.
  To instantiate the rule, we propose to allow the bank to intervene if $r \leq 0.98$ *for time period* $T_{int} = 1$ *day. During the intervention, the period is reset, so another intervention will only occur after* $T_{int} = 1$ *day at least. The bank will provide enough basecoins to get the pool price price to 99.5% of the oracle price max, but making sure to not spend more than 1% of its reserves for that.*

We also state following rules for the liquidity pool (which, otherwise, acts as an ordinary CP-AMM liquidity pool):

- Stopping withdrawals: if r is below some threshold, withdrawals from liquidity providers are stopped (i.e. liquidity providers cannot redeem their LP tokens for basecoins/stablecoins), so only swaps between basecoins and stablecoins are possible.
  To instantiate the rule, we propose to stop withdrawals immediately if $r \leq 0.98$.
- Second stabilization mechanism: what if the bank is out of reserves, but the basecoin price in the liquidity pool is still above the oracle basecoin price? In this case we restore price in the pool by removing liquidity, and there are two possible options here:

1. Burn: if the bank if empty, and r is below some threshold, it is allowed to burn stablecoins in the pool.
   To instantiate the rule, we propose to burn stablecoin if $r \leq 0.95$ *for a time period* T_{burn}. T_{burn} *must be quite big, e.g. 1 week. We burn enough stablecoins to return to the state of stopped withdrawals, so to* $r = 0.98$. *After burning, the period is reset, so another burn will be done only after* T_{burn} *amount of time at least.*
2. Extract for the future: if the bank is empty and r is below some threshold, it is allowed to extract stablecoins from the pool and lock them in a contract, only to be released in the future when the stablecoin price is above the stablecoin oracle price (i.e. the inverse of the basecoin price peg).
   To instantiate the rule, we propose to extract stablecoins if $r \leq 0.95$ *for time period* T_{burn} *(so the same as in burn). To prioritize extracted funds over arbitrage mint, we do not have delay in releasing contract. We burn enough to return to the state of stopped withdrawals, so close to* $r = 0.98$ *(exactly,* $0.97 \leq r \leq 0.98$*). After extraction, the period is reset, so another extraction will be done after* T_{burn} *amount of time at least.*

In addition, we introduce 2% redemption fee for liquidity providers to avoid excessive liquidity hopping.

5 Stability

With second stabilization mechanism (burning or extraction) in place, the price in the reference market will eventually stabilize. However, for liquidity holders, burning is painful, extraction not as much, but still not desirable. Thus, the Dexy protocol is trying to avoid it

(unlike other protocols, such as Gyroscope or Fei, where redemption rate fails and falls below 1 immediately when the collateralization ratio falls under 100%) by giving markets time to self-stabilize, and then doing interventions. However, this could mean slower stabilization, in comparison with other protocols, but slower stabilization helps the bank to play with possible adversaries in an environment with information assymetry, since humans always have access to more information than the algorithmic bank, and with more flexible decision making as well.

Dexy is also cautious about minting new stablecoins, which could mean slow growth of the number of stablecoins issued. This could be inconvenient, especially for big players, but the protocol is focused on stability in the first place.

Please note that the liquidity pool is disincentivizing massive bank runs due to its constant-product nature. Actually, massive bank runs are simpler for the bank to resolve, in comparison to the slow drain of the worst-case scenario.

6 Trust Assumptions

It is important to explicitly state the assumptions that the protocol is based on, so a user can choose whether to trust it or not.

- Oracle: the biggest trust issue in the protocol is the oracle delivering the target price. This trust issue is unavoidable, however, it can be relaxed by using a federation of oracles that produces an average price (after noise filtering) to be delivered to the Dexy central bank, instead of relying on a single entity. It is important to reward oracles from protocol activities (such as minting), otherwise, they will be incentivized to attack the protocol. See Section 7.1 for details.
- No governance: unlike most stablecoin protocols, we do not consider governance, as it is usually another trust issue. Real world instantiations can consider governance, we suppose that such instantiations should clearly inform about governance-related trust assumptions.

7 Dexy Gold

In this section we consider a concrete implementation of the Dexy protocol, a gold-pegged stablecoin named DexyGold on the Ergo blockchain.

7.1 Oracle Incentivization and Tokenomics

Oracles security is the most important issue for our stablecoin protocol, since this is the only component that does not function autonomously on-chain. Thus, the protocol needs to reward oracles.

Oracle Pools 2.0 framework [9] has support for paying rewards in custom tokens. So we will create GORT (Gold Oracle Reward Token), which will be used to reward oracles and also developers.

X oracles wil get up to $2 \cdot X$ GORTs per epoch (one datapoint update), per Oracle Pool v2 design [9] ($2 \cdot X$ if all the oracles are active). We consider $X = 30$ oracles with one hour long epoch, which means up to 60 GORTs will be released to oracles.

Any transaction in the protocol that requires the oracle datapoint to be provided as a data-input will charge an oracle fee in ERG. This ERG will be sent to a BuyBack contract. The BuyBack contract purchases GORT available in a LP on ErgoDex, which is then

distributed to the oracle operators. Thus, the oracle operators receiving GORT tokens can redeem them for the accumulated ERG from fees in the same LP.

In addition to oracles, GORT is used to reward developers. To do that, we consider a flat emission contract, which is releasing the same amount per one hour, so 2 GORT per block during initial period (2 years).

7.2 Implementation

The DexyGold protocol has been implemented on the Ergo blockchain. The ErgoScript smart-contracts follow a separation-of-concerns design pattern, thus each one handles its own part of the protocol instead of having one giant contract, this results in a total of 18 ErgoScript contracts required for the implementation. These 18 contracts are orchestrated together with the corresponding off-chain software for the different protocol interactions and trackers necessary to check the status of the oracle and pool peg, along with the relevant delay periods. The smart-contracts, off-chain code, simulations, and tests can be found in the following repository `https://github.com/kushti/dexy-stable`. A UI for interacting with the DexyGold protocol is available at `https://crystalpool.cc`.

References

1 ErgoDEX. `https://www.ergodex.io`.

2 SigUSD. `https://sigmausd.io`.

3 The Maker Protocol: MakerDAO's Multi-Collateral Dai (MCD) System. [Online], 2017. `https://makerdao.com/en/whitepaper/`.

4 Ampleforth protocol. [Online], 2018. `https://www.ampleforth.org/`.

5 XTN - Neutrino Index Token. [Online], 2023. `https://github.com/waves-exchange/neutrino-contract/blob/master/docs/Neutrino%202.0.%20XTN%20Litepaper.pdf`.

6 Hayden Adams, Noah Zinsmeister, River Keefer, Dan Robinson, and Moody Salem. Uniswap v3 core, 2021. `https://app.uniswap.org/whitepaper-v3.pdf`.

7 Andrew Breslin. Gyroscope: A Self-Stabilizing, "All-Weather" Reserve-Backed Stablecoin. [Online], 2022. `https://consensys.io/blog/gyroscope-a-self-stabilizing-all-weather-reserve-backed-stablecoin`.

8 Bruno Woltzenlogel Paleo, Luca D'Angelo, Mohammad Shaheer, and Giselle Reis. Gluon w: A cryptocurrency stabilization protocol. Cryptology ePrint Archive, Paper 2025/1372, 2025. URL: `https://eprint.iacr.org/2025/1372`.

9 scalahub. Eip-0023 oracle pool 2.0. `https://github.com/ergoplatform/eips/pull/41`.

10 Jiahua Xu, Krzysztof Paruch, Simon Cousaert, and Yebo Feng. Sok: Decentralized exchanges (dex) with automated market maker (amm) protocols. *ACM Computing Surveys*, 2021.

11 Joachim Zahnentferner, Dmytro Kaidalov, Jean-Frédéric Etienne, and Javier Díaz. Djed: A formally verified crypto-backed pegged algorithmic stablecoin. Cryptology ePrint Archive, Report 2021/1069, 2021. `https://eprint.iacr.org/2021/1069`.

12 Joachim Zahnentferner, Dmytro Kaidalov, Jean-Frédéric Etienne, and Javier Díaz. Djed: A formally verified crypto-backed pegged autonomous stablecoin. In *IEEE International Conference on Blockchain and Cryptocurrency*, 2023. `https://icbc2023.ieee-icbc.org/program/icbc-2023-final-program`.

Money Creation With Elastic Supply Via Trust And Blockchain Assets In Global Digital Peer-to-Peer Environment



Abstract. In this paper we introduce a blockchain-based protocol to create money in self-sovereign way via trust or collateral. The protocol allows for elastic money creation in peer-to-peer environment, where there is no party which can enforce global rules for money creation and acceptance. Thus acceptance of notes created or signed by other peers made an individual choice. Similarly to spender-signed currencies, every spender is signing and backing every note he spends. Every signature is associated with a reserve, which can be made of any blockchain token, or be empty. A note may be redeemed against any reserve of its signers, and then re-redeemed against a reserve associated with earlier signature. We have implemented our proposal in software in form of a payment server, supporting flexible rules for notes acceptance, based on blacklist, whitelist, collateralization ratio. We have shown how to implement some known monetary systems, such as a local exchange trading system (LETS), and also how to do mutual credit clearing, by using proposed stack made of the on-chain protocol and offchain software.

1 Introduction

Currently, most of monetary value is created by private banks (often, offshore banks as in so-called eurodollar system [1]), following central banks requirements. As an alternative, starting with Bitcoin [2] launch in 2009, a lot of cryptocurrencies and applications on top of public blockchains are experimenting with algorithmic money issuance. As another option, we also have alternative (usually, local), monetary systems, such as LETS (local exchange trading systems [3]), timebanks, local government currencies (such as famous Woergl demurrage-based local currency back in 1930s [4]), and so on.

Control in traditional fiat monetary systems is possessed by big players (with rich getting richer effect) creating money in non-transparent ways (especially when done in offshore circuits), at the same time, fiat monetary systems (in opposite to commodity money used before fiat, as well as alternative monetary systems) have best supply elasticity. Proof-of-work cryptocurrencies (and, sometimes, other tokens on top of public blockchains) have strict and publicly known emission schedule set in an algorithm for computers, which is making them perfect digital commodity assets, on the other hand, supply is disconnected from

economic activity and so not elastic. Local currencies usually considered more fair in segniorage distribution than fiat currencies, they are often successfully boosting local economies, but, in opposite to fiat and crypto-currencies, they are not global and rarely surviving in the long term due to loss of active supporters. They are also usually hard to get into for external actors.

In this work, we propose a new global peer-to-peer network based monetary system, with decentralized issuance, individual acceptance rules and elastic supply, called ChainCash. A digital banknote in ChainCash can be created by any party and collectively backed by the means of collateral and trust. Collateral for a ChainCash note comes from reserves network peers may have, and on spending a note, a peer is attaching its reserve to collective backing. At the same time, a newly issued note could be accepted by a peer without any backing provided, for example, if such a note is issued by a friend or a trusted charity. Every peer in the system is having own individual rules for accepting notes (widely accepted standards may exist at the same time), which provides basis for elasticity of supply. We are providing details in the next section.

The paper is organized as follows. In Section 2 we describe design of the system. Section 3 provides details on implementation. In Section 4 we outline possible applications on top of ChainCash. In Section 6 we provide initial analysis of advantages and drawbacks of the proposal.

2 Design

We consider money here via its medium-of-exchange property [5]. For existing currencies, there are usually many options to represent value to exchange, such as coins, paper or plastic banknotes, digital records in different ledgers, etc. For ChainCash, we define money as a set of digital notes, each has arbitrary nominal. Some existing widely recognizeable unit-of-account is used to represent value, for example, a milligram of gold.

We consider that an economy is consisting of known agents $a_1, ..., a_n$. Then we can define medium-of-exchange property of money via a set of agents accepting monetary objects (i.e. notes). Usually, set of agents accepting some kind of money (e.g. local or foreign currency) is the same for every monetary object (e.g. a note), so an agent is accepting any note as a mean of payment, or rejecting it. In opposite, for ChainCash money, similarly to spender signed currency in [6], the set is individual for a note, so when agent a_i sees a note n, it applies its personal predicate $P_i(n)$ to decide whether to accept or reject the note.

How ChainCash notes are different from each other then? Every note is collectively backed by all the previous spenders of the note. And every agent may create reserves to be used as collateral. When an agent spends a note, whether received previously from another agent or just created by the agent itself, it is attaching his signature to it. A note could be redemeed at any time against any of reserves of agents previously signed the note. However, any agent after the first one in signatures chain is getting redemption receipt which is indicating debt of previous signers before him, and then he may redeem the receipt against

a reserve of any previous signer, with a new redeemable receipt being generated, until the first signer is reached. Also, redemption fee should be paid from the note value, the fee is incentivizing both reserves provision and also using the notes instead of redeeming them. The protocol does not impose collateralization requirements, it is allowed for an agent to issue and spend notes with an empty reserve even. It is up to agent's counter-parties then whether to accept and so back after spending an issued note with collateral or agent's trust or not.

As an example, consider a small gold mining cooperative in Ghana issuing a note backed by (tokenized) gold. The note is then accepted by the national government as a mean of tax payment. Then the government is using the note (which is now backed by gold and also trust in Ghana government, so convertible to Ghanaian Cedi as well) to buy oil from a Saudi oil company. Then the oil company, having its own oil reserve also, is using the note to buy equipment from China. Now a Chinese company has a note which is backed by gold, oil, and Cedis. It could be hard for Chinese company to redeem from a small cooperative in Ghana, so it can redeem from Ghana government, and the government may re-redeem its receipt from the cooperative.

Agent's note quality estimation predicate $P_i(n)$ is considering collaterals and trust of previous spenders. Different agents may have different collateralization estimation algorithms (by analyzing history of the single note n, or e.g. all the notes issued by previous signers of n, other options are also possible), different whitelists, blacklists, or trust scores assigned to previous spenders of the note n etc. So in general case payment sender first need to consult with the receiver on whether the payment (consisting of one or multiple notes) can be accepted. However, in the real world likely there will be standard predicates, thus payment receiver (e.g. an online shop) may publish its predicate (or just predicate id) online, and then a payment can be done without interaction needed to check shop's acceptance policy.

From desiderata above, we may describe the protocol with three basic entities, a reserve, a note, and a receipt. Possible actions involving those entities are deposit into reserve, note redemption, receipt redemption, and note spending. It is possible to witdraw funds from a reserve by simply issuing a note, then spending with a signature associated with the reserve, and then redeeming the note.

3 Implementation

We propose to implement ChainCash monetary system on top of a public blockchain as:

- a blockchain provides an instant solution for public-key infrastructure
- public blockchain allows for a global ledger solution with minimal trust assumptions [7]. As a consequence, global public ledger allows for simple analysis of notes in existence.
- smart contracts minimize trust issues in payment execution and redemption. If native blockchain currency and trustless derivatives on top of it (such as

algorithmic stablecoins) are used in reserves, trust issues in redemption could be eliminated at all. If tokenized real-world commodities and fiat backed stablecoins (such as USDT) are used in reserves, redemption could not be completely trustless (as smart contracts do not have power off the chain), but at least there is transparent accounting in on-chain part of redemption

For efficient implementation, it is natural to use a blockchain with extended UTXO transactional model [8], to have one reserve per unspent transaction output (UTXO), to avoid having global state maintained on the blockchain, and notes progression is well described by underlying UTXOs progression also. Another feature which is critical for feasible implementation is possibility to manage signatures chain efficiently. We use Ergo as a blockchain to implement ChainCash as UTXO transactional model as well as AVL+ trees support are making notes and reserves implementation feasible.

For blockchain-based ChainCash implementation, we consider implementation of the following two main parts:

- contracts for notes, reserves, and redemption receipts. Here, for now we consider on-chain contracts as the most straightforward option. Then we may consider more scalable options, such as having reserves (and maybe receipts) only on chain, and have notes making progress on a side-chain or off-chain (on top of some Layer 2 solution)
- client software (which we refer to as ChainCash Server), which is interacting with the blockchain (in future, possibly, also a p2p network where notes are making progress off-chain). This software is implementing a_i agent's functionality from the Section 2, including note quality estimation predicate $P_i(n)$. For that, the client may potentially track all the reserves and notes on the blockchain. Client's $P_i(n)$ may be configured via whitelists, blacklists, collateralization requirements provided in a configuration file. ChainCash Server can be seen as a self-sovereign digital bank in peer-to-peer free banking system, issuing own private money using common unit-of-account.

Three on-chain contracts are implemented, namely, reserve, note and receipt contracts. Reserve contract locks ERG native tokens on top of Ergo blockchain and allow to redeem native or custom tokens when a note is presented. Note contract ensures that a note (UTXO) under its control has proper history, that is, on every spending a valid signature of corresponding reserve owner is added. It is also allowing for a note to be split into two parts (payment and change), as well as note redemption. On redemption, where both reserve and note contracts are involved, an output with receipt contract is created, which contains history of ownership copied from the note input, as well as position of reserve redeemed in ownership chain and note's value. With receipt it is possible then to redeem againt an earlier reserve (reserve contract allows for that).

Basic contracts implementation described is good for starters, but can be extended in many ways. We note that it is possible to add new features without need for the whole network to update. New features, such as new reserve and

note contracts, can be proposed in form of CCIPs (ChainCash Improvement Proposals). ChainCash Server may support new features, in particular, new forms of notes. If client is asked to accept a note with unknown contract, or a note backed by unknown contract, it is just refusing to accept the note.

On-chain contracts and reference server implementation are open-sourced and available under public domain license.

4 Applications

In this section, we show how ChainCash can be used as a foundation for other monetary systems, by providing some examples. In particular, we provide details on how to implement popular community currency systems, such as a local exchange trading system [3], and then a timebank [9] can be implemented as well, by using time as unit of account in a local exchange trading system. Then we show how a local currency can be created. We note that more currency systems can be implemented, for example, as ChainCash is deriving from spender-signed currency concept, such currencies, for example, iWat [6] and its variations [10,11,12] can trivially be built on top of it. For efficiency of ChainCash, it is important to reduce the aggregate indebtedness in the system, thus we describe how multilateral trade-credit set-off can be done in Section 4.3.

4.1 Local Exchange Trading System

Local exchange trading system (LETS) [3] is a form of non-collateralized community currency, where any member of a community can issue an IOC (I Owe Community) note and pay with it within the community. To implement a local exchange trading system on top of ChainCash, every LETS member needs to whitelist everyone else, so they will accept notes of each other regardless reserves backing the notes, and thus LETS can create money within the community (the LETS circle) with no limits (or with limits the community agreed on). On the other hand, unlike traditional LETS, notes can circulate outside the LETS circle easily, if there are actors willing to accept notes from community. Implementations may vary from LETS members whitelisting unconditionally only notes issued by other members to members whitelisting notes ever signed by LETS members.

4.2 Local Currencies

A local or even national government may issue notes and enforce their acceptance within its jurisdiction by enforcing economic agents to accept notes issued or spend by the government. As well as in a LETS implementation, enforced acceptance rules may vary, for example, there could be a limit for amount of money a government may create.

Often local currencies are introducing redemption fee, to promote local usage. In ChainCash, similar goals can be achieved via modifying the reserve contract

in a way that non-locals need to pay redemption fee while locals need not, alternatively, the note contract could be modified in a way that spending to non-local addresses incurs a fee. Local currencies are often associated with demurrage, after well-known Woergl experiment [4]. Demmurage could be implemented by modifying note contract.

4.3 Multilateral Trade-Credit Set-off

Multilateral Trade-Credit Set-off [13] (MTCS) is a technique which allows invoices in closed loops to be cleared against one another. In ChainCash, it is possible to clear mutual debts by just burning atomically notes backed by counterparties in a single transaction. This will allow them to issue more notes after.

MTCS can be used along with LETS for multiplying economic activity from charity. One of possible options is as follows. Charity funds can be put in a reserve associated with a public key A. Then a local trade exchange system for a community in need is formed, and it is not including A. MTCS takes place regularly, to reduce total amount of debt created in the community. Only after enough time (and maybe some additional checks) a note can be accepted by A, then A is sending it back to a LETS member which sent the note to A, and the note can be redeemed against the reserve by the LETS member. Thus charity funds are used to cover trade disbalance in the community only, which, in many cases, could be much more efficient than other options of helping communities in need.

5 ChainCash Possible extensions

In addition to basic on-chain contracts, there could be other options described via CCIPs (ChainCash Improvement Proposals), and optionally supported in software.

The most appealing option, likely, is having notes created and making progress off-chain, with on-chain reserves. It is not clear at the moment how to build such a system in pure peer-to-peer setting. What is doable, though, is notes tracking, with a signature needed for note transfer and redemption, done by a central server or a federation.

6 ChainCash Advantages and Drawbacks

In this section, we are providing some thoughts on possible advantages and drawbacks of ChainCash. Note that practice can show completely different picture from what we are providing here (as often happens).

At the moment of writing this paper, we can highlight following advantages:

- ChainCash is unique, to the best of our knowledge, framework, where trust and backing with collateral are seamlessly and transparently combined in money issuance.

– unlike native cryptocurrrencies and algorithmic stablecoins, ChainCash provides elasticity of supply without enforcing individual users to accept notes of lower quality - it is always up to users what to accept.
– as Section 4 shows, a variety of known monetary systems can be built on top of ChainCash. This possible foundational nature of the proposal worth further research.

At the same time, at this moment the proposed design has following drawbacks:

– ChainCash notes are non-fungible, while they share the same unit-of-account, each note has unique backing. This prevents ChainCash usage in many DeFi applications, such as liquidity pools, existing lending pools etc. We note that, similarly, DAI [14] stablecoins issued against CDPs (collateralized debt posistions) with different level of collateralization also should be priced differently. And like a DAI protocol is assigning the same price to DAIs of different quality, there could be index fund like services combining notes of certain quality, by providing index fund tokens in exchange for such notes, and the tokens can be used then freely in DeFi services.
– There is no privacy in ChainCash payments now. This topic is fully left for further research.

References

1. F. Machlup, "Euro-dollar creation: A mystery story," *PSL Quarterly Review*, vol. 23, no. 94, 1970.
2. S. Nakamoto and A. Bitcoin, "A peer-to-peer electronic cash system," *Bitcoin.–URL: https://bitcoin.org/bitcoin.pdf*, vol. 4, no. 2, 2008.
3. C. C. Williams, "The new barter economy: an appraisal of local exchange and trading systems (lets)," *Journal of Public Policy*, vol. 16, no. 1, pp. 85–101, 1996.
4. M. Unterguggenbercer, "The end results of the woergl experiment.," *Annals of Public and Cooperative Economics*, vol. 10, no. 1, pp. 60–63, 1934.
5. N. Kiyotaki and R. Wright, "On money as a medium of exchange," *Journal of political Economy*, vol. 97, no. 4, pp. 927–954, 1989.
6. K. Saito, "Peer-to-peer money: Free currency over the internet," in *Web and Communication Technologies and Internet-Related Social Issues—HSI 2003: Second International Conference on Human. Society@ Internet Seoul, Korea, June 18–20, 2003 Proceedings 2*, pp. 404–414, Springer, 2003.
7. "Know your assumptions." `https://www.ergoforum.org/t/know-your-assumptions/4198`. Accessed: 2023-01-30.
8. M. M. Chakravarty, J. Chapman, K. MacKenzie, O. Melkonian, M. Peyton Jones, and P. Wadler, "The extended utxo model," in *Financial Cryptography and Data Security: FC 2020 International Workshops, AsiaUSEC, CoDeFi, VOTING, and WTSC, Kota Kinabalu, Malaysia, February 14, 2020, Revised Selected Papers 24*, pp. 525–539, Springer, 2020.
9. R. McQuaid, S. Bond, and B. Christy, "A review of local exchange and trading schemes (lets) and time banks in scotland," 2004.
10. K. Saito, "Wot for wat: Spinning the web of trust for peer-to-peer barter relationships," *IEICE transactions on communications*, vol. 88, no. 4, pp. 1503–1510, 2005.
11. "Multiplication over time to facilitate peer-to-peer barter relationships," in *16th International Workshop on Database and Expert Systems Applications (DEXA'05)*, pp. 785–789, IEEE, 2005.
12. K. Saito, E. Morino, and J. Murai, "Reduction over time to facilitate peer-to-peer barter relationships," *IEICE TRANSACTIONS on Information and Systems*, vol. 89, no. 1, pp. 181–188, 2006.
13. E. Bottazzi, C. N. Ngo, and M. Tsutsumi, "Multilateral trade credit set-off in mpc via graph anonymization and network simplex," *Cryptology ePrint Archive*, 2024.
14. B. Woltzenlogel Paleo, "Stablecoin," in *Encyclopedia of Cryptography, Security and Privacy*, pp. 1–5, Springer, 2023.

Orb: Decentralized and Sustainable Oracles

Zahnentferner
The Stable Order

Sarthak Dengre
The Stable Order

Luca D'Angelo
zenGate Global
The Stable Order

Abstract

This paper presents the Orb Oracle Protocol. Orb Oracles are decentralized in the sense that any oracle token holder can submit values and thus can act as an oracle operator. And they are sustainable in the sense that oracle operators are rewarded for submitting values and thus have a source of revenue to cover their operational costs.

2012 ACM Subject Classification Applied computing → Economics

Keywords and phrases Blockchain, cryptocurrency, staking, reward distribution

Acknowledgements We thank the community of the Stability Nexus for their feedback and support.

1 Introduction

The great promise of *smart contracts* is to allow people to implement programs that automatically perform predefined actions when agreed conditions are satisfied. When smart contracts are deployed on decentralized blockchains, their execution is guaranteed to occur without reliance on a central authority above, or a single intermediary between, the people interested in the execution of the smart contract.

However, some of the most interesting applications of smart contracts require data that is external to the blockchain. For example, crypto-backed stablecoin [2] protocols (e.g. [1, 3, 4]) need to know the price of the underlying reserve asset in relation to the peg asset, in order to know how much of the reserve asset to give to a user when the user redeems an amount of stablecoins; a prediction market protocol needs to know the outcome of an event, in order to be able to transfer funds to the users who correctly predicted the outcome.

The components that allow data to be fetched from various sources and submitted to blockchains, to be used by smart contracts, are commonly called *Oracles*. Clearly, for an application to be fully decentralized, the oracles on which it depends need to be decentralized as well.

Despite their criticality, most oracles available nowadays are surprisingly centralized. Furthermore, the centralized entities operating these oracles often rely on venture capital, or grants from the blockchains on which they deploy their oracles, to fund their operations. This raises the concern of long-term sustainability of these oracles, since their future is unclear if their operators run out of capital[1].

Immutable, unstoppable, decentralized applications need a solid platform where every component on which they depend is sustainable and decentralized as well. This paper describes Orb, a decentralized and sustainable oracle protocol, aimed at providing a reliable,

[1] This is not only a hypothetical concern. For example, the oracle of the Milkomeda EVM sidechain of Cardano simply halted, when dcSpark decided to stop operating it due to financial considerations.

resilient, robust and anti-fragile foundation for truly decentralized applications that require real-world data.

2 Protocol Description

Orb is a protocol for the creation, operation and use of oracles where the data are temporal streams of values. Examples of temporal streams of values include: price of an asset over time; economic indicators (e.g. inflation indexes, base interest rates, unemployment, ...) over time; weather variables (e.g. temperature, rainfall, ...) in a location over time....

Every Orb oracle is associated with a fungible *oracle token*. Any oracle token holder can deposit oracle tokens into the oracle and become an *oracle operator*. The amount of tokens an operator has deposited is the operator's *stake*. The deposit and withdrawal of tokens, to increase or decrease one's stake, are subject to locking periods. During a deposit, the tokens are considered staked only after a *deposit-locking period* has elapsed. Likewise, an oracle operator can only withdraw tokens, after a *withdrawal-locking period* has elapsed since the last interaction of the oracle operator with the oracle.

Every oracle operator may submit values to the oracle. The oracle calculates a weighted average aggregate of the latest values submitted by all oracle operators. This weighted average takes into account the elapsed time of each submitted value, subjecting it to an exponential decay, and the stake of the oracle operator who submitted the value.

Anyone can fund an oracle with a *reward token* (typically, the underlying blockchain's native cryptocurrency). The balance of the oracle is a *reward pool* from which oracle operators are rewarded for every value submission. The reward for each value submission is calculated in a way that operators with higher stake and who submit values regularly (but not too frequently) receive higher rewards, since their submitted values contribute more to the average.

Oracle consumers are users or smart contracts who read data from the oracle. By default, anyone is a whitelisted oracle consumer. Oracle operators may vote to blacklist or whitelist oracle consumers, with voting power linearly proportional to their stake. If the weight of blacklist votes exceeds the weight of whitelist votes for an oracle consumer and certain quorum conditions are satisfied, the oracle consumer is disallowed to read data from the oracle.

3 Protocol Specification

Every oracle created via the Orb protocol has immutable parameters, chosen by the oracle creator at the moment of deployment, and state variables, which vary as users interact with the oracle after deployment.

An oracle's parameters are a tuple of constants

$$C = (h, q, \Delta_{\text{dep}}, \Delta_{\text{wd}}, \alpha, \tau_w, \tau_r) \tag{1}$$

where:

- h is the half-life constant.
- q is the quorum constant.
- Δ_{dep} is the deposit-locking period.
- Δ_{wd} is the withdrawal-locking period.
- α is a reward factor determining which fraction of the oracle's balance is paid as reward.

- τ_w is an identifier of the oracle token used for computing weights of oracle operators.
- τ_r is an identifier of the reward token.

An oracle's state is a tuple of variables

$$V = \big(L_\ell, L_f, T_{\text{dep}}, T_{\text{op}}, P_{\text{user}}, W_{\text{user}}, T_{\text{user}}, \bar{p}, \tilde{p}, Q, \mathcal{P}, \lambda, \mathcal{G}\big), \tag{2}$$

where:

- U is the set of users (people and smart contracts) that can interact with the oracle.
- $L_\ell, L_f : U \to \mathbb{R}_{\geq 0}$ track the locked and unlocked token balances (`lockedTokens` and `unlockedTokens`)[2].
- $T_{\text{dep}}, T_{\text{op}} : U \to \mathbb{R}_{\geq 0}$ store the last deposit timestamp and the last operation timestamp.
- $P_{\text{user}} : U \to \mathbb{Z}$ stores each submitter's last reported value (`pof`).
- $W_{\text{user}} : U \to \mathbb{R}_{\geq 0}$ stores the last submission weight (`wof`).
- $T_{\text{user}} : U \to \mathbb{R}_{\geq 0}$ stores the last submission time (`tof`).
- $\bar{p} \in \mathbb{Z}$ is the stored aggregate value (`aggregatedValue`).
- $\tilde{p} \in \mathbb{Z}$ is the latest submitted value (`latestValue`).
- $Q \in \mathbb{R}_{\geq 0}$ is the stored aggregate weight (`aggregatedWeight`).
- $\mathcal{P} = \big((t_1, \bar{p}_1, \tilde{p}_1), \ldots, (t_m, \bar{p}_m, \tilde{p}_m)\big)$ is the time-ordered value history[3].
- $\lambda = (t_{\text{sub}}, t_{\text{last}}, L_{\text{tot}})$ stores the last value submission time, the last oracle interaction time, and the current total of deposited tokens.
- $\mathcal{G} = \big(\mathcal{B}, V_{\text{black}}, V_{\text{white}}, M_{\text{black}}, M_{\text{white}}, \mathcal{W}_{\text{black}}, \mathcal{W}_{\text{white}}\big)$ is the governance state, where:
  - $\mathcal{B} : U \to \{0, 1\}$ the blacklist indicator,
  - $V_{\text{black}}, V_{\text{white}} : U \to \mathbb{R}_{\geq 0}$ the accumulated blacklist and whitelist weights,
  - $M_{\text{black}}, M_{\text{white}} : U \to 2^U$ the sets of targets each voter has supported,
  - $\mathcal{W}_{\text{black}}, \mathcal{W}_{\text{white}} : U \times U \to \mathbb{R}_{\geq 0}$ the per-target stored weights.

The user balances of oracle tokens and reward tokens outside are denoted by $B_{\text{user}} : U \times \{\tau_w, \tau_r\} \to \mathbb{R}_{\geq 0}$, whereas the balances of the oracle itself is denoted by $B_{\text{oracle}} : \{\tau_w, \tau_r\} \to \mathbb{R}_{\geq 0}$.

3.1 Auxiliary Definitions

▶ **Definition 1** (Exponential Decay Factor). *The exponential decay factor, used to halve weights every h seconds, is defined as:*

$$\delta(\Delta) = 2^{-\Delta/h} \tag{3}$$

▶ **Definition 2** (Time-Dependent Functions). *Given the current state V and time t, the following time-dependent functions are defined:*

- *The* decayed weight *of user u at time t is* $W_{\text{decayed}}(u, t) = W_{\text{user}}(u) \cdot \delta\big(t - T_{\text{user}}(u)\big)$.
- *The* decayed aggregate weight *at time t is* $Q_{\text{decayed}}(t) = Q \cdot \delta(t - t_{\text{sub}})$.
- *The* decayed aggregate value *at time t is* $\bar{p}_{\text{decayed}}(t) = \bar{p} \cdot \delta(t - t_{\text{sub}})$.
- *The* ideal decayed weighted mean *at time t is*

$$P(t) = \frac{\sum_{x \in U} P_{\text{user}}(x) \cdot W_{\text{decayed}}(x, t)}{\sum_{x \in U} W_{\text{decayed}}(x, t)}. \tag{4}$$

[2] The names between parentheses are the names of the state variables in our reference implementation in `https://github.com/StabilityNexus/OrbOracle-Solidity` and are intended to ease the mapping between the mathematical specification and the reference implementation.

[3] We write $\mathsf{append}(\mathcal{P}, x)$ for appending a triple x to the ordered value history.

3.2 Auxiliary State Update Procedures

The following procedures are used by many operations specified in subsection 3.3. The *unlock* procedure is called by any operation that requires knowing the current unlocked oracle token balance of the user. The *recompute* procedure is called whenever users vote to blacklist or whitelist. The *reweight* procedure updates the weights of every blacklist/whitelist vote cast by a user. It is called automatically when users withdraw or on demand by users when they deposit and explicitly synchronize their weights[4].

$$\text{Unlock}(u,t): \begin{cases} \text{if } t \geq T_{\text{dep}}(u) + \Delta_{\text{dep}} \text{ and } L_\ell(u) > 0 \\ \quad L_f(u) \leftarrow L_f(u) + L_\ell(u) \\ \quad L_\ell(u) \leftarrow 0. \end{cases} \tag{5}$$

$$\text{Recompute}(x): \begin{cases} \mathcal{B}(x) \leftarrow 1 & \text{if } V_{\text{black}}(x) - V_{\text{white}}(x) > q \cdot \big(L_{\text{tot}} - (V_{\text{black}}(x) + V_{\text{white}}(x))\big) \\ \mathcal{B}(x) \leftarrow 0 & \text{otherwise.} \end{cases} \tag{6}$$

$$\text{Reweight}(u,w): \begin{cases} \forall x \in M_{\text{black}}(u): \\ \quad V_{\text{black}}(x) \leftarrow V_{\text{black}}(x) - \mathcal{W}_{\text{black}}(x,u) + w \\ \quad \mathcal{W}_{\text{black}}(x,u) \leftarrow w \\ \quad \text{Recompute}(x) \\ \forall x \in M_{\text{white}}(u): \\ \quad V_{\text{white}}(x) \leftarrow V_{\text{white}}(x) - \mathcal{W}_{\text{white}}(x,u) + w \\ \quad \mathcal{W}_{\text{white}}(x,u) \leftarrow w \\ \quad \text{Recompute}(x). \end{cases} \tag{7}$$

3.3 Protocol Operations

Each operation defined in the folowing subsections is an action that a user may perform to interact with an oracle. The subsections define how they update the oracle's state.

[4] Reweight is an expensive procedure. This is the reason why it is called automatically only on withdrawals, when it is necessary, but only on explicit demand by users after deposits, if they wish.

3.3.1 Token Deposit

For $a > 0$, the deposit operation $\delta_{\text{dep}} : U \times \mathbb{R}_{>0} \times \mathbb{R}_{\geq 0} \to V \to V$ updates state as follows:

$$\delta_{\text{dep}}(u, a, t)(V) = \begin{cases} \text{Unlock}(u, t) \\ L_\ell(u) \leftarrow L_\ell(u) + a \\ T_{\text{dep}}(u) \leftarrow t \\ T_{\text{op}}(u) \leftarrow t \\ L_{\text{tot}} \leftarrow L_{\text{tot}} + a \\ t_{\text{last}} \leftarrow t \\ B_{\text{user}}(u, \tau_w) \leftarrow B_{\text{user}}(u, \tau_w) - a \\ B_{\text{oracle}}(\tau_w) \leftarrow B_{\text{oracle}}(\tau_w) + a \end{cases} \tag{8}$$

3.3.2 Token Withdrawal

For $a > 0$ with $L_f(u) \geq a$ and $T_{\text{op}}(u) + \Delta_{\text{wd}} \leq t$, the withdrawal operation $\chi : U \times \mathbb{R}_{>0} \times \mathbb{R}_{\geq 0} \to V \to V$ updates the state as follows:

$$\chi(u, a, t)(V) = \begin{cases} \text{Unlock}(u, t) \\ L_f(u) \leftarrow L_f(u) - a \\ L_{\text{tot}} \leftarrow L_{\text{tot}} - a \\ T_{\text{op}}(u) \leftarrow t \\ t_{\text{last}} \leftarrow t \\ B_{\text{user}}(u, \tau_w) \leftarrow B_{\text{user}}(u, \tau_w) + a \\ B_{\text{oracle}}(\tau_w) \leftarrow B_{\text{oracle}}(\tau_w) - a. \end{cases} \tag{9}$$

3.3.3 Value Submission

Only token holders with unlocked stake may submit values (`submitValue`). Let $v \in \mathbb{Z}$ denote the new submission and t the current timestamp. Set $w = L_f(u)$, $p_u = P_{\text{user}}(u)$, $w_u = W_{\text{user}}(u)$, and $t_u = T_{\text{user}}(u)$. Using the decay factor (3), we compute the updated aggregate weight and value as

$$Q' = \big(Q - w_u \cdot \delta(t_{\text{sub}} - t_u)\big) \cdot \delta(t - t_{\text{sub}}) + w, \tag{10}$$

$$\bar{p}' = \frac{\big(\bar{p} \cdot Q - p_u \cdot w_u \cdot \delta(t_{\text{sub}} - t_u)\big) \cdot \delta(t - t_{\text{sub}}) + v \cdot w}{Q'}. \tag{11}$$

The contract pays out

$$n = \alpha \cdot B_{\text{oracle}}(\tau_r) \cdot \frac{w}{Q'} \cdot \big(1 - \delta(t - t_u)\big), \tag{12}$$

The value submission operation $\phi : U \times \mathbb{Z} \times \mathbb{R}_{\geq 0} \to V \to V$ updates the state as follows:

$$\phi(u, v, t)(V) = \begin{cases} \text{Unlock}(u, t) \\ P_{\text{user}}(u) \leftarrow v \\ W_{\text{user}}(u) \leftarrow L_f(u) \\ T_{\text{user}}(u) \leftarrow t \\ \bar{p} \leftarrow \bar{p}' \\ \tilde{p} \leftarrow v \\ Q \leftarrow Q' \\ t_{\text{sub}} \leftarrow t \\ t_{\text{last}} \leftarrow t \\ \mathcal{P} \leftarrow \mathsf{append}\big(\mathcal{P}, (t, \bar{p}', v)\big) \\ B_{\text{user}}(u, \tau_r) \leftarrow B_{\text{user}}(u, \tau_r) + n \\ B_{\text{oracle}}(\tau_r) \leftarrow B_{\text{oracle}}(\tau_r) - n \end{cases} \tag{13}$$

3.3.4 Value Reading

Anyone who is not blacklisted may read oracle values (`readValue` and `readLatestValue`). This refreshes the liveness timestamp. The state update is:

$$\rho(u, t)(V) : \Big\{ t_{\text{last}} \leftarrow t. \tag{14}$$

3.3.5 Vote to Blacklist

The blacklist vote operation $\beta : U \times U \times \mathbb{R}_{\geq 0} \to V \to V$ requires $u \notin \mathcal{B}$, $L_f(u) > 0$, and that u has not voted on x before. It updates the state as follows:

$$\beta(u, x, t)(V) = \begin{cases} \text{Unlock}(u, t) \\ V_{\text{black}}(x) \leftarrow V_{\text{black}}(x) + L_f(u) \\ \mathcal{W}_{\text{black}}(x, u) \leftarrow L_f(u) \\ M_{\text{black}}(u) \leftarrow M_{\text{black}}(u) \cup \{x\} \\ T_{\text{op}}(u) \leftarrow t \\ t_{\text{last}} \leftarrow t \\ \text{Recompute}(x). \end{cases} \tag{15}$$

3.3.6 Vote to Whitelist

Analogously, the whitelist vote $\omega : U \times U \times \mathbb{R}_{\geq 0} \to V \to V$ enforces the same preconditions and updates the state as follows:

$$\omega(u, x, t)(V) = \begin{cases} \text{Unlock}(u, t) \\ V_{\text{white}}(x) \leftarrow V_{\text{white}}(x) + L_f(u) \\ \mathcal{W}_{\text{white}}(x, u) \leftarrow L_f(u) \\ M_{\text{white}}(u) \leftarrow M_{\text{white}}(u) \cup \{x\} \\ T_{\text{op}}(u) \leftarrow t \\ t_{\text{last}} \leftarrow t \\ \text{Recompute}(x). \end{cases} \tag{16}$$

3.3.7 Weight Synchronisation

This operation (`updateUserVoteWeights`) updates the state as follows:

$$\psi(u, t)(V) : \begin{cases} \text{Unlock}(u, t) \\ \text{Reweight}\big(u, L_f(u)\big) \\ t_{\text{last}} \leftarrow t. \end{cases} \tag{17}$$

3.3.8 Reward Token Funding

Funding operations advance the liveness timestamp and increase the reward pool balance:

$$\varphi(u, a, t)(V) : \begin{cases} B_{\text{user}}(u, \tau_r) \leftarrow B_{\text{user}}(u, \tau_r) - a \\ B_{\text{oracle}}(\tau_r) \leftarrow B_{\text{oracle}}(\tau_r) + a \\ t_{\text{last}} \leftarrow t. \end{cases} \tag{18}$$

Here a is the amount of Reward Token transferred to the contract. This action is unrestricted and prepares future submissions to receive rewards.

4 Theorems

Orb oracles enjoy many important properties. The following theorems show some of them. Theorem 3 shows that the constant-time update rule computes the same result as the Ideal Decayed Weighted Mean function (Equation 4). Since a naive computation of this function would require a sum over all previously submitted values, whenever a new value is submitted, this theorem shows that Orb oracles can efficiently compute a new decayed weighted mean value of an unbounded number of submitted values in constant time.

▶ **Theorem 3** (Equivalence of the Ideal Decayed Weight Mean Function and the Constant-Time Update Rule)**.** *The Ideal Decayed Weighted Mean function P(t) and the corresponding constant-time update rule $\bar{p}_k(t)$ - $\bar{p}$ after the k-th update - are equal.*

Proof. The equivalence is proven by induction on the number of updates k and a user set U.

- Base case ($k = 1$ and $|U| = 1$, so that $t(k) = t(1) = 0$): Initially, there is only one user and no previous submissions, so $Q = \bar{p} = p_u = w_u = 0$. Thus, $Q^{'} = w$ and $\bar{p}_1(0) = \frac{v \cdot w}{w} = v = P(0)$.

- Inductive case (Assume by the induction hypothesis that $P(t) = \bar{p}_k(t)$ and $|U| = n$): At the $k+1$ update, there is a user $u \in U$ with unlocked stake that submits a new value v at a time $t(k+1) = t > t_{\text{sub}}$. So,

$$
\begin{aligned}
Q^{'} &= [Q_k - w_u \cdot \delta(t_{\text{sub}} - t_u)] \cdot \delta(t - t_{\text{sub}}) + w && (19)\\
&= \left[\sum_{x \in U} W_{\text{user}}(x) \cdot \delta(t_{\text{sub}} - t_x) - w_u \cdot \delta(t_{\text{sub}} - t_u)\right] \cdot \delta(t - t_{\text{sub}}) + w && (20)\\
&= \sum_{x \neq u} W_{\text{user}}(x) \cdot \delta(t_{\text{sub}} - t_x) \cdot \delta(t - t_{\text{sub}}) + w && (21)\\
&= \sum_{x \neq u} W_{\text{user}}(x) \cdot \delta(t - t_x) + w && (22)\\
&= \sum_{x \in U} W_{\text{user}}(x) \cdot \delta(t - t_x) && (23)\\
&= \sum_{x \in U} W_{\text{decayed}}(x, t) && (24)
\end{aligned}
$$

and,

$$
\begin{aligned}
\bar{p}_{k+1}(t) &= \frac{(\bar{p}_k \cdot Q_k - p_u \cdot w_u \cdot \delta(t_{\text{sub}} - t_u)) \cdot \delta(t - t_{\text{sub}}) + v \cdot w}{Q^{'}} && (25)\\
&= \frac{(\sum_{x \in U} p_x \cdot w_x \cdot \delta(t_{\text{sub}} - t_x) - p_u \cdot w_u \cdot \delta(t_{\text{sub}} - t_u)) \cdot \delta(t - t_{\text{sub}}) + v \cdot w}{Q^{'}} && (26)\\
&= \frac{\sum_{x \neq u} p_x \cdot w_x \cdot \delta(t_{\text{sub}} - t_x) \cdot \delta(t - t_{\text{sub}}) + v \cdot w}{Q^{'}} && (27)\\
&= \frac{\sum_{x \neq u} p_x \cdot w_x \cdot \delta(t - t_x) + v \cdot w}{Q^{'}} && (28)\\
&= \frac{\sum_{x \in U} p_x \cdot w_x \cdot \delta(t - t_x)}{Q^{'}} && (29)\\
&= \frac{\sum_{x \in U} P_{\text{user}}(x) \cdot W_{\text{decayed}}(x, t)}{\sum_{x \in U} W_{\text{decayed}}(x, t)} && (30)\\
&= P(t) && (31)
\end{aligned}
$$

◀

The next theorems concern robustness against delays, which are defined as follows.

▶ **Definition 4** (Oracle Operator Delay). *Define an oracle operator's delay as:*

$$\Lambda_x(t) = \frac{W_{\text{decayed}}(x, t) \cdot (t - T_{\text{user}}(x))}{\sum_{x \in U} W_{\text{decayed}}(x, t)} \tag{32}$$

▶ **Definition 5** (Oracle Delay). *From the definition of the oracle operator delay, define the oracle's delay as:*

$$\Lambda(t) = \sum_{x \in U} \Lambda_x(t) \tag{33}$$

Theorem 6 shows that, if an oracle operator becomes inactive, the delay that it causes to the oracle gradually converges to zero. This is important, given that Orb oracles do not delete values submitted by inactive oracle operators, but only decay them.

▶ **Theorem 6** (Inactive Oracle Operator Delay). *If an oracle operator $u \in U$ becomes inactive, the operator delay $\Lambda_u(t)$ caused by this inactivity converges to 0.*

Proof. By definition, the oracle delay for an oracle operator $u \in U$ is:

$$\Lambda_u(t) = \frac{W_{\text{decayed}}(u,t) \cdot (t - T_{\text{user}}(u))}{\sum_{x \in U} W_{\text{decayed}}(x,t)} \tag{34}$$

$$= \frac{w_u \cdot \delta(t - t_u) \cdot (t - t_u)}{\sum_{x \in U} w_x \cdot \delta(t - t_x)} \tag{35}$$

$$= \frac{w_u \cdot 2^{\frac{-(t-t_u)}{h}} \cdot (t - t_u)}{\sum_{x \in u} w_x \cdot 2^{\frac{-(t-t_x)}{h}} + w_u \cdot 2^{\frac{-(t-t_u)}{h}}} \tag{36}$$

$$= \frac{w_u \cdot (t - t_u)}{\sum_{x \in u} w_x \cdot 2^{\frac{(t-t_u)}{h}} \cdot 2^{\frac{-(t-t_x)}{h}} + w_u} \tag{37}$$

$$= \frac{w_u \cdot (t - t_u)}{\sum_{x \in u} w_x \cdot 2^{\frac{(t_x-t_u)}{h}} + w_u} \tag{38}$$

$$\tag{39}$$

So, if the oracle operator u is inactive, then w_u and t_u will be constant as $t \to \infty$. As long as a subset $Y \subseteq U \setminus \{u\}$ remains active, then for $y \in Y$, $t_y \to \infty$ as $t \to \infty$, so that $t_y - t_u \to \infty$. Thus,

$$\lim_{t \to \infty} \Lambda_u(t) = 0$$

◀

Theorem 7 shows that the oracle delay is never greater than the difference between the current time and the submission time of the latest value submitted by the most delayed oracle operator. This theorem is true thanks to the fact that the constant-time update rule carefully removes the influence of non-latest values in the computed weighted average.

▶ **Theorem 7** (Optimally Small Oracle Delay). *The oracle delay $\Lambda(t)$ is bounded above by $(t - t^*)$, where $t^* = \min_{x \in U} t_x$.*

Proof. Expanding the definitions of oracle delay and oracle operator's delay, we have that:

$$\Lambda(t) = \sum_{x \in U} \Lambda_x(t) \tag{40}$$

$$= \sum_{x \in U} \frac{W_{\text{decayed}}(x,t) \cdot (t - T_{\text{user}}(x))}{\sum_{x \in U} W_{\text{decayed}}(x,t)} \tag{41}$$

$$= \sum_{x \in U} \frac{W_{\text{decayed}}(x,t) \cdot (t - t_x)}{\sum_{x \in U} W_{\text{decayed}}(x,t)} \tag{42}$$

$$\leq \sum_{x \in U} \frac{W_{\text{decayed}}(x,t) \cdot \max_{x \in U}(t - t_x)}{\sum_{x \in U} W_{\text{decayed}}(x,t)} \tag{43}$$

$$= \sum_{x \in U} \frac{W_{\text{decayed}}(x,t) \cdot (t - \min_{x \in U} t_x)}{\sum_{x \in U} W_{\text{decayed}}(x,t)} \tag{44}$$

$$= \sum_{x \in U} \frac{W_{\text{decayed}}(x,t) \cdot (t - t^*)}{\sum_{x \in U} W_{\text{decayed}}(x,t)} \tag{45}$$

$$= \frac{\sum_{x \in U} W_{\text{decayed}}(x,t) \cdot (t - t^*)}{\sum_{x \in U} W_{\text{decayed}}(x,t)} \tag{46}$$

$$= (t - t^*) \tag{47}$$

Thus,

$$\Lambda(t) \leq (t - t^*)$$

◀

Theorem 8 shows that as long as the oracle has been funded at least once, the balance of the oracle's reward pool, will always remain positive, ensuring that there is always a positive reward to incentivize oracle operators to continue submitting values.

▶ **Theorem 8** (Sustainable Rewards). *If* $0 \leq \alpha < 1$ *and* $B_{\text{oracle}}(\tau_r) > 0$ *at a given time* t*, then* $B_{\text{oracle}}(\tau_r) > 0$ *for all times* $t' > t$*.*

Proof. By assumption, since $\alpha \in [0, 1)$ and $B_{\text{oracle}}(\tau_r) > 0$ at a given time t, then any update must by definition occur at a time $t' > t$, where the remaining oracle balance will be:

$$\begin{aligned} B_{\text{oracle}}(\tau_r) - n(t') &= B_{\text{oracle}}(\tau_r) - \alpha \cdot B_{\text{oracle}}(\tau_r) \cdot \frac{w}{Q'} \cdot \big(1 - \delta(t' - t_u)\big) && (48) \\ &= B_{\text{oracle}}(\tau_r)(1 - \alpha \cdot \frac{w}{Q'} \cdot \big(1 - \delta(t' - t_u)\big)) && (49) \\ &> B_{\text{oracle}}(\tau_r)(1 - \alpha) && (50) \\ &> 0 && (51) \end{aligned}$$

Thus, for all future updates, and therefore all future times, since the remaining balance is positive and becomes the new balance, the new balance is also positive. ◀

5 Discussion

Orb Oracles are decentralized in the sense that any oracle token holder can submit values and thus can act as an oracle operator. Thanks to its efficient constant-time weighted average value update rule, an arbitrarily large number of oracle operators can participate, with no performance impact. There are, therefore, no limits to the decentralization.

Furthermore, the exponential decay ensures that, if any oracle operator becomes inactive, the contribution of its latest value submission to the average eventually and quickly becomes negligible. Considering only the latest value submitted by each oracle operator prevents a single operator from dominating the average and bounds the oracle delay.

Orb oracles as defined here deliberately avoid outlier rejection, since the rejection of a new outlier value that was submitted in reaction to a sharp change in the source value would mean that the oracle would delay to take that sharp change into account. However, variations of the Orb oracle protocol with outlier rejection could be defined and implemented. There is a trade-off between speed and robustness against possibly, but not necessarily, "wrong' outlier values.

Locking periods for depositing tokens to become an oracle operator and for withdrawing tokens to cease to be an oracle operator deter governance attacks whereby a malicious user would briefly buy or borrow tokens just to act as an oracle operator for a short time.

Orb oracles are sustainable in the sense that oracle operators are rewarded for submitting values and thus have a source of revenue to cover their operational costs. The reward formula discourages too frequent submission and prevents draining of the reward pool by a single oracle operator. It also ensures that the reward pool, if it is ever funded at least once, never becomes empty and thus there are always incentives for oracle operators to continue submiting values.

Orb oracles are also resistant to free-riding. Oracle operators are free to set their own terms and conditions for using the oracle and, if an oracle consumer is not satisfying the terms and conditions, a weighted majority of oracle operators can blacklist this consumer. In particular, the terms and conditions may involve requirements to fund the oracle. In this way, free-riding consumers, who use the oracle but do not contribute to its funding, may be blacklisted. This governance-based free-riding resistance further contributes to oracle sustainability.

The token-based oracle operation is flexible enough to encompass a wide-range of operational arrangements. For example: a centralized first-party oracle could be set up by keeping all the tokens under the control of the first-party; a permissioned consortium of oracle operators could be set up by distributing oracle tokens to the members of the consortium and making these tokens non-transferable; a fully permissionless oracle could be set up by using an oracle token that can be minted to oracle consumers in proportion to their alignment to the oracle's correct operation.

References

1 Bruno Woltzenlogel Paleo, Luca D'Angelo, Mohammad Shaheer, and Giselle Reis. Gluon w: A cryptocurrency stabilization protocol. Cryptology ePrint Archive, Paper 2025/1372, 2025. URL: https://eprint.iacr.org/2025/1372.

2 Bruno Woltzenlogel Paleo. *Stablecoin*, pages 1–5. Springer Berlin Heidelberg, Berlin, Heidelberg, 2019. doi:10.1007/978-3-642-27739-9_1671-1.

3 Joachim Zahnentferner, Dmytro Kaidalov, Jean-Frédéric Etienne, and Javier Díaz. Djed: A formally verified crypto-backed pegged algorithmic stablecoin. Cryptology ePrint Archive, Report 2021/1069, 2021. https://eprint.iacr.org/2021/1069.

4 Joachim Zahnentferner, Dmytro Kaidalov, Jean-Frédéric Etienne, and Javier Díaz. Djed: A formally verified crypto-backed pegged autonomous stablecoin. In *IEEE International Conference on Blockchain and Cryptocurrency*, 2023. https://icbc2023.ieee-icbc.org/program/icbc-2023-final-program.

A License



This work is licensed under the CreativeCommons Attribution-NonCommercial-NoDerivatives 4.0 International license[5]. For derivative and commercial uses, contact the authors first.

B Official Implementations and Deployments of Orb

https://github.com/StabilityNexus

C Official Orb Channels

- https://x.com/StabilityNexus

[5] https://creativecommons.org/licenses/by-nc-nd/4.0/

- https://discord.gg/7jS9qJNjJv

RainDrop: Reward Distribution Protocol

Zahnentferner
The Stable Order

Sarthak Dengre
The Stable Order

Abstract

This paper presents the RainDrop protocol for efficient proper reward distribution to token stakers: stakers receive rewards proportionally to the their staked balances; and each reward is only distributed to stakers who were already staking before the reward is distributed.

2012 ACM Subject Classification Applied computing → Economics

Keywords and phrases blockchain, cryptocurrency, staking, reward distribution

Acknowledgements We thank the community of the Stability Nexus for their feedback and support.

1 Introduction

Distributing rewards to token holders is a key financial operation. Note that dividend distribution, which is widely used in the legacy financial world, is a special case of reward distribution, where the reward is a fiat currency and the token is a company share.

Here we address the more general case where:

1. the held asset can be any fungible token;
2. the rewards to be distributed can be of any fungible token as well, possibly different from the held token;
3. the distribution is to be made automatically via smart contracts on a blockchain;
4. the distribution ought to be *proper*, in the sense that each token holder should receive an amount of rewards proportional to their token balances.

Surprisingly, this problem is not trivial. The first challenge lies in the fact that we cannot do the distribution to all token holders in a single transaction. The cost of this transaction would be unbounded and, for a sufficiently large number of token holders, could exceed the cost limit for the transaction to be included in a block. In other words, such an operation would be linear in the number of token holders. This challenge is solved with the idea of *staking*. Instead of performing a single linear distribution operation, the following is done: token holders are required to *stake* their tokens (i.e. deposit their tokens into the smart contract, so that the smart contract knows their balances); the rewards are *distributed* into a common pool; and then each token holder is expected to *claim* their portion of rewards from the pool. Each of these actions is a constant-time operation. So, staking turns a single linear-time transaction into a linear number of constant-time transactions.

The second challenge, however, is how to implement these actions in such a way that the amounts received by the stakers are correctly proportional to their staked balances. Naive implementations do not work.

Here is an example. Suppose that Alice and Bob staked 100 ABC tokens each and 50 XYZ tokens are distributed into a common reward pool. Alice then claims her portion (25 XYZ) from the pool. Then a second distribution of 50 XYZ tokens is made into the pool. Now the pool has 75 XYZ. Suppose that Alice claims again. She would now get 37.5 XYZ. In total, she has already received 62.5 XYZ, which is more than the 50 XYZ that she should have received.

The approach illustrated in the example above is obviously too naive. Popular staking reward distribution contracts[1] have mechanisms to track that Alice has already claimed and reduce her share of the pool, so that she would be able to claim a smaller amount from the second distribution. But such mechanisms can be partially circumvented by unstaking and staking again with a different account.

Unlike such popular but flawed staking reward distribution contracts, the RainDrop Protocol systematically tracks each distribution (*drop*) independently and with sufficient data to ensure that every staker can only claim the correct amount from each drop.

2 Protocol Description

Each RainDrop vault allows user to stake a single funglible token and receive rewards in any fungible token. Users can perform 5 actions:

1. **Stake:** a user deposits an amount of the vault's token into the vault and the user's entry point in the drop timeline is registered.
2. **Unstake:** the user withdraws their staked tokens from the vault.
3. **Drop:** any user can distribute any fungible token by creating a drop.
4. **Claim:** a user can claim their proportional share of a drop that occurred after their staking event.
5. **Withdraw:** after a drop's claim deadline, the original drop creator can withdraw all unclaimed tokens.

3 Protocol Specification

The *vault state* at time t is defined as:

$$V(t) = (S_{total}(t), S_{user}(t), E_{user}(t), D(t), k(t), H(t)) \tag{1}$$

Where:

- $S_{total}(t) \in \mathbb{R}^+$: Total staked amount
- $S_{user}(t) : U \to \mathbb{R}^+$: User stake mapping
- $E_{user}(t) : U \to \mathbb{N}$: User entry drop counter
- $D(t) = \{d_1, d_2, \ldots, d_k\}$: Sequence of drops
- $k(t) \in \mathbb{N}$: Current drop counter
- $H(t) : U \times \mathbb{N} \to \{0, 1\}$: Claim history mapping

We also define the *balance state*:

- $B_{user}(u, \tau) \in \mathbb{R}^+$: External balance of user u for token τ
- $B_{vault}(\tau) \in \mathbb{R}^+$: Vault balance for token τ
- τ_{stake}: The vault's designated staking token type

A *drop* d_i is a 5-tuple:

$$d_i = (A_i, \tau_i, S_i, T_i, C_i) \tag{2}$$

Where:

[1] e.g. Pancake Swap's Masterchef v2 (`https://bscscan.com/address/0xa5f8C5Dbd5F286960b9d90548680aE5ebFf07652#code`) and SushiSwap's Masterchef v1 (`https://etherscan.io/address/0xc2edad668740f1aa35e4d8f227fb8e17dca888cd#code`)

- $A_i \in \mathbb{R}^+$: Total amount dropped
- $\tau_i \in \Omega$: Token address
- $S_i \in \mathbb{R}^+$: Total staked at drop time
- $T_i \in \mathbb{R}^+$: Claim deadline (timestamp)
- $C_i \in \mathbb{R}^+$: Total amount already claimed from this drop

3.1 Protocol Operations

RainDrop's operations are precisely specified in the following subsections.

3.1.1 Stake

The *stake* operation $\sigma : U \times \mathbb{R}^+ \to V \to V$ is defined as:

$$\sigma(u, n)(V) = \begin{cases} S_{total} \leftarrow S_{total} + n \\ S_{user}(u) \leftarrow S_{user}(u) + n \\ E_{user}(u) \leftarrow k \\ B_{user}(u, \tau_{stake}) \leftarrow B_{user}(u, \tau_{stake}) - n \\ B_{vault}(\tau_{stake}) \leftarrow B_{vault}(\tau_{stake}) + n \end{cases} \tag{3}$$

3.1.2 Unstake

The *unstake* operation $\mu : U \times \mathbb{R}^+ \to V \to V$ is defined as:

$$\mu(u, n)(V) = \begin{cases} S_{total} \leftarrow S_{total} - n \\ S_{user}(u) \leftarrow S_{user}(u) - n \\ B_{user}(u, \tau_{stake}) \leftarrow B_{user}(u, \tau_{stake}) + n \\ B_{vault}(\tau_{stake}) \leftarrow B_{vault}(\tau_{stake}) - n \end{cases} \tag{4}$$

Preconditions: $S_{user}(u) \geq n$ (user has sufficient staked balance)

3.1.3 Drop

The *drop* operation $\delta : U \times \mathbb{R}^+ \times \Omega \times \mathbb{R}^+ \to V \to V$ is defined as:

$$\delta(u, A, \tau, T)(V) = \begin{cases} D \leftarrow D \cup \{(A, \tau, S_{total}, T, 0)\} \\ k \leftarrow k + 1 \\ \forall u' \in U : H(u', k) \leftarrow 0 & \text{initialize claim history for new drop} \\ B_{user}(u, \tau) \leftarrow B_{user}(u, \tau) - A \\ B_{vault}(\tau) \leftarrow B_{vault}(\tau) + A \end{cases} \tag{5}$$

3.1.4 Claim

The *claim* operation $\gamma : U \times \mathbb{N} \to V \to \mathbb{R}^+$ is defined as:

$$\gamma(u, i)(V) = \begin{cases} H(u, i) \leftarrow 1 \\ C_i \leftarrow C_i + q \\ B_{user}(u, \tau_i) \leftarrow B_{user}(u, \tau_i) + q \\ B_{vault}(\tau_i) \leftarrow B_{vault}(\tau_i) - q \end{cases} \tag{6}$$

where:

$$q = \begin{cases} \frac{S_{user}(u) \times A_i}{S_i} & \text{if } i > E_{user}(u) \text{ and } H(u,i) = 0 \\ 0 & \text{otherwise} \end{cases} \tag{7}$$

3.1.5 Withdraw

The *withdraw* operation $\zeta : U \times \mathbb{N} \to V \to \mathbb{R}^+$ is defined as:

$$\zeta(u,i)(V) = \begin{cases} B_{user}(u,\tau_i) \leftarrow B_{user}(u,\tau_i) + R_i \\ B_{vault}(\tau_i) \leftarrow B_{vault}(\tau_i) - R_i \\ C_i \leftarrow C_i - R_i \end{cases} \tag{8}$$

where:

$$R_i = \begin{cases} A_i - C_i & \text{if } u = P_i \text{ and } t_{now} > T_i \\ 0 & \text{otherwise} \end{cases} \tag{9}$$

4 Related Work

The RainDrop protocol improves various reward distribution mechanisms. Below we mention and compare some of the most influential ones in the conception of RainDrop.

- **Hodlcoin:** hodlCoin [1] is a staking protocol that introduced the concept of unstaking fee and proved that the price of the staking token never decreases. Like RainDrop, hodlCoin's reward distribution is also efficient and proper, and hodlCoin is even significantly simpler than RainDrop. But, in hodlCoin, the tokens distributed as rewards must the same as the staked token. This limitation prevents cross-token incentivization, which RainDrop addresses by supporting arbitrary reward tokens.
- **Popular DEX Staking Contracts:** PancakeSwap's MasterChef v2 and SushiSwap's MasterChef v1 are widely adopted staking contracts that attempt to solve proportional reward distribution. However, these systems suffer from the gaming vulnerability mentioned in the introduction: users can unstake and re-stake with different accounts to manipulate their reward eligibility. RainDrop prevents this by binding reward eligibility to specific drop timestamps rather than dynamic staking status.
- **Other Staking Protocols:** The DeFi ecosystem includes numerous staking implementations including DappFactory, Team Finance, UNCX Network, UniCrypt, Geysr, Decubate, Add3, Houdini Swap Staking, Syncus, Yummy, GenesisDAO, and Synthetix. While each offers unique features, most suffer from either the gaming vulnerabilities described above or lack the flexibility to distribute arbitrary reward tokens. Synthetix's approach is particularly notable for its sophisticated reward calculation mechanisms, but it remains limited to protocol-specific tokens.

RainDrop's key innovation lies in combining temporal fairness (rewards only for pre-existing stakers) with cross-token flexibility and mathematical guarantees of proportional distribution.

5 Theorems

In this section, we prove that RainDrop distributes rewards efficiently and properly.

▶ **Theorem 1** (Constant-Time Operations). *All RainDrop protocol operations execute in constant time $O(1)$ with respect to the number of users and the number of historical drops.*

Proof. We analyze each operation's computational complexity:

- **Stake Operation σ:** Updates three state variables (S_{total}, $S_{user}(u)$, $E_{user}(u)$) using simple arithmetic operations and map lookups/updates, all of which are $O(1)$.
- **Unstake Operation μ:** Similar to stake, involves constant-time arithmetic and map operations.
- **Drop Operation δ:** Appends one element to the drops sequence D, increments counter k, and initializes tracking state. All operations are $O(1)$.
- **Claim Operation γ:** Computes a proportional share using arithmetic operations on stored values from the drop record. The key insight is that the formula $\frac{S_{user}(u) \times A_i}{S_i}$ uses only pre-stored values: $S_{user}(u)$ from the user map, and (A_i, S_i) from the immutable drop record d_i. No iteration over users or drops is required.
- **Withdraw Operation ζ:** Uses the running total C_i that is maintained incrementally during claim operations, avoiding the need to sum over all users. The remaining amount $R_i = A_i - C_i$ is computed in constant time.

Therefore, all operations achieve $O(1)$ complexity through careful state design and incremental maintenance of aggregate values. ◀

▶ **Theorem 2** (Proper Reward Distribution). *For any user u staking when drop d_i occurs, the user receives exactly their proportional share of the drop amount, and this share remains invariant under any sequence of actions by other users. Formally, if $S_{user}(u) > 0$ and $i > E_{user}(u)$ at the time of drop d_i, then for any sequence of actions $a_1, \ldots, a_k$ performed by other users, the claim amount satisfies:*

$$\gamma(u, i) = \frac{S_{user}(u) \times A_i}{S_i}$$

where $S_{user}(u)$ and S_i are the user's stake and total stake at the time drop d_i was created.

Proof. Proof by induction on the sequence of actions $a_1, \ldots, a_k$ performed after drop d_i:

- **Base case ($k = 0$):** Immediately after drop d_i is created, by definition of the claim operation:
$$\gamma(u, i) = \frac{S_{user}(u) \times A_i}{S_i}$$
where $S_{user}(u)$ and S_i are recorded at the time of drop creation.
- **Inductive step:** Assume the claim amount is preserved after actions $a_1, \ldots, a_k$. We show it remains preserved after action a_{k+1} by case analysis:

  - **Case 1 - Stake by another user $v \neq u$:** The stake operation updates S_{total} and $S_{user}(v)$ but does not modify the drop record $d_i = (A_i, \tau_i, S_i, T_i, C_i)$. Since $\gamma(u, i)$ depends only on the immutable values A_i, S_i, and the user's stake at drop time, it remains unchanged.
  - **Case 2 - Unstake by another user $v \neq u$:** Similar to Case 1, unstaking modifies current state but not the historical drop record d_i. The claim amount $\gamma(u, i)$ remains invariant.

- **Case 3 - Claim by another user $v \neq u$:** The claim operation is a pure function that computes a value from the vault state without modifying it. It does not affect drop records or other users' potential claims.
- **Case 4 - New drop by any user:** Creating a new drop d_j with $j \neq i$ does not modify the existing drop record d_i, preserving $\gamma(u, i)$.
- **Case 5 - Withdraw remaining by drop creator:** This operation only affects unclaimed tokens after the deadline and does not modify the drop record d_i or affect valid claims made before the deadline.

In all cases, the drop record d_i containing $(A_i, \tau_i, S_i, T_i, C_i)$ remains immutable, and user u's claim amount is preserved:

$$\gamma(u, i) = \frac{S_{user}(u) \times A_i}{S_i}$$

Therefore, users are guaranteed to receive exactly their proportional share of any drop they were eligible for, regardless of subsequent actions by other participants. ◀

6 Discussion

The RainDrop protocol provides a foundation for efficient and proper distribution of rewards to token holders. The temporal fairness mechanism prevents gaming of the system while encouraging long-term participation. The flexibility to distribute any fungible token as rewards opens possibilities for diverse incentive structures and community-driven tokenomics. The minimalism of RainDrop makes it suitable for composition with other protocols in decentralized systems. Most importantly, RainDrop's mathematical soundness and rigour paves the way for adoption of blockchain-based dividend distribution across the economy.

References

1 Joachim Zahnentferner. hodlCoin: A financial game. Cryptology ePrint Archive, Paper 2023/1029, 2023. URL: https://eprint.iacr.org/2023/1029.

A License



This work is licensed under the CreativeCommons Attribution-NonCommercial-NoDerivatives 4.0 International license[2]. For derivative and commercial uses, contact the authors first.

B Official Implementations and Deployments of RainDrop

https://github.com/StabilityNexus

[2] https://creativecommons.org/licenses/by-nc-nd/4.0/

C Official RainDrop Channels

- https://x.com/StabilityNexus
- https://discord.gg/7jS9qJNjJv

Fate Protocol: Perpetual Prediction Pools

Zahnentferner
The Stable Order

Anjali Jha
The Stable Order

Raj Jitendra Shah
The Stable Order

Abstract

This paper introduces the Fate Protocol, an approach to prediction markets that is novel in two ways: firstly, it uses the new concept of *prediction pools* instead of traditional order books; and, secondly, it is *perpetual*, with no expiration dates.

2012 ACM Subject Classification Applied computing → Economics; Theory of computation → Algorithmic mechanism design

Keywords and phrases blockchain, prediction markets, liquidity pools, decentralized finance

Acknowledgements We thank the community of the Stability Nexus for their feedback and support.

1 Introduction

Traditional prediction markets have fixed expiration dates and an order book approach that may suffer from insufficient liquidity. The Fate Protocol addresses these issues by introducing *perpetual prediction pools* that enable investing in and hedging against the future values of any variable that is a function of time (e.g. asset prices, weather variables, ...).

The Fate Protocol allows users to buy and sell *bullCoins* and *bearCoins*, depending on whether they believe that the value of the tracked variable will continue to go up or down, respectively. These coins are backed by reserves of the asset deposited by users who buy the coins. When the tracked variable goes up, a portion of the reserve backing the bearCoins is redistributed to the reserve backing the bullCoins. Conversely, when the tracked variable goes down, a portion of the reserve backing the bullCoins is redistributed to the reserve backing the bearCoins. A core innovation of the protocol lies in its path-independent redistribution mechanism, which ensures that final reserves depend solely on the initial and final values of the variable, regardless of its intermediate values. This mathematical property, combined with a zero-sum fee structure that rewards long-term participants, creates sustainable liquidity pools without traditional order book limitations.

This paper presents the protocol's mathematical foundations and mechanics, and proves the above-mentioned path independence property.

2 Protocol Description

The Fate Protocol allows users to mint and burn two dual *prediction tokens*: *bullCoins* ($B^{\uparrow}$) that appreciate when the tracked variable increases, or *bearCoins* ($B^{\downarrow}$) that appreciate when the variable decreases. Each token type is backed by dedicated reserves of a base fungible asset: $R^{\uparrow}$ and $R^{\downarrow}$, respectively. The protocol has three fundamental operations:

- **Token Purchase:** Users mint an amount of a prediction token of their choice by depositing an amount of the base asset into the corresponding reserve.
- **Token Redemption:** Users burn an amount of a prediction token to withdraw a proportional amount of the base asset from the respective reserve.

- **Variable Update:** Users may call an external oracle to update the variable value and redistribute a portion of the losing token's reserve to the winning token's reserve. Furthermore, a variable update is always called immediately before any token purchase or redemption, to ensure that purchase or redemption occurs with a token price that reflects that latest variable value.

3 Protocol Specification

The protocol's state is a tuple $(R^{\uparrow}, R^{\downarrow}, S^{\uparrow}, S^{\downarrow}, p)$, where:

- $R^{\uparrow}$ and $R^{\downarrow}$ are the bullCoin and bearCoin reserves.
- $S^{\uparrow}$ and $S^{\downarrow}$ are the bullCoin and bearCoin supplies.
- p is the variable value.

The protocol has a single fixed parameter: the fee ϕ. The prices of bullCoins and bearCoins are defined by $P^{\uparrow} = R^{\uparrow}/S^{\uparrow}$ and $P^{\downarrow} = R^{\downarrow}/S^{\downarrow}$, respectively. The *total reserve* is defined by $R = R^{\uparrow} + R^{\downarrow}$.

The following subsections mathematically define the behavior of each protocol operation.

3.1 Token Purchase

For a deposit of an amount n of the base asset, the protocol calculates the amount m of prediction tokens to be received and updates the state as follows.

For bullCoin purchases:

$$m^{\uparrow} = \frac{(1-\phi)n}{P^{\uparrow}} \tag{1}$$

$$R^{\uparrow}_{\text{new}} \leftarrow R^{\uparrow} + n \tag{2}$$

$$S^{\uparrow}_{\text{new}} \leftarrow S^{\uparrow} + m^{\uparrow} \tag{3}$$

For bearCoin purchases:

$$m^{\downarrow} = \frac{(1-\phi)n}{P^{\downarrow}} \tag{4}$$

$$R^{\downarrow}_{\text{new}} \leftarrow R^{\downarrow} + n \tag{5}$$

$$S^{\downarrow}_{\text{new}} \leftarrow S^{\downarrow} + m^{\downarrow} \tag{6}$$

3.2 Token Redemption

For a redemption of n prediction tokens, the protocol calculates the amount q of base asset to be received and updates the state as follows:

For bullCoin redemptions:

$$z = n \cdot P^{\uparrow} \tag{7}$$

$$m = (1-\phi)z \tag{8}$$

$$R^{\uparrow}_{\text{new}} \leftarrow R^{\uparrow} - z \tag{9}$$

$$R^{\downarrow}_{\text{new}} \leftarrow R^{\downarrow} + \phi z \tag{10}$$

$$S^{\uparrow}_{\text{new}} \leftarrow S^{\uparrow} - n \tag{11}$$

For bearCoin redemptions:

$$z = n \cdot P^{\downarrow} \tag{12}$$
$$m = (1 - \phi)z \tag{13}$$
$$R^{\downarrow}_{\text{new}} \leftarrow R^{\downarrow} - z \tag{14}$$
$$R^{\uparrow}_{\text{new}} \leftarrow R^{\uparrow} + \phi z \tag{15}$$
$$S^{\downarrow}_{\text{new}} \leftarrow S^{\downarrow} - n \tag{16}$$

3.3 Variable Update

When time passes from t to $t+1$ and the tracked variable changes from p_t to p_{t+1}, the reserves are updated using the following two-step algorithm:

Step 1: Update

$$Q^{\uparrow}_{t+1} = R^{\uparrow}_t \cdot \frac{p_{t+1}}{p_t} \tag{17}$$
$$Q^{\downarrow}_{t+1} = R^{\downarrow}_t \cdot \frac{p_t}{p_{t+1}} \tag{18}$$

Step 2: Normalize

$$R^{\uparrow}_{t+1} = \frac{Q^{\uparrow}_{t+1}}{Q^{\uparrow}_{t+1} + Q^{\downarrow}_{t+1}} \cdot R_t \tag{19}$$
$$R^{\downarrow}_{t+1} = \frac{Q^{\downarrow}_{t+1}}{Q^{\uparrow}_{t+1} + Q^{\downarrow}_{t+1}} \cdot R_t \tag{20}$$

In the equations above, Step 1 computes intermediate values Q, which are temporary rescaled reserve values based on relative price changes. These serve as a transitional step used to capture how the reserves would shift if left unbounded. Step 2 then normalizes these intermediate values to ensure the updated reserves remain within the total constraint of the original reserve R_t. This prevents the system from deviating from its initial total reserve while still adjusting proportionally to price dynamics.

This two-step process can be simplified to a single step process by combining equation 19 with equation 17 and equation 20 with equation 18, resulting in the following equations:

$$R^{\uparrow}_{t+1} = R_t \cdot \frac{R^{\uparrow}_t \cdot (p_{t+1}/p_t)}{R^{\uparrow}_t \cdot (p_{t+1}/p_t) + R^{\downarrow}_t \cdot (p_t/p_{t+1})} \tag{21}$$
$$R^{\downarrow}_{t+1} = R_t \cdot \frac{R^{\downarrow}_t \cdot (p_t/p_{t+1})}{R^{\uparrow}_t \cdot (p_{t+1}/p_t) + R^{\downarrow}_t \cdot (p_t/p_{t+1})} \tag{22}$$

4 Example

To illustrate the Fate Protocol in action, consider a scenario where the tracked variable is the price of Bitcoin (BTC), and Ethereum (ETH) is the base reserve asset. Suppose at time $t = 0$, the BTC price is $p_0 = 100$ (in some unit), and the protocol is initialized with equal reserves and token supplies:

- Bull reserve: $R^{\uparrow} = 1$ ETH, $S^{\uparrow} = 1$ bullCoins
- Bear reserve: $R^{\downarrow} = 1$ ETH, $S^{\downarrow} = 1$ bearCoins

Each bullCoin and bearCoin is thus initially worth 1 ETH.

Alice buys bullCoins.

Alice deposits 50 ETH into the bull reserve. Assuming a protocol fee of $\phi = 1\%$, she receives:

$$m^{\uparrow} = \frac{(1-\phi)\cdot 50}{P^{\uparrow}} = \frac{0.99 \cdot 50}{1} = 49.5$$

bullCoins. The bull reserve and supply update to:

$$R^{\uparrow} = 50 \text{ ETH}, \quad S^{\uparrow} = 49.5 \text{ bullCoins}$$

Bob buys bearCoins.

Bob deposits 50 ETH into the bear reserve and receives 49.5 bearCoins under the same fee and price conditions. The bear reserve and supply update to:

$$R^{\downarrow} = 50 \text{ ETH}, \quad S^{\downarrow} = 49.5 \text{ bearCoins}$$

Price goes up.

The oracle updates the BTC price to $p_1 = 110$. According to the price update mechanism:

$$Q^{\uparrow} = 50 \cdot \frac{110}{100} = 55, \quad Q^{\downarrow} = 50 \cdot \frac{100}{110} \approx 45.45$$

These are normalized to maintain the total reserve of 100 ETH:

$$R^{\uparrow}_{\text{new}} = 100 \cdot \frac{55}{55 + 45.45} \approx 54.75$$

$$R^{\downarrow}_{\text{new}} = 100 - 54.73 = 45.27$$

New prices:

$$P^{\uparrow} = \frac{54.73}{49.5} \approx 1.1, \quad P^{\downarrow} = \frac{45.27}{49.5} \approx 0.914$$

Alice redeems her bullCoins.

Alice holds 49.5 bullCoins. If she redeems them now, she receives:

$$(1-\phi) \cdot 49.5 \cdot 1.1 \approx 0.99 \cdot 49.5 \cdot 1.1 \approx 53.9 \text{ ETH}$$

This is a profit of approximately 3.9 ETH over her original deposit, reflecting the upward price movement.

Further price movement.

Suppose the BTC price later falls to $p_2 = 90$. The reserves update accordingly:

$$Q^{\uparrow} = 54.73 \cdot \frac{90}{110} \approx 44.77, \quad Q^{\downarrow} = 45.27 \cdot \frac{110}{90} \approx 55.33$$

Normalize:

$$R^{\uparrow}_{\text{new}} = 100 \cdot \frac{44.77}{44.77 + 55.33} \approx 44.725, \quad R^{\downarrow}_{\text{new}} = 100 - 44.725 = 55.275$$

New token prices:

$$P^{\uparrow} \approx \frac{44.725}{49.5} \approx 0.903, \quad P^{\downarrow} \approx \frac{55.275}{49.5} \approx 1.116$$

Now bearCoin holders like Bob are in profit, and bullCoin holders would incur a loss if redeeming.

Observation.

This example shows how users interact with the protocol by purchasing or redeeming bull/bearCoins, and how oracle-driven price updates affect token values and reserve balances. BullCoin holders benefit from upward price movements, while BearCoin holders profit when prices fall. By Theorem 4, the final state depends only on the start and end prices, not the intermediate path taken.

5 Theorems

The Fate protocol enjoys four important theorems. The first one shows that variable updates preserve the total reserve. This is important, since it shows that base assets will neither disappear (be burned) nor appear from nothing (be minted) as a result of variable updates.

▶ **Theorem 1** (Reserve Preservation). *If a variable update action occurs at time t, then $R_{t+1} = R_t$.*

Proof. The variable update at time t consists of two steps:

Step 1 (Update):

$$Q^{\uparrow}_{t+1} = R^{\uparrow}_t \cdot \frac{p_{t+1}}{p_t},$$
$$Q^{\downarrow}_{t+1} = R^{\downarrow}_t \cdot \frac{p_t}{p_{t+1}}.$$

Step 2 (Normalization):

$$R^{\uparrow}_{t+1} = R_t \cdot \frac{Q^{\uparrow}_{t+1}}{Q^{\uparrow}_{t+1} + Q^{\downarrow}_{t+1}},$$
$$R^{\downarrow}_{t+1} = R_t \cdot \frac{Q^{\downarrow}_{t+1}}{Q^{\uparrow}_{t+1} + Q^{\downarrow}_{t+1}}.$$

Summing the two normalized reserves yields:

$$\begin{aligned} R_{t+1} &= R^{\uparrow}_{t+1} + R^{\downarrow}_{t+1} \\ &= R_t \cdot \left(\frac{Q^{\uparrow}_{t+1}}{Q^{\uparrow}_{t+1} + Q^{\downarrow}_{t+1}} + \frac{Q^{\downarrow}_{t+1}}{Q^{\uparrow}_{t+1} + Q^{\downarrow}_{t+1}} \right) \\ &= R_t \cdot \frac{Q^{\uparrow}_{t+1} + Q^{\downarrow}_{t+1}}{Q^{\uparrow}_{t+1} + Q^{\downarrow}_{t+1}} \\ &= R_t. \end{aligned}$$

Thus, the total reserve remains unchanged by the variable update step:

$$R_{t+1} = R_t.$$

◀

The second and third theorems show that Fate behaves as a prediction market, rewarding bullCoin holders when the variable increases and rewarding bearCoin holders when the variable decreases. This happens indirectly, because the prices increase. To realize these gains, holders must redeem their prediction tokens.

▶ **Theorem 2** (Upward Prediction Adequacy)**.** *If a variable update action occurs at time t and $p_{t+1} > p_t$, then $P^{\uparrow}_{t+1} > P^{\uparrow}_t$ and $P^{\downarrow}_{t+1} < P^{\downarrow}_t$.*

Proof. Assume $p_{t+1} > p_t$, which implies $\frac{p_{t+1}}{p_t} > 1$ and $\frac{p_t}{p_{t+1}} < 1$.

From the variable update mechanism, we have:

Step 1 (Update):

$$Q^{\uparrow}_{t+1} = R^{\uparrow}_t \cdot \frac{p_{t+1}}{p_t} \tag{23}$$

$$Q^{\downarrow}_{t+1} = R^{\downarrow}_t \cdot \frac{p_t}{p_{t+1}} \tag{24}$$

Step 2 (Normalization):

$$R^{\uparrow}_{t+1} = R_t \cdot \frac{Q^{\uparrow}_{t+1}}{Q^{\uparrow}_{t+1} + Q^{\downarrow}_{t+1}} \tag{25}$$

$$= R_t \cdot \frac{R^{\uparrow}_t \cdot \frac{p_{t+1}}{p_t}}{R^{\uparrow}_t \cdot \frac{p_{t+1}}{p_t} + R^{\downarrow}_t \cdot \frac{p_t}{p_{t+1}}} \tag{26}$$

$$= R_t \cdot \frac{R^{\uparrow}_t \cdot \frac{p^2_{t+1}}{p^2_t}}{R^{\uparrow}_t \cdot \frac{p^2_{t+1}}{p^2_t} + R^{\downarrow}_t} \tag{27}$$

Similarly:

$$R^{\downarrow}_{t+1} = R_t \cdot \frac{R^{\downarrow}_t}{R^{\uparrow}_t \cdot \frac{p^2_{t+1}}{p^2_t} + R^{\downarrow}_t} \tag{28}$$

To prove $P^{\uparrow}_{t+1} > P^{\uparrow}_t$, we need to show $R^{\uparrow}_{t+1} > R^{\uparrow}_t$ (since token supplies are unchanged during variable updates).

We need to verify:

$$R_t \cdot \frac{R^{\uparrow}_t \cdot \frac{p^2_{t+1}}{p^2_t}}{R^{\uparrow}_t \cdot \frac{p^2_{t+1}}{p^2_t} + R^{\downarrow}_t} > R^{\uparrow}_t \tag{29}$$

Since $R_t = R^{\uparrow}_t + R^{\downarrow}_t$, this becomes:

$$(R^{\uparrow}_t + R^{\downarrow}_t) \cdot \frac{R^{\uparrow}_t \cdot \frac{p^2_{t+1}}{p^2_t}}{R^{\uparrow}_t \cdot \frac{p^2_{t+1}}{p^2_t} + R^{\downarrow}_t} > R^{\uparrow}_t \tag{30}$$

Cross-multiplying (all terms are positive):

$$(R^{\uparrow}_t + R^{\downarrow}_t) \cdot R^{\uparrow}_t \cdot \frac{p^2_{t+1}}{p^2_t} > R^{\uparrow}_t \cdot \left(R^{\uparrow}_t \cdot \frac{p^2_{t+1}}{p^2_t} + R^{\downarrow}_t \right) \tag{31}$$

$$R^{\uparrow}_t \cdot R^{\downarrow}_t \cdot \frac{p^2_{t+1}}{p^2_t} > R^{\uparrow}_t \cdot R^{\downarrow}_t \tag{32}$$

$$\frac{p^2_{t+1}}{p^2_t} > 1 \tag{33}$$

Since $p_{t+1} > p_t$, we have $\frac{p^2_{t+1}}{p^2_t} > 1$, so $P^{\uparrow}_{t+1} > P^{\uparrow}_t$.

For bearCoins, we need to show $R^{\downarrow}_{t+1} < R^{\downarrow}_{t}$:

$$(R^{\uparrow}_{t} + R^{\downarrow}_{t}) \cdot \frac{R^{\downarrow}_{t}}{R^{\uparrow}_{t} \cdot \frac{p^2_{t+1}}{p^2_{t}} + R^{\downarrow}_{t}} < R^{\downarrow}_{t} \tag{34}$$

Cross-multiplying:

$$(R^{\uparrow}_{t} + R^{\downarrow}_{t}) \cdot R^{\downarrow}_{t} < R^{\downarrow}_{t} \cdot \left(R^{\uparrow}_{t} \cdot \frac{p^2_{t+1}}{p^2_{t}} + R^{\downarrow}_{t} \right) \tag{35}$$

$$R^{\uparrow}_{t} \cdot R^{\downarrow}_{t} < R^{\downarrow}_{t} \cdot R^{\uparrow}_{t} \cdot \frac{p^2_{t+1}}{p^2_{t}} \tag{36}$$

$$1 < \frac{p^2_{t+1}}{p^2_{t}} \tag{37}$$

Since $p_{t+1} > p_t$, we have $\frac{p^2_{t+1}}{p^2_{t}} > 1$, so $P^{\downarrow}_{t+1} < P^{\downarrow}_{t}$.
Therefore, when $p_{t+1} > p_t$, bullCoin prices increase and bearCoin prices decrease. ◀

▶ **Theorem 3** (Downward Prediction Adequacy). *If a variable update action occurs at time t and $p_{t+1} < p_t$, then $P^{uparrow}_{t+1} < P^{uparrow}_{t}$ and $P^{downarrow}_{t+1} > P^{downarrow}_{t}$.*

Proof. Analogous and dual to the proof of Theorem 2 ◀

Finally, the last theorem shows *path independence*: in a sequence of variable updates, the outcome depends exclusively on the initial protocol state, the initial variable value, and the final variable value, rendering intermediate variable movements irrelevant. This property ensures that the final outcome is independent of when variable updates and oracle consultations are triggered. It ensures that participants cannot try to earn more simply by calling variable update at specific intermediary times.

▶ **Theorem 4** (Path Independence). *Consider a variable value path:*

$$p_0 \to p_1 \to \cdots \to p_t \tag{38}$$

where only variable updates occur (no token purchases or redemptions). Then the final reserves $R^{\uparrow}_{t}$ and $R^{\downarrow}_{t}$ depend only on:

- *Initial protocol state $(R^{\uparrow}_{0}, R^{\downarrow}_{0}, S^{\uparrow}_{0}, S^{\downarrow}_{0}, p_0)$,*
- *Final variable value p_t.*

Intermediate variable values $p_1, \ldots, p_{t-1}$ have no effect on the final reserves, and the token supplies remain unchanged: $S^{\uparrow}_{t} = S^{\uparrow}_{0}$ and $S^{\downarrow}_{t} = S^{\downarrow}_{0}$.

Proof. Since variable updates only affect reserves $(R^{\uparrow}, R^{\downarrow})$ and the variable value p, while preserving supplies $(S^{\uparrow}, S^{\downarrow})$, we focus on the reserve transformations.

By recursively applying equations (21) and (22) to unfold definitions of $R^{\uparrow}_{k}$ and $R^{\downarrow}_{k}$ for all k and simplifying the resulting expanded equations, we obtain:

$$R^{\uparrow}_{t} = R_0 \cdot \frac{R^{\uparrow}_{0} \cdot \prod_{i=1}^{t} \frac{p_i}{p_{i-1}}}{R^{\uparrow}_{0} \cdot \prod_{i=1}^{t} \frac{p_i}{p_{i-1}} + R^{\downarrow}_{0} \cdot \prod_{i=1}^{t} \frac{p_{i-1}}{p_i}} \tag{39}$$

$$= R_0 \cdot \frac{R^{\uparrow}_{0} \cdot (p_t/p_0)}{R^{\uparrow}_{0} \cdot (p_t/p_0) + R^{\downarrow}_{0} \cdot (p_0/p_t)} \tag{40}$$

Since $\prod_{i=1}^{t} \frac{p_i}{p_{i-1}} = \frac{p_t}{p_0}$, the final reserves depend only on the initial state components $R_0^{\uparrow}$, $R_0^{\downarrow}$, p_0, and the final variable value p_t, regardless of the intermediate values $p_1, \ldots, p_{t-1}$.

Therefore, any path from initial state $(R_0^{\uparrow}, R_0^{\downarrow}, S_0^{\uparrow}, S_0^{\downarrow}, p_0)$ to final variable value p_t yield the same final state:

$$R_t^{\uparrow} = R_0 \cdot \frac{R_0^{\uparrow} \cdot (p_t/p_0)}{R_0^{\uparrow} \cdot (p_t/p_0) + R_0^{\downarrow} \cdot (p_0/p_t)} \tag{41}$$

$$R_t^{\downarrow} = R_0 \cdot \frac{R_0^{\downarrow} \cdot (p_0/p_t)}{R_0^{\uparrow} \cdot (p_t/p_0) + R_0^{\downarrow} \cdot (p_0/p_t)} \tag{42}$$

$$S_t^{\uparrow} = S_0^{\uparrow} \tag{43}$$

$$S_t^{\downarrow} = S_0^{\downarrow} \tag{44}$$

$$p_t = p_t \tag{45}$$

◀

6 Related Work

The Fate Prediction Pool Protocol is inspired by the hodlCoin Staking Protocol [**?**], which in turn evolved from the Djed Stablecoin Protocol [**?**, **?**]. In hodlCoin, there is a single reserve for a single token, and there is only a fee for redemptions. Fate generalizes this to two reserves for two tokens, introduces a fee for purchases, and changes the beneficiary of the redemption fee (making it be the dual token's reserve instead of the reserve of the token being redeemed). Finally, most crucially, it introduces the variable update action, which makes the tokens, and their reserves, compete with each other, depending on the evolution of the value of an external oracle variable.

In the same way that liquidity pools [**?**, **?**] introduced the concept of pools to overcome scalability, liquidity and cost limitations of order books in the context of on-chain decentralized exchanges, the Fate Protocol introduces the concept of prediction pools to overcome similar limitations in the context of on-chain prediction markets.

7 Discussion and Future Work

Due to its simplicity, adequacy and generality, the Fate Protocol has the potential to become a new composable primitive in decentralized finance. Furthermore, since any asset can be used as a base asset for prediction pools, Fate can bring more utility for such assets, increasing their total value locked (TVL) and reducing their circulating supply.

Fate's main intended use case is as a perpetual prediction market for any variable with values that change over time. As such, Fate can be used to manage and insure against risks related to weather data (e.g. temperature, precipitation, ...), asset prices, macro-economic indicators, ... Particularly, Fate can potentially capture one key use case for decentralized stablecoins such as Djed [**?**, **?**] and Gluon [**?**]: holders of crypto-backed stablecoins often buy stablecoins with the underlying backing crypto asset to hedge against the risk that the price of this asset will fall; with Fate, they could achieve a similar effect by purchasing bearCoins associated with the price of this asset.

In our current and planned future work, we are investigating other variable update mechanisms, besides the one described in Subsection 3.3. This is likely going to lead to a family of Fate protocols. Moreover, although Fate currently depends on an external oracle, we are exploring the possibility of making Fate itself serve as an oracle.

References

A License



This work is licensed under the CreativeCommons Attribution-NonCommercial-NoDerivatives 4.0 International license[1].

For derivative and commercial uses, please contact the authors first.

B Official Implementations and Deployments of Fate Protocol

https://github.com/StabilityNexus/

C Official Fate Protocol Channels

- https://x.com/StabilityNexus
- https://discord.gg/7jS9qJNjJv

[1] https://creativecommons.org/licenses/by-nc-nd/4.0/

Succinct Non-Interactive Share Proofs in Proof-of-Work Cryptocurrencies

Kirat Singh
Lithos Protocol, United States of America

Abstract

Proof-of-work (PoW) based consensus protocols rely on the process of "mining" to secure their blockchain. Although mining is a crucial process for the security of such blockchains, there are few ways to calculate the amount of work a miner has done in a trustless and efficient way. Such an ability would allow for the creation of decentralized mining pools, along with other processes that integrate mining into on-chain protocols.

In this work, we construct a new primitive known as a Non-Interactive Share Proof (NISP), which allows for efficient calculation of a miner's work. Our construction functions using a worst-case constant space complexity, regardless of the total number of shares of work that a miner submits. We pull heavy inspiration from previous works on Non-Interactive Proofs of Proof of Work (NiPoPoWs).

Using NISPs, one can prove that a miner performed work using smart contracts on programmable blockchains such as Ergo and Alephium. Moreover, we show that our construction is blockchain agnostic, meaning it may even be used to prove work was performed on non-programmable PoW blockchains such as Bitcoin and Flux. We later show how our construction may be applied to different decentralized applications outside of decentralized mining pools.

2012 ACM Subject Classification Security and privacy → Logic and verification

Keywords and phrases Cryptocurrency, proof-of-work, mining, hash, target, miner, difficulty

Digital Object Identifier 10.4230/LIPIcs.STABILITY.2025.01

1 Background

Before delving into NISPs, it is important to explain the processes behind PoW mining, along with how mining pools work, and common terms used in this area of research. To begin, we start with the PoW process that is used by many blockchains today. PoW blockchains define a certain target T, a hash function H and some value w which is derived from a potential block candidate. Valid solutions must satisfy the inequality $H(w) < T$ in order to be accepted onto the blockchain [3]. To efficiently satisfy this inequality, miners must constantly apply H to different values of w until a solution is found. It should be noted that the target may sometimes be represented as *difficulty*, which is just the average number of hashes needed to find a block at a certain target.

The rate at which H may be applied to w is known as a miner's hashrate. Hashrate is limited by the physical computation power of a miner's hardware, and so cannot be artificially increased. Miners with higher hashrates may perform more computations in the same amount of time compared to miners with lower hashrates. Hashrate is therefore directly related to the chances of a miner satisfying the inequality and submitting a solution onto the blockchain within a certain amount of time.

Due to the random nature of PoW mining, as well as the constant adjustment of T in order to reach stable average block times, miners with lower hashrates are unable to consistently find blocks in a short time span. To solve this problem, mining pools have been created to combine the resources of multiple miners in order to find blocks quicker [2]. Mining pools then distribute these block rewards to miners proportionally according to the amount of work each miner performed while searching for solutions. In traditional mining pools, each miner's work is measured using shares, which are applications of H which


© Kirat Singh;
licensed under Creative Commons License CC-BY 4.0



do not satisfy the inequality given by the blockchain. Shares instead satisfy the inequality $H(w) < T_p$, where T_p is a special target set by the mining pool such that $T_p > T$.

Share Processing

Shares are constantly being submitted by miners on pools in order to prove that valid work was performed by them. These shares are then incorporated into different share processing algorithms which output a number which shall now be referred to as the share score. The share score provides a simple scalar value which allows the pool to derive miner payouts by taking the proportion of the miner's share score to the share score of the entire pool.

One common share processing algorithm is known as the Pay-Per-Share algorithm (PPS) [2]. The PPS algorithm pays miners proportionally according to the number of shares they submitted divided by the pool target T_p. Specifically, we define the equation $s = N/T_p$ where s represents a miner's share score, N represents the number of valid shares submitted by the miners, and T_p represents the pool target given to the miner by the pool. Although in many cases T_p may be a constant set by the pool, some pools incorporate "variable difficulty" mining to allow for efficient share submissions that lower network load. We have included T_p into the equation to show that miners working on shares with a lower target are given a higher score per share submission.

The PPS algorithm is used by many mining pools today. However, one strict disadvantage of it is that the algorithm is susceptible to pool hopping attacks. To prevent such attacks, alternative algorithms such as Pay-Per-Last-N-Shares (PPLNS) and Pay-Per-Last-N-Timeshares (PPLNT) are used [2]. Both algorithmsusing some window W in which submitted shares may be used in processing. For the PPLNS algorithm, W is a constant value representing the last N shares submitted to the pool before a certain block was found. In PPLNT, W is a set period of time before a certain block is found. In both of these algorithms, shares must be within W in order to be processed into a share score by the algorithm. A miner's share score for both of these algorithms can be derived by the equation $s = N_W/T_p$, where N_W represents the miner's shares present within the window.

2 Initial Construction

It is apparent from the above share processing algorithms that share scores are usually some proportion of the number of shares divided by the pool target. Although this works well for traditional mining pools, such an equation would struggle to work in a decentralized setting. For one, data storage is limited on programmable blockchains, having a miner submit the total number of shares created while mining is not viable. High-target shares could easily become hundreds of thousands of bytes of data to parse. However, submitting a small amount of low-target shares can lead to a large variability in profits, which would defeat the purpose of the mining pool itself.

To create functional and secure NISPs, we must look to other places to draw inspiration from. Specifically, let us analyze NiPoPoWs as proofs that achieve similar levels of succinctness and security in PoW blockchains.

NiPoPoW Background

Non-interactive Proofs-of-Proof-of-Work represent succinct proofs that may be generated from certain PoW blockchains, and which may prove various predicates about the blockchain itself. NiPoPoWs function by using blocks with especially low targets (called *super-blocks*) as

checkpoints to show that work was done on the blockchain [4]. Specifically, these proofs focus on the property of *goodness*, which is the tendency of block hashes on an honestly-mined blockchain to follow a their expected mean in relation to their difficulty. Generally, the distribution of super-blocks must be sufficiently good for a NiPoPoW to be considered valid. In the case of a malicious adversary, "certificates of badness" may be used to justify why a certain NiPoPoW may lack goodness.

Although the problems that NiPoPoWs attempt to solve may differ from our own goals with NISPs, it is evident that the properties of PoW that are used in the prior setting can also apply to our own. Specifically, we can see that:

- PoW blockchains and PoW Mining Pools both have a threshold target for valid hashes.
- Mining pools operating on a certain blockchain must ensure their miners use the hashing function H in order to create valid block solutions.
- Applying H to w ideally produces a random value
- Each iteration of a unique nonce over H can be considered an independent Bernoulli Trial, regardless of the threshold target used.

Given these properties, and in order to ensure the succinctness of our share proofs, we shall focus our attention onto goodness. Specifically, we wish to evaluate a set of shares based on how good the set is and whether or not it constitutes a valid, target-based binomial distribution.

Goodness

Evaluating goodness in the context of a decentralized mining pool may actually be easier versus the setting of a live blockchain. Attempts to attack goodness in NiPoPoWs all require an adversary who is withholding super-blocks and making the blockchain inherently bad, *i.e*, not good. No such problem exists here, as each miner is submitting their own set of hashes that they independently computed while performing work. Moreover, Lemmas 2 - 5 of Kiayias et. al prove that an honest miner has an overwhelming probability of creating a set of hashes which are good [4].

With this in mind, we propose a share proof that is evaluating a miner's work based on the goodness of their share set, rather than the number of shares submitted. Moreover, we shall ensure succinctness by extending the concept of super-blocks to *super-shares*, which shall represent shares whose value is sufficiently lower than the threshold target given by the pool. We'll similarly use the idea of *levels* to represent subsets of super-shares.

2.1 Model

For clarity, we will now describe our model for miners performing PoW mining on a decentralized mining pool. To begin, we assume that all actors in our mining pool are *rational*. Miners on the pool are constantly mining, and all miner's are assumed to have stable hashrates with few fluctuations. Everytime a block is found by a member of the pool, all miners in the pool must submit a share proof to a smart contract which then evaluates the proof. Each new block find on the pool constitutes a new *round* in our protocol. Each miner is therefore a *prover*, while our ideal smart contract is a *verifier* which is independently validating the share proofs given to it. If a miner's share proof is considered invalid, they gain no reward for the work they performed.

All miners in the pool are producing shares with a personal maximum target τ. τ can be seen as the threshold that divides hashes from shares, similar to how the blockchain target T divides hashes from blocks. For a hash to be considered a share, it must satisfy the inequality

$H(w) < \tau$. Shares created in this way are constantly being placed into some share set S. Upon the start of a new round, miners will submit some share proof P to the verifier in order to receive their mining rewards. The value of τ is set by each miner as they see fit, and is submitted in conjunction with P. P itself simply represents a collection of super-shares derived from S. In order to ensure consistency of mining rewards, we propose that in an ideal scenario, two miners mining with the same hashrate should each have equal values of τ submitted with their share proofs.

Using Super-Shares

We shall adapt the idea of super-shares so that S_i represents all super-shares at level i in the share set S. In our model, miners will be aiming to prove that they performed the work of creating N shares whose hash value is less than the target τ. We define a level i to be a subset of shares in S such that for each share s in S_i, the value of s conforms to the inequality $s < \tau \times (i/N)$. In addition, the shares submitted in P must be made within some window of time W seconds before the end of the round. The addition of W ensures that miners may not submit old share proofs that have no relevance to the work currently being performed by the pool.

In order to ensure the succinctness of our share proofs, we will use a security parameter μ, which represents the maximum number of shares which may be present in the share proof. W, μ and N will be global parameters that all miners in the pool must follow.

For a share proof P to be considered valid, the number of shares present in the proof must be less than or equal to μ. In addition, P may only be considered valid if all shares present in the proof are at some constant level i. To ensure that goodness is achieved, a proof P at level i must have exactly i shares inside of it. These constraints mean that the maximum size of a proof P is μ shares, and that the maximum target for each share in a proof of length μ is $\tau \times (\mu/N)$. They also show that μ acts not only as a security parameter ensuring succinctness, but also as the maximum level of shares that a miner may submit.

We can now finally say that the goal of each miner on the pool is to mine N shares and place them into the share set S. Upon the end of a round, miners will find the best subset of super-shares in S, and form the proof P. For P to be valid, each super-share s in a proof of length i must satisfy the inequality $s < \tau \times (i/N)$. To ensure succinctness, we constrain i to the domain $1 \leq i \leq \mu$. We can therefore say that miners here must only prove goodness at a single level i in the entire share set S in order to have a valid proof. Finally, we add the constraint that each share s must have been made within a window of time W seconds before the end of the round.

Evaluation

From the above construction, we can see that goodness of the share set is evaluated by having a miner submit a set of sufficiently low target super-shares which are all existing at some level i where $1 \leq i \leq \mu$. It is also apparent that our share proofs have an $O(1)$ space complexity, due to the limit on data size that μ is enforcing. It is clear then that this model is able to achieve both succinctness and proper evaluation of goodness. As one final addition, we shall create a method to convert our share proofs into share scores which may be used to award miners proportions of the block reward. The equation to find a miner's share score given a valid share proof is simply $2^{256}/\tau$. The constant 2^{256} is derived from the maximum value of a 256-bit hash (which blockchains such as Bitcoin and Ergo employ in their PoW algorithms). Keen observers may notice that this in fact the equation to calculate *difficulty*,

which can be considered a different representation than the target. We use difficulty here for it's cleaner representation of the target. We omit N unlike previous share score equations, due to N being a protocol-wide constant.

As shown above, the most important variable affecting a miner's reward is τ. Although this initial model correctly evaluates whether or not an honest miner is performing work with a share target threshold at τ, there is an issue here which jeopardizes the consistency of payouts. Because τ is submitted in addition to P, adversarial miners may simply use their lowest target hash to represent P. Accordingly, these adversaries may then adjust τ so as to satisfy the above goodness constraints, while still earning more than honest miners working at the same hashrate as them. In this way, dynamic adjustment of τ after the end of a round allows two miners with the same hashrate to obtain wildly different payouts. We will henceforth refer to attacks of this nature as *target adjustment attacks*.

3 Optimizing τ

In this section, we aim to append our initial construction so as to make it resistant to adversaries aiming to maximize profits via target adjustment attacks. Specifically, we can add a simple change which prevents adjustment of τ after a round ends. To achieve this goal, we now require that all miners on the pool must make an initial commitment to τ, such that the τ submitted with P must remain constant for some minimum number of rounds r. Each time a miner wishes to change their value of τ, they must wait at least one round for their update to take affect. In addition, every update to τ requires that a miner must wait r rounds before submitting a new commitment to their τ value. In the context of a smart-contract verifier, miners will be making a cryptographic commitment to their value of τ via a pool-wide authenticated dictionary, such as an AVL+ tree.

The affects of this change are plain to see, miners can no longer freely adjust τ to maximize rewards based on their lowest target shares. Instead, miners now receive share scores based on their commitment for at least $r + 1$ rounds before being able to change their τ value. Let us now analyze how our appended construction functions when a miner with some arbitrary, stable hashrate commits to different values of τ.

Idealized τ

We first propose that for any hashrate h, there exists a corresponding value of τ which will on average produce N shares within the window of time W. This ties back into our previous statement that two miners with the same hashrate will ideally have equal values of τ. For a miner who has committed to their ideal τ, we theorize that the miner is exceedingly likely to produce a valid proof. We base this claim using the fact that an honest miner's share set is overwhelmingly likely to follow a binomial distribution, such that the number of shares at level i occur at their expected mean. We also propose that the calculation for the ideal τ, τ_{ideal}, should be $\tau_{ideal} = (2^{256} \times N)/(h \times W)$

Inflated τ

We define an inflated τ value to be any value of τ which is deviating above a miner's ideal τ. Miners who commit to inflated τ values are, on average, producing more than N shares every round. We argue that there is no incentive for rational miners to commit to an inflated τ value. Given that share scores are calculated via the equation $2^{256}/\tau$, any miner who commits to an inflated τ value is receiving a smaller share score than other miners in the

pool who have the same hashrate. It should also be known that as τ deviates higher from its ideal value, a miner is increasingly likely to create a good and valid share proof, even more so than miners using their ideal τ.

Deflated τ

We define a deflated τ value to be any value of τ which is deviating below a miner's ideal τ. Miners who commit to deflated τ values are, on average, producing less than N shares every round. Commitment to such a τ value increases a miners expected share score, however, it also increases the likelihood that a miner is unable to produce a valid share proof for a given round. We base this claim again on the fact that S represents a binomial distribution, and that there is a real hardware-based limit on the number of hashes a miner may produce within the window.

First, let us remember that each hash produced by a miner represents an independent Bernoulli trial, and that decreasing the target also decreases the probability that any given hash satisfies the NISP inequality. Given that the total number of hashes a miner may produce in the window is a constant value, we can see that the only result of a miner deflating their τ is reduced probability of creating a valid share proof in a single round, due to the expected mean of such super shares being lower within the total amount of hashes. We therefore argue that there is little incentive for rational miners to deflate their τ value. Mining pools are created in order to deliver consistent payouts to miners. By setting τ lower than its ideal value, miners are needlessly increasing payout variability, which therefore defeats the purpose of pool mining itself.

Although miners have no incentive to commit to significant deviations from their ideal τ, there may be some miners who make small adjustments according to their personal risk-reward ratio. Such changes have little affect on other miners, and can instead be seen as a benefit of the protocol by placing more power into the hands of miners.

4 Concrete Example

From our analysis, we can see that rational miners are likely to be mining at or near their ideal τ value. With this, we have achieved our goals of succinctness, and consistency of miner rewards. It can be argued that non-interactivity has been violated by requiring that miners commit to their value of τ. However, given that miners are operating under the assumptions of stable hashrates, and that the parameter r limits the number of times τ may be changed, rational miners are unlikely to change their τ value often. We can therefore say that our proofs are non-interactive for all rounds after the initial ideal τ value is committed to. Now, we will go into an example with set parameters to provide a clearer explanation of the protocol.

Let's imagine a decentralized NISP pool, which is operating with the parameters $N = 10000$, $\mu = 20$, $W = 3600$ seconds, and $r = 10$. A new miner, with a consistent hashrate h of 500 MH/s wishes to join the pool. First, the miner must calculate their ideal τ and submit it to the pool smart contract. We can calculate the ideal τ by finding the number of expected hashes within the window, dividing it by N, and multiplying it's inverse by 2^{256}, which is the maximum value for a 256-bit hash (Used by Bitcoin, Ergo, and other blockchains).

Our miner is creating 5×10^8 hashes every second. Within the window, the miner is expected to create a total of $h \times W$ or 1.8×10^{12} hashes. We can then say that for our miner, $\tau_{ideal} = (2^{256} \times 10000)/(1.8 \times 10^{12})$, or 6.4328938×10^{68}. The share score (which may also be

seen as a miner's threshold *difficulty*) received for mining at this τ can simply be expressed as 180M or 1.8×10^8.

As stated in our model, for a share proof of level i, a miner must submit exactly i shares within P such that each share s satisfies the inequality $s < \tau \times (i/N)$. Given these parameters, valid share proofs would include 1 share with value $s < (\tau/10000)$, 2 shares with values $s < (\tau/5000)$, and 3 shares with values $s < (\tau/3333)$. For the maximum size share proof of length μ, 20 shares must satisfy the inequality $s < (\tau/500)$.

Given that the domain of i is constrained to the values $1 \leq i \leq \mu$, we can determine both the best and worst case data size for our NISPs. Let us assume that our current NISP pool is mining blocks on the Ergo blockchain. For verification of shares, we will assume that each share is holding only a block header, along with its hash derived from Ergo's PoW algorithm. The size of a standard Ergo header is 283 bytes, and Ergo uses 32 byte hashes. Therefore, each share will be 305 bytes in total. The minimum valid share proof size will be 305 bytes for 1 share whose hash value is greater than $\tau/10000$, while the maximum size will be 6100 bytes for 20 shares greater than $\tau/500$.

5 Empirical Data

We collected data from a single trial in order to perform tests on the viability of the NISP construction. Our trial involved the usage of a single miner, mining at a steady value of 61.21 MH/s for exactly one hour on Ergo's testnet. We were able to save data on 2349 potential shares, whose difficulty values were greater than 72M. We then applied filters on this data set in order to find shares which satisfied the inequality $H(w) < \tau$ for different values of τ.

Test 1

Our first test involved using the values $W = 3600$, and $N = 1000$ over the dataset. Using these parameters, we calculated the τ_{ideal} to be $\tau_{ideal} = (2^{256} \times 1000)/(62.21 \times 10^9 \times 3600) = 5.2547736044 \times 10^{68}$. This is equivalent to a difficulty value of 220.356M. We then rated the viability of different τ values by calculating the number of levels for which a valid share proof could be made using that τ. Our calculations used a constant value of $\mu = 20$, meaning a maximum rating of 20. We chose this rating system to be able to see how different values of τ affect the number of valid share proofs that could be generated.

Table 1

τ	Valid Proofs
$2.0 \times \tau_{ideal}$	20
$1.75 \times \tau_{ideal}$	20
$1.25 \times \tau_{ideal}$	20
$1.0 \times \tau_{ideal}$	11
$0.75 \times \tau_{ideal}$	8
$0.5 \times \tau_{ideal}$	2
$0.25 \times \tau_{ideal}$	0

From the data, we were able to see that inflated values of τ always had valid share proofs associated with them. On the other hand, deflated values of τ were less likely to have valid proofs for different levels, with the most deflated value being unable to provide a valid proof at all.

Test 2

Our second test involved dividing our sample set into discrete time increments based on the timestamp available on each share. We broke up our sample set into 12 intervals of 5 minutes, such that $W = 300$. We set $N = 100$, and calculated the τ_{ideal} to equal $6.3057283253 \times 10^{68}$ or a difficulty of 183.63M. We then calculated whether or not each of these trials held at least one valid share proof. This was done in order to simulate whether or not the miner would get paid for the work performed during that trial.

Table 2

Trial Window	≥ 1 Valid Proof for τ_{ideal}
00:00 - 00:05	true
00:05 - 00:10	true
00:10 - 00:15	true
00:15 - 00:20	true
00:20 - 00:25	true
00:25 - 00:30	true
00:30 - 00:35	false
00:35 - 00:40	false
00:40 - 00:45	true
00:45 - 00:50	true
00:50 - 00:55	true
00:55 - 01:00	true

In total, 83% of the trials held valid proofs, lending credence to the idea that the ideal τ was the best value to balance profitability with relative consistency in terms of payouts. Although these tests were limited in nature, we hope that they provide merit to the NISP construction we have provided, and that future works delve deeper into the statistics behind NISPs.

6 Applications

The most obvious and discussed application of NISPs is their usage in decentralized mining pools. For a smart contract based mining pool, NISPs allow calculation of a miner's work into a single share score, which can then be used to derive each miners reward proportional to the amount of work the miner did. There could be multiple designs for NISP pools which each have their own benefits and drawbacks. One design of interest may be the usage of an *optimistic rollup*, which could have great scalability benefits. Assuming miners are honest and using *fraud proofs* in order to prove them wrong would allow for less computation on the blockchain, and would ensure that a NISP-based pool can scale to thousands of users. Such a method would be impossible using past decentralized mining protocols such as SmartPool and P2Pool.

Because NISPs are based on the standard PoW inequality used by blockchains and mining pools, NISP pools may be deployed for any standard PoW-based blockchain. One caveat is that although NISPs can be used for any blockchain, they must be deployed in smart contracts to ensure full security guarantees. NISP-based pools may therefore be created on blockchains such as Ergo and Alephium, both of which represent programmable PoW blockchains. Pools on these blockchains may then service other PoW blockchains which lack programmability. For example, a NISP pool on Ergo could potentially serve as a decentralized

Bitcoin or Flux mining pool. The integration of the standard PoW inequality ensures that NISPs remain blockchain agnostic.

Another potential idea may involve the creation of NISPs which use a different share calculation algorithm. The NISPs presented in this paper most closely resemble the PPLNT share processing algorithm, however application of NISPs into algorithms like PPS may also be interesting. NISP protocols may also be allowed to have different verification algorithms which have different requirements. For example, a stricter protocol could require goodness be maintained at multiple levels, rather than just one. A more lax protocol may conversely reward miners who provide partial share proofs. Such a protocol could designate proportional share scores based on how *good* the partial share proof is compared to a completely valid proof.

Finally, more research into finding values for τ_{ideal} may be necessary. One informal trial we ran on an Ergo miner found that adjusting for the table set generation length used in the *Autolykos 2* PoW algorithm allowed for more successful NISP generation. Every time a new block is found on the Ergo blockchain, *Autolykos* miners must stop hashing and regenerate the table for some period of time. Our miner's tableset generation length had a mean and median value of 4.57 seconds. This proved to have significant implications on the ideal τ, especially because the testing network had only a 45 second block time.

Other usages of NISPs may involve smart-contract based lending of hashrate and direct work-based governance on decentralized applications. In reality, any application which provides the right incentive structures and incorporates work can use NISPs within their their protocol. With such incentive structures, the NISP system tends towards miners using their ideal τ.

References

1. Luu, L., Velner, Y., Teutsch, J., & Saxena, P. (2017, August 16). *SmartPool: Practical Decentralized Pooled Mining.* 17th USENIX Security Symposium. Retrieved from Usenix.org
2. Meni Rosenfeld (2011). *Analysis Of Bitcoin Pooled Mining Reward Systems.* CoRR, abs/1112.4980
3. Satoshi Nakamoto (2009). *Bitcoin: A peer-to-peer electronic cash system.* Retrieved from bitcoin.org
4. Kiayias A., Miller A. & Zindros D. (2018, May 31). *Non-Interactive Proofs of Proof-of-Work.* Retrieved from nipopows.org.

zkFFT

Extending Halo2 with Vector Commitments & More

Aram Jivanyan[1], Gohar Hovhannisyan[1], Hayk Hovhannisyan*[1,2], and Nerses Asaturyan[1,2]

[1]Yerevan State University,
aram@skycryptor.com,goharhovhannisyann@gmail.com
[2]Layerswap Labs,{hayk,nerses}@layerswap.io

Abstract

This paper introduces zkFFT, a novel zero-knowledge argument designed to efficiently generate proofs for Fast Fourier Transform (FFT) relations. Our approach enables the verification that one committed vector is the FFT of another, addressing an efficiency need in general-purpose non-interactive zero-knowledge proof systems where the proof relation utilizes vector commitments inputs.

We present a concrete enhancement to the Halo2 proving system, demonstrating how zkFFT optimizes proofs in scenarios where the proof relation includes one or more vector commitments. Specifically, zkFFT incorporates streamlined logic within Halo2 and similar systems, augmenting proof and verification complexity by only $O(\log N)$, where N is the vector size. This represents a substantial improvement over conventional approach, which often necessitates specific circuit extensions to validate the integrity of vector commitments and their corresponding private values in the arithmetic framework of the proof relation. The proposed zkFFT method supports multiple vector commitments with only a logarithmic increase in extension costs, making it highly scalable. This capability is pivotal for practical applications involving multiple pre-committed values within proof statements.

Apart from Halo2, our technique can be adapted to any other zero-knowledge proof system that relies on arithmetization, where each column is treated as an evaluation of a polynomial over a specified domain, computes this polynomial via FFT, and subsequently commits to the resulting polynomial using a polynomial commitment scheme based on inner product arguments. Along with efficient lookup and permutation arguments, zkFFT will streamline and significantly optimize the generation of zero-knowledge proofs for arbitrary relations.

Beyond the applications in augmenting zero-knowledge proof systems, we believe that the formalized zkFFT argument can be of independent interest.

Contents

1 Introduction **2**
- 1.1 Our contribution . . . 2
- 1.2 Related Work . . . 3
- 1.3 Applications . . . 3
- 1.4 Structure of the paper . . . 3

2 Preliminaries **3**
- 2.1 Polynomial Commitment Schemes . . . 3
- 2.2 Fast Fourier Transforms . . . 4
- 2.3 Inner Product Proofs . . . 4

*corresponding author: hayk@leyerswap.io

3 Modified Inner Product Argument **4**
3.1 MIPA Protocol 4
3.2 Security Analysis 8

4 zkFFT: Zero-Knowledge Argument for FFT Relation Check **11**
4.1 zkFFT Protocol 11
4.2 Aggregating zkFFT Proofs 13
4.3 Batch Verification of zkFFT proofs 14

5 Supporting Vector Commitment Inputs in Halo2 **14**

6 Curve Tree Relation **16**

7 Implementation and Performance **17**

8 References **18**

A Detailed benchmarks **19**

1 Introduction

In this paper, we discuss a method to generalize the Halo2 [16] proof system by including input vector commitments in the relation where the commitment openings are known to the prover. This method allows significant optimization of Plonkish constructions by leveraging the fact that commitment openings can participate in the AIR constraints without the extra need to prove that these values, locked in the AIR and participating in the constraint equations,match the original openings of the input commitments.

A concrete motivation for this work has been the desire to implement an efficient membership-proof algorithm referred to as Curve Trees which uses the cycle of curves and a d-ary tree-like structure to commit to a set of cryptographic commitments and generate a proof of knowledge of a secret element belonging to the committed set. The original Curve Trees [7] were implemented with a Bulletproof [6] proof system that was analogously extended to support vector commitments. Replacing the underlying proof system with Halo2 might allow us to leverage the recursive proof composition features of the latter and get smaller membership proofs which are also faster to verify compared to the Bulletproof-based proofs. The Bulletproof proof system can be extended to support extra vector commitments quite naturally as its core R1CS commitment [5] approach is aligned with the Pedersen commitment method [15]. Halo2's reliance on AIR and polynomial commitment logic makes it nontrivial to efficiently support extra vector commitments.

1.1 Our contribution

Our contribution is threefold:

zkFFT Proofs. We formalize and provide an efficient proof system for the zkFFT relation. The zkFFT logic can prove that the first vector commitment hides the evaluation points of a concrete polynomial over a known domain whose coefficients are committed through the second commitment. The need for zkFFT proof arises from the following consideration: The Halo2 system interprets the witness as advice column values in its AIR. For each column, the corresponding values are interpreted as the values of a polynomial evaluated over the given fixed domain. This polynomial is computed using FFT operations, and the computed polynomial is committed using some polynomial commitment scheme. Halo2 uses an inner product arguments based polynomial commitment scheme, which commits to the polynomial coefficients using a generalized Pedersen commitment scheme. Assuming the initial vector

commitment is given as $C_v = g_0^{v_0} g_1^{v_1} \dots g_{n-1}^{v_{n-1}}$ and its opening values $v_0, v_1, \dots, v_{n-1}$ will form a separate advice column in the AIR representation. The corresponding polynomial $P(x) = a_0 + a_1 x + \dots + a_{n-1} x^{n-1}$ will be computed using FFT operations such that $P(\omega^i) = v_i$. The polynomial $P(x)$ is then committed through the Pedersen commitment scheme as $C_c = h_0^{a_0} h_1^{a_1} \cdots h_{n-1}^{a_{n-1}}$ which becomes part of the final Halo2 proof. We end up with a situation where there are two commitments C_v and C_c committing to the same polynomial's values and coefficients respectively and there is a need to prove the concrete FFT relation between their openings.

Halo2 Generalization to Support Vector Commitments. We provide a full description of the generalized Halo2 system which can take as input numerous vector commitments, and create zkFFT proofs for the given vectors.

Membership Proofs: We implement our modifications in the Halo2 proof system and run benchmarks to showcase the efficiency of the applied method in building full circuits for curve trees, allowing the building of efficient membership proofs.

1.2 Related Work

The Bulletproof[6] proof system has been generalized[1] recently to support arbitrary vector commitment inputs. The extension of the Bulletproof system comes quite naturally as its underlying R1CS commitment technique is simply generalized Pedersen commitment to all gate values, and thus structure-wise is equivalent to the extra input commitment structure. This extension has allowed optimizing Curve Tree based membership proofs implemented through Bulletproofs.

1.3 Applications

There are numerous zero-knowledge applications where the prover relation may contain public vector commitment inputs including membership and non-membership proofs, digital identity schemes, or private defi systems.

1.4 Structure of the paper

Section 2 introduces all the necessary preliminary notions. In Section 3, we introduce the modified inner product arguments, and formally prove their security. In Section 4 we introduce the zkFFT technique, and additionally show how multiple zkFFT proofs can be aggregated and batched. Section 5 details the extension of Halo2 with vector commitments, and Section 6 explains why zkFFT is essential for Curve Trees in Halo2. The performance of zkFFT is discussed in Section 7, and detailed benchmarks are provided in Appendix.

2 Preliminaries

2.1 Polynomial Commitment Schemes

Polynomial commitment schemes form a fundamental building block in many modern arguments of knowledge. In these schemes, a prover can construct commitments to polynomials and then later provably evaluate the committed polynomials at arbitrary points. We will use the polynomial commitment scheme from [4] as we discuss and extend the well-known Halo2 system. This polynomial commitment scheme is based on generalized Pedersen commitments.

2.2 Fast Fourier Transforms

The FFT (Fast Fourier Transform) [9] algorithm efficiently performs computations over polynomials.Specifically, for $d = 2^D$ over a field F with characteristic $p = qd + 1$, and ω as a primitive d-th root of unity it computes:

- $P(\omega^i)$ for $i = 0, \ldots, d-1$ given coefficients of $P(X)$ with $\deg(P) < d$
- Coefficients of $P(X)$ with $\deg(P) < d$ given $P(\omega^i)$ for $i = 0, \ldots, d-1$

2.3 Inner Product Proofs

1. By the vector commitment of a given vector $\vec{a} = (a_0, a_1, ..., a_{n-1})$, we will denote $C_{\vec{a}} = g_0^{a_0} \cdot g_1^{a_1} \cdot ... \cdot g_{n-1}^{a_{n-1}}$.
2. By power-vector of a we are going to denote the following vector $\boldsymbol{a} = (a^0, a^1, ..., a^{n-1})$
3. The $w = [\omega^1, ..., \omega^{n-1}, \omega^n = \omega^0 = 1]$ are the $\mathbf{n}$-th roots of unity.
4. So the $\mathbf{n}$ power-vectors of the roots of unity are going to be
   - $\boldsymbol{\omega^0} = (\omega^0, \omega^0, \cdots, \omega^0) = (1, 1, \cdots, 1)$
   - $\boldsymbol{\omega^1} = (\omega^0, \omega^1, \cdots, \omega^{n-1})$
     $\vdots$
   - $\boldsymbol{\omega^{n-1}} = (\omega^0, \omega^{n-1}, \cdots, \omega^{(n-1)\cdot(n-1)})$
5. The dot product of 2 vectors is denoted as $\langle \vec{a}, \vec{b} \rangle = \sum_{i=0}^{n-1} a_i \cdot b_i$.
6. We will say that the vector commitment C_v is the Fast Fourier Transformation of another vector commitment C_a and denote it as $C_v = \mathbf{FFT}(C_a)$ if
   (a) The polynomial whose coefficients are the elements of the vector $\vec{a}$
   $P(X) = a_0 + a_1 \cdot X + ... + a_{n-1} \cdot X^{n-1}$
   (b) Evaluates to the elements of the vector $\vec{v}$ at the $\mathbf{n}$-th roots of unity.
   $\forall i \in [0, 1, ..., n-1] \quad P(\omega^i) = v_i$

3 Modified Inner Product Argument

The original inner product argument introduced in [3] provides an efficient proof system for the following relation,

$$\{(\mathbf{g}, \mathbf{h}, u \in \mathbf{G}^n, P \in G, \alpha \in Z_p; \mathbf{a}, \mathbf{b} \in Z_p^n) : P = \mathbf{g}^{\mathbf{a}} \mathbf{h}^{\mathbf{b}} u^{<\mathbf{a},\mathbf{b}>} h^{\alpha}\}$$

but a single inner product argument is not sufficient for zkFFT, and using multiple IPAs is not efficient that's why we introduce a new protocol called *modified inner product arguments*.

3.1 MIPA Protocol

Modified inner product argument corresponds to the following relation

$$\mathrm{REL}_{\mathrm{MIPA}} := \left\{ \begin{pmatrix} \mathbf{g}, \mathbf{h}_1, \ldots, \mathbf{h}_k, \in \mathbf{G}^n \\ h, u_1, \ldots, u_k, P \in G; \\ \mathbf{a}, \mathbf{b}_1, \ldots, \mathbf{b}_k \in Z_p^n, \alpha \in Z_p, \end{pmatrix} \;\middle|\; P = \mathbf{g}^{\mathbf{a}} \mathbf{h}_1^{\mathbf{b}_1} \cdots \mathbf{h}_k^{\mathbf{b}_k} u_1^{<\mathbf{a},\mathbf{b}_1>} \cdots u_k^{<\mathbf{a},\mathbf{b}_k>} h^{\alpha} \right\}$$

Note that in our modified inner product relation, there are numerous vectors $\mathbf{b}_i$ whose inner product values with the committed vector $\mathbf{a}$ are committed through independent bases.

The grand product value P is also blinded by an extra factor h^α in the commitment key, all generator vectors $\mathbf{g}, \mathbf{h}_1, \ldots, \mathbf{h}_k$ are comprised of mutually orthogonal generator points. The provided proof system for the modified inner product argument will be further tailored for our specific needs in the extension of the Halo2 system, where the vectors $\mathbf{b}_1, \ldots, \mathbf{b}_k$ are public and known to the verifier. This fact will allow us to drastically reduce the final proof size.

Overview. The inputs to the inner product argument are independent generators $g, h_1, \cdots, h_k \in \mathbb{G}_n$, scalars $c_1, \cdots, c_k \in \mathbb{Z}_p$, and $P \in \mathbb{G}$. The argument allows the prover to convince a verifier that the prover knows $1 + k$ vectors $\mathbf{a}, \mathbf{b_i} \in \mathbb{Z}_p^n$ such that

$$P = \mathbf{g}^{\mathbf{a}} \mathbf{h}_1^{\mathbf{b}_1} \cdots \mathbf{h}_k^{\mathbf{b}_k} u_1^{<\mathbf{a},\mathbf{b}_1>} \cdots u_k^{<\mathbf{a},\mathbf{b}_k>} h^\alpha$$

We refer to P as a binding vector commitment to $\mathbf{a}, \mathbf{b_1}, \cdots, \mathbf{b_k}$. Throughout the section, we assume that the dimension n is a power of 2. If need be, one can easily pad the inputs to ensure that this holds.

More precisely, the inner product argument is an efficient proof system for the following relation:

$$\mathrm{REL}_{\mathrm{MIPA}} := \left\{ \begin{pmatrix} \mathbf{g}, \mathbf{h}_1, \ldots, \mathbf{h}_k, \in \mathbf{G}^n \\ h, u_1, \ldots, u_k, P \in G; \\ \mathbf{a}, \mathbf{b}_1, \ldots, \mathbf{b}_k \in Z_p^n, \alpha \in Z_p, \end{pmatrix} \;\middle|\; P = \mathbf{g}^{\mathbf{a}} \mathbf{h}_1^{\mathbf{b}_1} \cdots \mathbf{h}_k^{\mathbf{b}_k} u_1^{<\mathbf{a},\mathbf{b}_1>} \cdots u_k^{<\mathbf{a},\mathbf{b}_k>} h^\alpha \right\}$$

To give some intuition for how the proof system for the relation works, let us define a hash function $H : \mathbb{Z}_p^{n+(n+1)\cdot k+1} \to \mathbb{G}$ as follows. First, set $n' = n/2$ and fix generators $g, h_1, \cdots, h_k \in \mathbb{G}_n$, $u \in \mathbb{G}$. Then the hash function H takes as input $(a, a', b_1, b_1', \cdots, b_k, b_k') \in \mathbb{Z}_p^{n'}$, $c_1, \cdots, c_k \in \mathbb{Z}_p$ and α and outputs

$$H(a, a', b_1, b_1', \cdots, b_k, b_k', c_1, \cdots, c_k, \alpha) = g_{[:n']}^{\mathbf{a}} \cdot g_{[n':]}^{\mathbf{a}'} \cdot h_{1\,[:n']}^{b_1} \cdot h_{1\,[n':]}^{b_1'} \cdots h_{k\,[:n']}^{b_k} \cdot h_{k\,[n':]}^{b_k'} \cdot u_1^{c_1} \cdots u_k^{c_k} \cdot h^\alpha \in \mathbb{G}.$$

Now, using the setup in the relation, we can write P as
$P = H(a_{[:n']}, a_{[n':]}, b_{1[:n']}, b_{1[n':]}, \cdots, b_{k[:n']}, b_{k[n':]}, \langle \mathbf{a}, \mathbf{b_1} \rangle, \cdots, \langle \mathbf{a}, \mathbf{b_k} \rangle, \alpha)$. Note that H is additively homomorphic in its inputs, i.e.,

$$H(a_1, a_{11}, b_1, b_{11}, c_1) \cdot H(a_2, a_{12}, b_2, b_{12}, c_2) = H(a_1 + a_2, a_{11} + a_{12}, b_1 + b_2, b_{11} + b_{12}, c_1 + c_2).$$

Consider the following protocol for the relation, where $P \in \mathbb{G}$ is given as input:

1. The prover chooses 2 random values $d_L, d_R \in \mathbb{Z}_p$and computes $L, R \in \mathbb{G}$ as follows:

$$L = H(\mathbf{0}^{n'}, \mathbf{a}_{[:n']}, b_{1[n':]}, \mathbf{0}^{n'}, \cdots b_{k[n':]}, \mathbf{0}^{n'}, \langle \mathbf{a}_{[:n']}, b_{1[n':]} \rangle, \cdots, \langle \mathbf{a}_{[:n']}, b_{k[n':]} \rangle, d_L)$$

$$R = H(\mathbf{a}_{[n':]}, \mathbf{0}^{n'}, \mathbf{0}^{n'}, b_{1[:n']}, \cdots, \mathbf{0}^{n'}, b_{k[:n']}, \langle \mathbf{a}_{[n':]}, b_{1[:n']} \rangle, \cdots, \langle \mathbf{a}_{[n':]}, b_{k[:n']} \rangle, d_R)$$

   and recall that $P = H(\mathbf{a}_{[:n']}, \mathbf{a}_{[n':]}, b_{1[:n']}, b_{1[n':]}, \cdots, b_{k[:n']}, b_{k[n':]}, \langle \mathbf{a}, \mathbf{b_1} \rangle, \cdots, \langle \mathbf{a}, \mathbf{b_k} \rangle, \alpha')$.
   It sends $\mathbf{L}, \mathbf{R}$ to the verifier.

2. The verifier chooses a random $x \in \mathbb{Z}_p$ and sends x to the prover.

3. The prover computes

   (a) $a' = x a_{[:n']} + x^{-1} a_{[n':]} \in \mathbb{Z}_p^{n'}$
   (b) $\forall i \; b_i' = x^{-1} b_{i[:n']} + x b_{i[n':]} \in \mathbb{Z}_p^{n'}$
   (c) $\alpha' = d_L x^2 + \alpha + d_R x^{-2}$

   and sends $a', b_1', \cdots, b_k' \in \mathbb{Z}_p^{n'}$ to the verifier.

4. Given $(L, R, a', b'_1, \cdots, b'_k)$, the verifier computes $P_1 = L(x^2) \cdot P \cdot R(x^{-2})$ and outputs "accept" if

$$P' = H(x^{-1}a', xa', xb'_1, x^{-1}b'_1, \cdots, xb'_k, x^{-1}b'_k, \langle \mathbf{a'}, \mathbf{b'_1} \rangle, \cdots, \langle \mathbf{a'}, \mathbf{b'_k} \rangle)$$

It is easy to verify that a proof from an honest prover will always be accepted. Indeed, the left hand side of the final equation is

$$L(x^2) \cdot P \cdot R(x^{-2}) = H(a_{[:n']} + x^{-2}a_{[n':]}, x^2 a_{[:n']} + a_{[n':]}, x^2 b_{1[n':]} + b_{1[:n']}, b_{1[n':]} + x^{-2} b_{1[:n']}, \cdots$$

$$x^2 b_{k[n':]} + b_{k[:n']}, b_{k[n':]} + x^{-2} b_{k[:n']}, \langle a', b'_1 \rangle, \cdots, \langle a', b'_k \rangle)$$

which is the same as the right hand side of the final equation. In this proof system, the proof sent from the prover is the tuple $(L, R, a', b'_1, \cdots, b'_k, \alpha')$ and contains only $(k+1) \cdot \frac{n}{2} + 2$ elements. This is about half the length of the trivial proof where the prover sends the complete $\mathbf{a}, \mathbf{b_1}, \cdots, \mathbf{b_k} \in \mathbb{Z}_p^n$ to the verifier.

Shrinking the proof by recursion. Observe that the test in (4) is equivalent to testing that

$$P' = \left(g_{[:n']}^{x^{-1}} \cdot g_{[n':]}^{x}\right)^{a'} \left(h_{1[:n']}^{x} \cdot h_{1}^{x^{-1}}{}_{[n':]}\right)^{b'_1} \cdots \left(h_{k[:n']}^{x} \cdot h_{k}^{x^{-1}}{}_{[n':]}\right)^{b'_k} \cdot u^{\langle a, b'_1 \rangle} \cdots u^{\langle a, b'_k \rangle} h^{\alpha'}.$$

Hence, instead of the prover sending the vectors $a', b'_1, \cdots, b'_k, \alpha'$ to the verifier, they can recursively engage in an inner product argument for P' with respect to generators $\left(g_{[:n']}^{x^{-1}} \circ g_{[n':]}^{x}, h_{1[:n']}^{x} \circ h_{1}^{x^{-1}}{}_{[n':]}, \cdots, h_{k[:n']}^{x} \circ h_{k}^{x^{-1}}{}_{[n':]}, u_1, \cdots, u_k, \alpha'\right)$. The dimension of this problem is only $n' = \frac{n}{2}$.

The resulting $\log_2 n$ depth recursive protocol is shown in figure 1. This $\log_2 n$ round protocol is public coin and can be made non-interactive using the Fiat-Shamir heuristic. The total communication of the Protocol is only $\lceil 2\log_2(n) \rceil$ elements in $\mathbb{G}$ plus $1 + k$ elements in $\mathbb{Z}_p$. Specifically, the prover sends the following terms:

$$(L_1, R_1), \ldots, (L_{\log_2 n}, R_{\log_2 n}), a, b_1, \cdots, b_k$$

where $a, b_1, \cdots, b_k \in \mathbb{Z}_p$ are sent at the tail of the recursion.

$$\text{REL}_{\text{MIPA}} := \left\{ \left(\begin{array}{r} \mathbf{g}, \mathbf{h}_1, \ldots, \mathbf{h}_k \in \mathbf{G}^n \\ h, u_1, \ldots, u_k, P \in G; \\ \mathbf{a}, \mathbf{b}_1, \ldots, \mathbf{b}_k \in Z_p^n, \\ \alpha \in Z_p, \end{array} \right) \;\middle|\; P = \mathbf{g}^{\mathbf{a}} \mathbf{h}_1^{\mathbf{b}_1} \cdots \mathbf{h}_k^{\mathbf{b}_k} u_1^{<\mathbf{a},\mathbf{b}_1>} \cdots u_k^{<\mathbf{a},\mathbf{b}_k>} h^{\alpha} \right\}$$

If $n = 1$:

P: $r, s_1, \cdots, s_k, \delta, \eta \xleftarrow{\$} \mathbb{Z}_p$ and computes

$A = g^r h_1^{s_1} \cdots h_k^{s_k} u_1^{<r,b_1>+<s_1,a>} \cdots u_k^{<r,b_k>+<s_k,a>} h^{\delta} \in \mathbb{G}$

$B = u_1^{<r,s_1>} \cdots u_k^{<r,s_k>} h^{\eta} \in \mathbb{G}$

P $\rightarrow$ V: A, B

V: $x \xleftarrow{\$} \mathbb{Z}_p$

P $\leftarrow V$: x

P: computes

$r' = r + a \cdot x \in \mathbb{Z}_p$

$s'_i = s_i + b \cdot x \in \mathbb{Z}_p \quad \forall i \in [1, k]$

$\delta' = \eta + \delta \cdot e + \alpha \cdot x^2 \in \mathbb{Z}_p$

P $\rightarrow$ V: $r', \delta', s'_i \quad \forall i \in [1, k]$

V: outputs *Accept* if the following equality holds

$A^x \cdot P^{x^2} \cdot B = g^{r' \cdot x} h_1^{s'_1 \cdot x} \cdots h_k^{s'_k \cdot x} u_1^{<r', s'_1>} \cdots u_k^{<r', s'_k>} h^{\delta'} \in \mathbb{G}$

Else ($\mathbf{n > 1}$):

P: $d_L, d_R \xleftarrow{\$} \mathbb{Z}_p$ and computes

Let $n' = \frac{n}{2}$

$c_{Li} = \langle a_{[:n']}, b_{[n':]} \rangle, c_{Ri} = \langle a_{[n':]}, b_{[:n']} \rangle \in \mathbf{Z_p} \quad \forall i \in [1, k]$

$L = g_{[n':]}^{a_{[:n']}} h_{1\,[:n']}^{b_{1\,[n':]}} \cdots h_{k\,[:n']}^{b_{k\,[n':]}} u_1^{c_{L1}} \cdots u_k^{c_{Lk}} h^{d_L} \in \mathbf{G}$

$R = g_{[:n']}^{a_{[n':]}} h_{1\,[n':]}^{b_{1\,[:n']}} \cdots h_{k\,[n':]}^{b_{k\,[:n']}} u_1^{c_{R1}} \cdots u_k^{c_{Rk}} h^{d_R} \in \mathbf{G}$

P $\rightarrow$ V: L, R

V: $x \xleftarrow{\$} \mathbb{Z}_p$

P $\leftarrow V$: x

P and V: compute

$g' = g_{[:n']}^{x^{-1}} \circ g_{[n':]}^{x} \in \mathbf{G^{n'}}$

$h'_i = h_{i\,[:n']}^{x} \circ h_i^{x^{-1}}{}_{[n':]} \in \mathbf{G^{n'}} \quad \forall i \in [1, k]$

$P' = L^{x^2} P R^{x^{-2}} \in \mathbf{G}$

P: computes

1. $a' = x a_{[:n']} + x^{-1} a_{[n':]} \in \mathbb{Z}_p^{n'}$
2. $b'_i = x^{-1} b_{i[:n']} + x b_{i[n':]} \in \mathbb{Z}_p^{n'} \quad \forall i \in [1, k]$
3. $\alpha' = d_L x^2 + \alpha + d_R x^{-2}$

P and V: recursively run the protocol on

$(g', h'_1, \cdots, h'_k, u_1, \cdots, u_k, h, P'; a', b'_1, \cdots, b'_k, \alpha')$

Figure 1: Zero Knowledge Argument for MIPA relation

3.2 Security Analysis

Now we are going to construct an efficient extractor for the MIPA protocol. The construction of the extractor is similar to the extractor of the IPA protocol.
First, let us consider $n = 1$ case. In the first move, the prover sends A and B to the verifier. By rewinding the oracle $\langle P^*, V\rangle$ four times with five distinct challenges x_1, x_2, x_3, x_4 and x_5 while using the same A and B, the extractor obtains five tuples $(r'_i, s'_{i,j}, \delta'_i)$ satisfying the following verification equation:

$$P^{x_i^2} A^{x_i} B = g^{r'_i \cdot x_i} \prod_{j=1}^{k} \left(h_j^{s'_{i,j} \cdot x_i} u_j^{\langle r'_i, s'_{i,j}\rangle} \right) h^{\delta'_i} \quad \text{for} \quad i = 1, \ldots, 4 \tag{20}$$

We can use the first three challenges x_1, x_2, x_3, to compute $\nu_1, \nu_2, \nu_3 \in \mathbb{Z}_p$ such that

$$\sum_{i=1}^{3} \nu_i \cdot x_i^2 = 1, \quad \sum_{i=1}^{3} \nu_i = 0, \quad \sum_{i=1}^{3} \nu_i \cdot x_i^{-2} = 0.$$

Then, taking a linear combination of the first three equalities , with ν_1, ν_2, ν_3 as the coefficients, we can compute $a_A, a_P, a_B, b_{j,A}, b_{j,P}, b_{j,B}, c_{j,A}, c_{j,P}, c_{j,B}, d_A, d_P, d_B$ such that

$$P = g^{a_P} \prod_{j=1}^{k} \left(h_j^{b_{jP}} u_j^{c_{jP}} \right) h^{d_P},$$

$$A = g^{a_A} \prod_{j=1}^{k} \left(h_j^{b_{jA}} u_j^{c_{jA}} \right) h^{d_A},$$

$$B = g^{a_B} \prod_{j=1}^{k} \left(h_j^{b_{jB}} u_j^{c_{jB}} \right) h^{d_B}.$$

Using the above three equations and the verification equation, we obtain for each $x_i \in \{x_1, x_2, x_3, x_4, x_5\}$:

$$g^{r'_i x_i - a_P x_i^2 - a_A x_i - a_B} \prod_{j=1}^{k} \left(h_j^{s'_{i,j} x_i - b_{jP} x_i^2 - b_{jA} x_i - b_{jB}} \cdot u_j^{\langle r'_i, s'_{i,j}\rangle - c_{jP} x_i^2 - c_{jA} x_i - c_{jB}} \right) h^{\delta'_i - d_P x_i^2 - d_A x_i - d_B} = 1_G.$$

Thus, under the discrete logarithm relation assumption, we have $2k + 2$ equations of exponents according to the bases $g, h_1, \cdots, h_k, u_1, \cdots, u_k, h$:

$$\begin{gathered} r'_i x_i - a_P x_i^2 - a_A x_i - a_B = 0 \\ s'_{i,j} x_i - b_{jP} x_i^2 - b_{jA} x_i - b_{jB} = 0 \quad \forall j \in [1; k] \\ \langle r'_i, s'_{i,j}\rangle - c_{jP} x_i^2 - c_{jA} x_i - c_{jB} = 0 \quad \forall j \in [1; k] \\ \delta'_i - d_P x_i^2 - d_A x_i - d_B = 0 \end{gathered}$$

and, equivalently:

$$\begin{gathered} r'_i = a_P x_i + a_A + a_B x_i^{-1} \\ s'_{i,j} = b_{jP} x_i + b_{jA} + b_{jB} x_i^{-1} \\ \langle r'_i, s'_{i,j}\rangle = c_{jP} x_i^2 + c_{jA} x_i + c_{jB} \\ \delta'_i = d_P x_i^2 + d_A x_i + d_B. \end{gathered}$$

By eliminating r'_i and $s'_{i,j}$ from the equations we have for $i \in \{1, \ldots, 5\}$:

$$\langle a_P, b_{jP}\rangle \cdot x_i^2 + (\langle a_P, b_{jA}\rangle + \langle b_{jP}, a_A\rangle) \cdot x_i + (\langle a_P, b_{jB}\rangle + \langle b_{jP}, a_B\rangle + \langle a_A, b_{jA}\rangle) +$$

$$(\langle a_A, b_{jB}\rangle + b_{jA}, a_B\rangle)x_i^{-1} + \langle a_B, b_{jB}\rangle \cdot x_i^{-2} = c_{jP}x_i^2 + c_{jA}x_i + c_{jB} \in \mathbb{Z}_p \quad \forall j \in [1;k].$$

Since equality holds for all the 5 challenges x_1, x_2, x_3, x_4, x_5 and there are five variable terms $x^2, x, 1, x^{-1}, x^{-2}$ then each coefficient on the left-hand side of must be equal to the corresponding coefficient on the right-hand side:

$$\langle a_P, b_{jP}\rangle = c_{jP} \quad \forall j \in [1;k].$$

As intended, the extractor either extracts a witness or a discrete logarithm relation between the generators.

Next, we show that for each recursive step ($n > 1$ case) on input $(g, h_1, \cdots, h_k, u_1, \cdots, u_k, P)$ we can efficiently extract from the prover a witness $a, b_1, \cdots, b_k$ or a non-trivial discrete logarithm relation between $g, h_1, \cdots, h_k, u_1, \cdots, u_k$. The extractor runs the prover to get L and R. Then, by rewinding the prover four times and giving it four challenges x_1, x_2, x_3, x_4, such that $x_i \neq x_j$ for $1 \leq i < j \leq 4$, the extractor obtains $a'_i, b'_{j,i}, \alpha'_i \in \mathbb{Z}_p^{n'}$ where $i \in [1,4], j \in [1,k]$ such that

$$L^{x_i^2} P R^{x_i^{-2}} = \left(g_{[:n']}^{x_i^{-1}} \circ g_{[n':]}^{x_i}\right)^{a'_i} \cdot \left(h_1^{x_i}{}_{[:n']} \circ h_1^{x_i^{-1}}{}_{[n':]}\right)^{b'_{1,i}} \cdots \left(h_k^{x_i}{}_{[:n']} \circ h_k^{x_i^{-1}}{}_{[n':]}\right)^{b'_{k,i}} \cdot$$

$$u_1^{\langle a'_i, b'_{1,i}\rangle} \cdots u_k^{\langle a'_i, b'_{k,i}\rangle} \cdot h^{\alpha'_i}$$

We can use the first three challenges x_1, x_2, x_3, to compute $\nu_1, \nu_2, \nu_3 \in \mathbb{Z}_p$ such that

$$\sum_{i=1}^{3} \nu_i \cdot x_i^2 = 1, \quad \sum_{i=1}^{3} \nu_i = 0, \quad \sum_{i=1}^{3} \nu_i \cdot x_i^{-2} = 0.$$

Then, taking a linear combination of the first three equalities , with ν_1, ν_2, ν_3 as the coefficients, we can compute $a_L, a_P, a_R, b_{j,L}, b_{j,P}, b_{j,R}, c_{j,L}, c_{j,P}, c_{j,R}, d_L, d_P, d_R$ such that

$$L = g^{a_L} h_1^{b_{1,L}} \cdots h_k^{b_{k,L}} u_1^{c_{1,L}} \cdots u_1^{c_{1,L}} h^{d_L}, \tag{1}$$
$$P = g^{a_P} h_1^{b_{1,P}} \cdots h_k^{b_{k,P}} u_1^{c_{1,P}} \cdots u_1^{c_{1,P}} h^{d_P}, \tag{2}$$
$$R = g^{a_R} h_1^{b_{1,R}} \cdots h_k^{b_{k,R}} u_1^{c_{1,R}} \cdots u_1^{c_{1,R}} h^{d_R}. \tag{3}$$

Now, for each $x \in \{x_1, x_2, x_3, x_4\}$ and the corresponding $a', b'_j \in \mathbb{Z}_p^{n'}$, we can rewrite (eq. 1) as:

$$g^{a_L \cdot x^2 + a_P + a_R \cdot x^{-2}} \cdot \prod_{j=1}^{k} (h_j^{b_{j,L} \cdot x^2 + b_{j,P} + b_{j,R} \cdot x^{-2}} \cdot u_j^{c_{j,L} \cdot x^2 + c_{j,P} + c_{j,R} \cdot x^{-2}}) =$$

$$L^{x^2} P R^{x^{-2}} =$$

$$g_{[:n']}^{a' \cdot x^{-1}} \cdot g_{[n':]}^{a' \cdot x} h_1^{b'_1 \cdot x^{-1}}{}_{[:n']} \cdot h_1^{b'_1 \cdot x}{}_{[n':]} \cdots h_k^{b'_k \cdot x^{-1}}{}_{[:n']} \cdot h_k^{b'_k \cdot x}{}_{[n':]} \cdot u_1^{x^{a',b'_1}} \cdots u_k^{x^{a',b'_k}}.$$

This implies that:

$$a' \cdot x^{-1} = a_{L,[:n']} \cdot x^2 + a_{P,[:n']} + a_{R,[:n']} \cdot x^{-2}$$
$$a' \cdot x = a_{L,[n':]} \cdot x^2 + a_{P,[n':]} + a_{R,[n':]} \cdot x^{-2}$$
$$b'_1 \cdot x = b_{1L,[:n']} \cdot x^2 + b_{1P,[:n']} + b_{1R,[:n']} \cdot x^{-2},$$
$$b'_1 \cdot x^{-1} = b_{1L,[n':]} \cdot x^2 + b_{1P,[n':]} + b_{1R,[n':]} \cdot x^{-2},$$
$$\vdots$$

$$b'_k \cdot x = b_{kL,[:n']} \cdot x^2 + b_{kP,[:n']} + b_{kR,[:n']} \cdot x^{-2},$$
$$b'_k \cdot x^{-1} = b_{kL,[n':]} \cdot x^2 + b_{kP,[n':]} + b_{kR,[n':]} \cdot x^{-2},$$
$$\langle a', b'_1 \rangle = c_{1,L} \cdot x^2 + c_{1,P} + c_{1,R} \cdot x^{-2}.$$
$$\vdots$$
$$\langle a', b'_k \rangle = c_{k,L} \cdot x^2 + c_{k,P} + c_{k,R} \cdot x^{-2}.$$

If any of these equalities do not hold, we directly obtain a non-trivial discrete logarithm relation between the generators. If the equalities hold, we can deduce that for each challenge $x \in \{x_1, x_2, x_3, x_4\}$:

$$a_{L,[:n']} \cdot x^3 + \left(a_{P,[:n']} - a_{L,[n':]}\right) \cdot x + \left(a_{R,[:n']} - a_{P,[n':]}\right) \cdot x^{-1} - a_{R,[n':]} \cdot x^{-3} = 0$$

This equality follows from the first 2 equations above, as the equality holds for all the 4 challenges x_1, x_2, x_3, x_4 and there are 4 variable terms x^3, x, x^{-1}, x^{-3} then

$$a_{L,[:n']} = a_{R,[n':]} = 0,$$
$$a_{L,[n':]} = a_{P,[:n']}, \quad a_{R,[:n']} = a_{P,[n':]}$$

We can do the same for each $\mathrm{b}_j \forall j \in [1; k]$.

$$b_{jL,[n':]} \cdot x^3 + \left(b_{jP,[n':]} - b_{jL,[:n']}\right) \cdot x + \left(b_{jR,[n':]} - b_{jP,[:n']}\right) \cdot x^{-1} - b_{jR,[:n']} \cdot x^{-3} = 0$$
$$b_{jR,[:n']} = b_{jL,[n':]} = 0,$$
$$b_{jL,[:n']} = b_{jP,[n':]}, \quad b_{jR,[n':]} = b_{jP,[:n']}$$

Now using these relations we obtain that for every $x \in \{x_1, x_2, x_3, x_4\}$ we have:

$$a' = a_{P,[:n']} \cdot x + a_{P,[n':]} \cdot x^{-1} \quad \text{and} \quad b_j = b_{jP,[:n']} \cdot x^{-1} + b_{jP,[n':]} \cdot x.$$

Now, using these values, we can see that the extracted c_{jL}, c_{jP}, and c_{jR} have the expected form $\forall j \in [1 : k]$:

$$c_{jL} \cdot x^2 + c_{jP} + c_{jR} \cdot x^{-2} = \langle a', b'_j \rangle = \langle a_{P,[:n']} \cdot x + a_{P,[n':]} \cdot x^{-1}, b_{jP,[:n']} \cdot x + b_{jP,[n':]} \cdot x^{-1} \rangle =$$
$$= \langle a_{P,[:n']}, b_{jP,[n':]} \rangle \cdot x^2 + \langle a_{P,[:n']}, b_{jP,[:n']} \rangle + \langle a_{P,[n':]}, b_{jP,[n':]} \rangle + \langle a_{P,[n':]}, b_{jP,[:n']} \rangle \cdot x^{-2}$$
$$= \langle a_{P,[:n']}, b_{jP,[n':]} \rangle \cdot x^2 + \langle a_P, b_{jP} \rangle + \langle a_{P,[n':]}, b_{jP,[:n']} \rangle \cdot x^{-2}.$$

Since this relation holds for all $x \in \{x_1, x_2, x_3, x_4\}$, it must be that:

$$\langle a_P, b_{jP} \rangle = c_{jP} \quad \forall j \in [1; k].$$

Thus, the extractor either extracts a discrete logarithm relation between the generators or the witness. We can see that for each recursive step the extractor uses 4 transcripts and 5 transcripts for the last step $5 \cdot 4^{log_2(n)} = 5n^2$ transcripts in total and thus runs in expected polynomial time.

4 zkFFT: Zero-Knowledge Argument for FFT Relation Check

Given 2 vector commitments $C_{\vec{v}} = u_0^{v_0} \cdot u_1^{v_1} \cdot ... \cdot u_{n-1}^{v_{n-1}}$ and $C_{\vec{a}} = g_0^{a_0} \cdot g_1^{a_1} \cdot ... \cdot g_{n-1}^{a_{n-1}}$, the Verifier wants to check that the first one is the Fast Fourier Transformation of the second one $C_{\vec{v}} = \mathbf{FFT}(C_{\vec{a}})$.
The Prover knows the polynomial $P(x)$ whose 2 different commitments are the $C_{\vec{v}}$ and $C_{\vec{a}}$. The first one commits to the coefficients of the polynomial $P(x)$,while the second one commits to the evaluations of the polynomial at the **n** roots of unity.
Let's observe that the evaluation of the polynomial $P(x)$ at some x is the dot product of it's coefficients and the power-vector of x

$$P(x) = a_0 \cdot x^0 + a_1 \cdot x^1 + ... + a_{n-1} \cdot x^{n-1} = \langle \vec{a}, \vec{\mathbf{x}} \rangle$$

So if v_i is the evaluation of the polynomial $P(x)$ at the **i**-th root of unity,then it should be equal to the dot product of the $\vec{a}$ and power-vector of the ω^i.

$P(w^0) = \langle \vec{a}, w^0 \rangle = \sum_{i=0}^{n-1} a_i \cdot \omega^{0 \cdot i} = \langle \vec{a}, (1, 1, ..., 1) \rangle = \sum_{i=0}^{n-1} a_i = v_0$

$P(w^1) = \langle \vec{a}, w^1 \rangle = \sum_{i=0}^{n-1} a_i \cdot \omega^{i} = \langle \vec{a}, (1, \omega, \ldots, \omega^{n-1}) \rangle = v_1$
$\vdots$

$P(w^{n-1}) = \langle \vec{a}, w^{n-1} \rangle = \sum_{i=0}^{n-1} a_i \cdot \omega^{(n-1) \cdot i} = \langle \vec{a}, (1, \omega^{n-1}, ..., \omega^{(n-1) \cdot (n-1)}) \rangle = v_{n-1}$

Trivial Approach. The Prover generates **n** IPA proofs, which prove that

$$\langle \vec{a}, \mathbf{w}^i \rangle = v_i, \forall i \in [0, 1, ..., n-1]$$

This would have been a great solution if the Verifier had the commitment of each v_i, but the Verifier only has the commitment to the whole vector $C_{\vec{v}} = u_0^{v_0} \cdot u_1^{v_1} \cdot ... \cdot u_{n-1}^{v_{n-1}}$.
Of course, the Prover could send all the commitments $u_i^{v_i}$ to the Verifier, then the Verifier will be able to multiply them and check if the product is equal to the committed $C_{\vec{v}}$

$$\prod_{i=0}^{n-1} u_i^{v_i} = C_{\vec{v}}$$

however this will make the proof size $O(N)$.

Our Approach. Instead The Prover uses our modified IPA on the following relation.

$$P = C_a h_0^{\boldsymbol{\omega^0}} \cdots h_{n-1}^{\boldsymbol{\omega^{n-1}}} C_v h^r = g^a h_0^{\boldsymbol{\omega^0}} \cdots h_{n-1}^{\boldsymbol{\omega^{n-1}}} u_0^{<\mathbf{a}, \mathbf{w}_0>} \cdots u_{n-1}^{<\mathbf{a}, \mathbf{w}_{n-1}>} h^r$$

4.1 zkFFT Protocol

We provide a argument of knowledge for the following relation

$$\mathrm{REL}_{\mathrm{zkFFT}} := \left\{ \left(\begin{array}{r} (\mathrm{g}_0, \ldots, \mathrm{g}_{n-1}) \in \mathbf{G}^n \\ (\mathrm{u}_0, \ldots, \mathrm{u}_{n-1}), \in \mathbf{G}^n \\ \mathrm{f}, \mathrm{C}_{\mathrm{values}}, \mathrm{C}_{\mathrm{coeff}} \in \mathbf{G} \\ (\omega_0, \omega_1, \ldots, \omega_{n-1}) \in \mathbf{F}^n; \\ (\mathrm{c}_0, \mathrm{c}_1, \ldots, \mathrm{c}_{n-1}) \in \mathbf{F}^n, \\ (\mathrm{v}_0, \mathrm{v}_1, \ldots, \mathrm{v}_{n-1}) \in \mathbf{F}^n, \\ r_C, r_V \in F, \end{array} \right) \middle| \begin{array}{r} \mathrm{C}_{\mathrm{coeff}} = \mathrm{g}_0^{\mathrm{c}_0} \mathrm{g}_1^{\mathrm{c}_1} \cdots \mathrm{g}_{n-1}^{\mathrm{c}_{n-1}} \mathrm{f}^{r_c} \\ \mathrm{C}_{\mathrm{values}} = \mathrm{u}_0^{\mathrm{v}_0} \mathrm{u}_1^{\mathrm{v}_1} \cdots \mathrm{u}_{n-1}^{\mathrm{v}_{n-1}} \mathrm{f}^{r_v} \\ v_0 = P(\omega_0) \\ \vdots \\ v_{n-1} = P(\omega_{n-1}) \\ \text{where} \\ P(x) = c_0 + \cdots + c_{n-1} x^{n-1} \in F^n[x] \end{array} \right\}$$

As previously mentioned, we are going to use our modified IPA to prove this relation.

$$P = g^c h_0^{\boldsymbol{\omega^0}} \cdots h_{n-1}^{\boldsymbol{\omega^{n-1}}} u_0^{v_0} \cdots u_{n-1}^{v_{n-1}} h^r = C_a h_0^{\boldsymbol{\omega^0}} \cdots h_{n-1}^{\boldsymbol{\omega^{n-1}}} C_v h^r$$

Recall that during the last round of MIPA, the prover was sending b_i values to the verifier, so it may seem that MIPA is not that efficient as the proof size will be $O(n)$. However, in the case of zkFFT the b_i values are the power-vectors of the roots of unity. Thus, they are public, and there is no need to blind them, and the verifier can independently compute all the values of b_i. This means that the relation $\mathbf{P}$ will be

$$P = g^c u_0^{v_0} \cdots u_{n-1}^{v_{n-1}} h^r = C_a C_v h^r$$

during the proof generation the prover only sends the left and right points in each round, and A, r' and δ' in the final round, reducing the proof size to just $2\log n + 3$ elements.
Of course, since the verifier is computing the b_i values, the verification time increases, but only logarithmically. With $logn$ rounds and $\mathbf{k}$ roots of unity, the verification time is increased by $k \log n$. The security of zkFFT follows directly from the security of MIPA.

$$\mathrm{REL}_{\mathrm{zkFFT}} := \left\{ \begin{pmatrix} \mathbf{g} \in \mathbf{G}^n \\ h, u_1, \ldots, u_k, P \in G; \\ \mathbf{a}, \mathbf{b}_1, \ldots, \mathbf{b}_k \in Z_p^n, \\ \alpha \in Z_p, \end{pmatrix} \;\middle|\; P = \mathbf{g}^{\mathbf{a}} u_1^{<\mathbf{a},\mathbf{b}_1>} \cdots u_k^{<\mathbf{a},\mathbf{b}_k>} h^{\alpha} \right\}$$

If $n = 1$:

P: $r, \delta \xleftarrow{\$} \mathbb{Z}_p$ and computes
$A = g^r u_1^{<r,b_1>} \cdots u_k^{<r,b_k>} h^{\delta} \in \mathbb{G}$

P $\rightarrow$ V: A

V: $x \xleftarrow{\$} \mathbb{Z}_p$

P $\leftarrow$ V: x

P computes
$r' = r + a \cdot x \in \mathbb{Z}_p$
$\delta' = \delta + \alpha \cdot x \in \mathbb{Z}_p$

P $\rightarrow$ V: r', δ'

V: outputs *Accept* if the following equality holds
$A \cdot P^x = g^{r'} u_1^{<r',b_1>} \cdots u_k^{<r',b_k>} h^{\delta'} \in \mathbb{G}$

Else ($n > 1$):

P: $d_L, d_R \xleftarrow{\$} \mathbb{Z}_p$ and computes
Let $n' = \frac{n}{2}$
$c_{Li} = \langle a_{[:n']}, b_{i[n':]} \rangle, c_{Ri} = \langle a_{[n':]}, b_{i[:n']} \rangle \in \mathbf{Z_p} \quad \forall i \in [1, k]$

$L = g_{[n':]}^{a_{[:n']}} u_1^{c_{L1}} \cdots u_k^{c_{Lk}} h^{d_L} \in \mathbf{G}$
$R = g_{[:n']}^{a_{[n':]}} u_1^{c_{R1}} \cdots u_k^{c_{Rk}} h^{d_R} \in \mathbf{G}$

P $\rightarrow$ V: L, R

V: $x \xleftarrow{\$} \mathbb{Z}_p$

P $\leftarrow$ V: x

P and V: compute
$g' = g_{[:n']}^{x^{-1}} \circ g_{[n':]}^{x} \in \mathbf{G^{n'}}$
$b'_i = x^{-1} b_{i[:n']} + x b_{i[n':]} \in \mathbb{Z}_p^{n'} \quad \forall i \in [1, k]$
$P' = L^{x^2} P R^{x^{-2}} \in \mathbf{G}$

P: computes

1. $a' = x a_{[:n']} + x^{-1} a_{[n':]} \in \mathbb{Z}_p^{n'}$
2. $\alpha' = d_L x^2 + \alpha + d_R x^{-2}$

P and V: recursively run the protocol on $(g' u_1, \cdots, u_k, h, P'; a', b'_1, \cdots, b'_k, \alpha')$

Figure 2: Zero Knowledge Argument for zkFFT relation

4.2 Aggregating zkFFT Proofs

In many real-world applications, the prover often needs to handle multiple pre-committed vectors. Creating zkFFT proofs for each vector separately would be inefficient, particularly

because these vectors are committed using the same bases. While the naive approach involves performing multi-scalar multiplications independently for each vector, the aggregated approach combines all the shared bases' multiplications into one.

$$
\begin{aligned}
P_1 &= g^{a_1} u_1^{\langle a_1, b_1 \rangle} \cdots u_k^{\langle a_1, b_k \rangle} h^{\alpha_1} \\
P_2 &= g^{a_2} u_1^{\langle a_2, b_1 \rangle} \cdots u_k^{\langle a_2, b_k \rangle} h^{\alpha_2} \\
&\vdots \\
P_m &= g^{a_m} u_1^{\langle a_m, b_1 \rangle} \cdots u_k^{\langle a_m, b_k \rangle} h^{\alpha_m}
\end{aligned}
$$

After obtaining the $\mathbf{m}$ pre-committed vectors,the Verifier samples a random challenge $\mathbf{s}$ and sends it to the Prover. The Prover then uses this challenge to combine these vectors.

$$
\begin{aligned}
\mathbf{P} = \prod_{j=1}^{m} P_j^{s^j} &= \prod_{j=1}^{m} g^{s^j a_j} u_1^{\langle s^j a_j, b_1 \rangle} \cdots u_k^{\langle s^j a_j, b_k \rangle} h^{s^j \alpha_j} \\
&= g^{\sum_{j=1}^{m} a_j s^j} u_1^{\langle \sum_{j=1}^{m} a_j s^j, b_1 \rangle} \cdots u_k^{\langle \sum_{j=1}^{m} a_j s^j, b_k \rangle} h^{\sum_{j=1}^{m} \alpha_j s^j}
\end{aligned}
$$

If we define the sum $\sum_{j=1}^{m} a_j s^j$ as a, where a represents the coefficients a_j scaled by s^j, and similarly $\sum_{j=1}^{m} \alpha_j s^j$ as α, Then $\mathbf{P}$ becomes:

$$
\mathbf{P} = g^{a} u_1^{\langle a, b_1 \rangle} \cdots u_k^{\langle a, b_k \rangle} h^{\alpha}
$$

Now we can see that the Aggregated-zkFFT boils down to the zkFFT protocol with $P_1, P_2, \cdots, P_k$ added to the proof.

4.3 Batch Verification of zkFFT proofs

Having discussed the scenario where the prover aggregates multiple pre-committed vectors into a single zkFFT proof, let's now focus on the case where the verifier receives multiple proofs and wants to efficiently batch-verify them, rather than verifying each one individually.

Given $\mathbf{m}$ independently generated proofs, each consisting of

$$
\begin{aligned}
&\{P_1, L_{1,1}, R_{1,1}, \cdots, L_{1,\log n}, R_{1,\log n}, A_1, r'_1, \delta'_1\} \\
&\{P_2, L_{2,1}, R_{2,1}, \cdots, L_{2,\log n}, R_{2,\log n}, A_2, r'_2, \delta'_2\} \\
&\vdots \\
&\{P_m, L_{m,1}, R_{m,1}, \cdots, L_{m,\log n}, R_{m,\log n}, A_m, r'_m, \delta'_m\}
\end{aligned}
$$

the verifier samples a random challenge $\mathbf{s}$ and replaces the final verification check on each proof, with a new aggregated check that verifies all the proofs simultaneously.

$$
\prod_{i=1}^{m} \left(A_i^{s^i} \cdot P_i^{s^i x_i} \right) = g_i^{\sum_{i=1}^{m} s^i r'_i} u_1^{<\sum_{i=1}^{m} s^i r'_i, b_1>} \cdots u_k^{<\sum_{i=1}^{m} s^i r'_i, b_k>} h^{\sum_{i=1}^{m} s^i \delta'_i} \in \mathbb{G}
$$

This will drastically reduce the number of exponentiations preformed by the Verifier.

5 Supporting Vector Commitment Inputs in Halo2

The arithmetization used by Halo2 is derived from UltraPLONK [12], which is an extension of the PLONK system [13]. Halo2[16] supports custom gates and lookup arguments, and the arithmetic circuit is represented through a rectangular matrix where the cell values (for

a given statement and witness) are elements of a fixed finite field $\mathbb{F}$. The circuit depends on a configuration consisting of Fixed Columns, Advice Columns and Instant Columns. Fixed Columns will encode values predetermined by the circuit,while the Instance Columns are usually used for public inputs and for any shared elements between the prover and verifier. The Advice Columns will encode the prover's witness data. We will not describe the full Halo2 proof system here but for a general context let's summarize that the proof system works through the following logical steps:

1. Create the arithmetic circuit comprised of Fixed, Advance and Instance Columns which will encode both the public and witness data.
2. Encode the circuit-satisfiability logic through polynomials.
3. Commit to the circuit data and logic through polynomial commitment.
4. Open the polynomial commitment at random points to ensure circuit satisfiability.

During the circuit commitment phase, each column's data is represented through a specific polynomial, which coefficients are computed using discrete numerical transformation (FFT over finite fields) over the column values and a fixed domain of evaluation points. The polynomial is next committed using a polynomial commitment scheme. We extend the Halo2 to additionally prove that these polynomials are the Fourier transforms of pre-committed vectors. The generic relation of the Halo2 proof system is given as follows:

Let $\omega \in \mathbb{F}$ be a primitive $n = 2^k$ root of unity, forming the domain $D = (\omega^0, \omega^1, \ldots, \omega^{n-1})$, with $t(X) = X^n - 1$ as the vanishing polynomial over this domain.n_g, n_e,n_a, and p are positive integers, where n_a is the number of unique commitments from which p is the number of pre-committed vectors, n_e is the maximum number of points in any query set, and n_g is the maximum degree of the constraint system. Additionally ensure $n_e, n_a < n$ and $n_g > 4$.

$$R = \left\{ \begin{array}{l} \left(g\left(X, C_0, \ldots, C_{n_a-1}, a_0(X), \ldots, a_{n_a-1}\left(X, C_0, \ldots, C_{n_a-1}, a_0(X), \ldots, a_{n_a-2}(X)\right)\right)\right), \\ \left(a_0(X), a_1\left(X, C_0, a_0(X)\right), \ldots, a_{n_a-1}\left(X, C_0, \ldots, C_{n_a-1}, a_0(X), \ldots, a_{n_a-2}(X)\right)\right), \\ \boldsymbol{a_{l+1}\left(w_1, \ldots, w_n\right), \ldots, a_{l+p}\left(v_1, \ldots, v_n\right)}, g\left(\omega^i, \ldots\right) = 0 \quad \forall i \in [0, n) \end{array} \right\}$$

Here, g represents the complete PLONKish constraint system that enforces all circuit relations to zero. This includes standard and custom gates, rules for lookup argument, equality constraint permutation and commitment arguments.The polynomials a_i, for all $i \in [0, n) \setminus \{l+1, \ldots, l+p\}$, embody the witness values. Meanwhile, the formal variables C_i represent the verifier's challenges.

The arguments of each a_i, specifically for $i \in \{l+1, \ldots, l+p\}$, are vectors precommitted in advance.This aspect utilizes MIPA to extend Halo2, enabling support for precommitted vectors. The resulting commitment is expressed as:

$$C_{\overline{v}} = \text{Commit}(v_1, \ldots, v_n) = \prod_{j=1}^{n} {g_j}^{v_j}.$$

The proving system is divided into five phases, plus our extension.

1. Commit to polynomials to encode the circuit.
2. Prove that these polynomials are FFTs of the pre-committed values.
3. Construct the vanishing argument to enforce that all relations equal zero.
4. Evaluate the given polynomials at all required points.

5. Create the multi-point opening argument to ensure that all evaluations align with their respective commitments.

6. Run the inner product argument to provide a polynomial commitment opening proof for the multi-point opening argument polynomial.

The provided relation is generic enough to support any type of input, including vector commitment inputs, and prove the circuit satisfiability. We augmented the existing scheme to support zkFFT proofs with vector commitments inputs in a more efficient way.

6 Curve Tree Relation

In this section, we show how zkFFT can be used in membership proofs. Using Halo2 extended with zkFFT instead of multi-scalar multiplication (MSM), we can improve the efficiency of the Curve Tree structure.

The Curve Tree[7] provides efficient means for proving private set membership in a fully transparent setup. Its architecture is modeled after a shallow Merkle tree, where both the leaves and the internal nodes are points on an elliptic curve.

The hashing mechanism in a Curve Tree involves a specifically instantiated Pedersen hash, alternating curves at each layer to maintain a 2-cycle of curves, thereby enhancing the security of data commitments. Zero-knowledge proofs of membership within Curve Trees are facilitated using commit-and-prove capabilities integrated with Bulletproofs [6], leveraging the algebraic properties of the data structure.

The functionality of the Curve Tree is encapsulated in the mathematical relation below, detailing the conditions for set membership:

$$\mathbf{R}^{\text{CURVE TREE}} := \left\{ (i, r, \delta, \vec{x}, y) : \begin{array}{l} C = \left\langle [\vec{x}], \vec{G}^{x}_{(-)} \right\rangle + [r] \cdot H_{(-)} \\ \wedge\, (x_i, y) \in P_{\text{other}(-)} \\ \wedge\, \hat{C} = (x_i, y) + [\delta] \cdot H_{\text{other}(-)} \end{array} \right\}$$

where field scalars are denoted by [ ], and vectors by $\vec{}$. The inner product is represented by $\left\langle \vec{x}, \vec{G}^{x}_{(-)} \right\rangle$, and elliptic curve points by $P_{(-)}$ and $P_{\text{other}(-)}$. Random scalars δ and r are included as $[r] \cdot H_{(-)}$ and $[\delta] \cdot H_{\text{other}(-)}$.

To implement scalar multiplication, the secret scalar is decomposed into 3-bit windows, and tables T are defined accordingly.

For $i \in 1, \ldots, m-1$,table T_i is defined as:

$$T_i = \left\{ \left[j \cdot 2^{3(i-1)} + 2^{3i} \right] \cdot H \mid j \in 0, \ldots, 2^3 - 1 \right\}$$

The definition of T_m is given by:

$$T_m = \left\{ \left[j \cdot 2^{3(m-1)} - \sum_{i=1}^{m-1} 2^{3i} \right] \cdot H \mid j \in 0, \ldots, 2^3 - 1 \right\}$$

To enforce $(\tilde{x}, \tilde{y}) = [r] \cdot H + (x, y)$, it is expressed as:

$$\begin{aligned} (\tilde{x}, \tilde{y}) &= \text{Rerand}(x, y) \\ &\equiv \{(\tilde{x}, \tilde{y}) = (x, y) + T_m + (T_{m-1} \oplus T_{m-2} \oplus \ldots \oplus T_1)\} \end{aligned}$$

The proposed approach for scalar multiplication necessitates a considerable amount of pre-computed tables to achieve efficiency. Specifically, for a single 256-bit scalar multiplication $[x] \cdot H$, the method requires $\left\lfloor \frac{256}{3} \right\rfloor + 1 = 86$ pre-computed tables. When this method is extended to handle n scalar multiplications, the pre-computed tables demand scales linearly.

Mathematically, if we consider n scalars $k_1, k_2, \ldots, k_n$, each decomposed into 3-bit windows, the scalar multiplication can be expressed as:

$$[k_j] \cdot H = \sum_{i=1}^{m_1} k_{j,i} \cdot 2^{3(i-1)} \cdot H \quad \forall j \in [1; n]$$

where $k_{j,i}$ represents the 3-bit window for the i-th position of the j-th scalar.

The total number of pre-computed tables required for n scalar multiplications is $n \cdot 86$, where each scalar is 256 bits. The definition of table T_i (for $i = 1, \ldots, m-1$) and T_m is described earlier in this section.

For multiple scalar multiplications, the final points $(\tilde{x}_j, \tilde{y}_j)$ for each j-th scalar k_j are computed as:

$$(\tilde{x}_j, \tilde{y}_j) = (x_j, y_j) + T_{m_j} + (T_{m_j-1} \oplus T_{m_j-2} \oplus \ldots \oplus T_j) \quad \forall j \in [1; n]$$

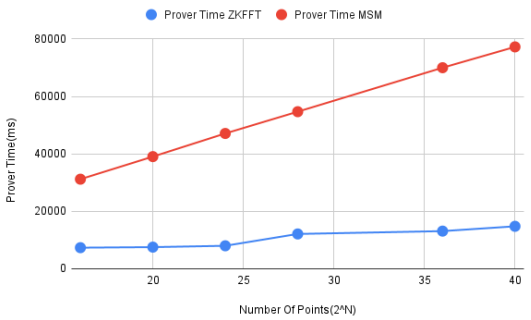
The proposed method of scalar multiplication using 3-bit windows and precomputed tables can be enhanced by replacing it with advice columns in Halo2 to store pre-committed vectors, making the use of zkFFT more efficient than MSM.

7 Implementation and Performance

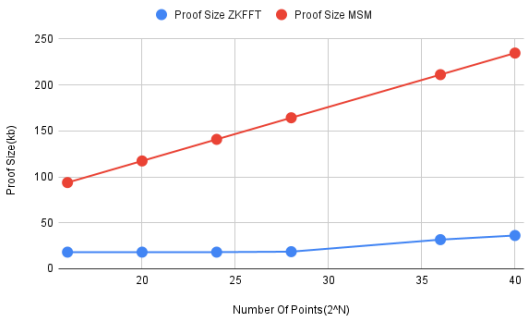
Below are the benchmark results demonstrating the performance of zkFFT compared to MSM for the Curve Tree relation. Since the zkFFT Verifier must compute all the b_i values, it is in general slower than the MSM Verifier. However, this is the only downside of zkFFT, and the verification speed can be improved by the aggregation or batching techniques discussed in the paper.

As for prover time and proof size, our benchmarks indicate that zkFFT's prover is approximately five times faster, and the proof sizes are about 5.5-6 times smaller. All benchmarks were performed using the MacBook Pro M2. For more detailed results, refer to Appendix

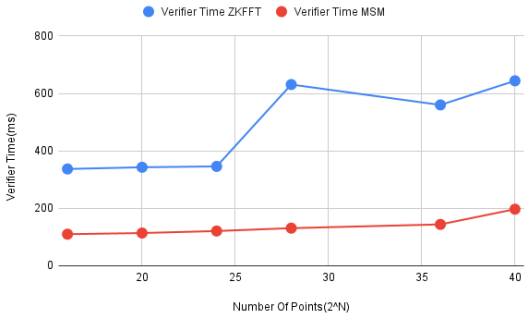
Code. The Implementations are open source zkFFT library, Halo2 with vector commitments library.



(a) Proving Time



(b) Proof Size



(c) Verification Time

8 References

[1] Generalized bulletproof. https://hackmd.io/6g5oC5xWRLOoYcTnYBuE7Q?view.

[2] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Scalable, transparent, and post-quantum secure computational integrity. Cryptology ePrint Archive, Paper 2018/046, 2018.

[3] Jonathan Bootle, Andrea Cerulli, Pyrros Chaidos, Jens Groth, and Christophe Petit. Efficient zero-knowledge arguments for arithmetic circuits in the discrete log setting. Cryptology ePrint Archive, Paper 2016/263, 2016.

[4] Sean Bowe, Jack Grigg, and Daira Hopwood. Recursive proof composition without a trusted setup. Cryptology ePrint Archive, Paper 2019/1021, 2019.

[5] Vitalik Buterin. Quadratic arithmetic programs. https://medium.com/@VitalikButerin/quadratic-arithmetic-programs-from-zero-to-hero-f6d558cea649, 2016.

[6] Benedikt Bünz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, and Greg Maxwell. Bulletproofs: Short proofs for confidential transactions and more. Cryptology ePrint Archive, Paper 2017/1066, 2017. https://eprint.iacr.org/2017/1066.

[7] Matteo Campanelli, Mathias Hall-Andersen, and Simon Holmgaard Kamp. Curve trees: Practical and transparent zero-knowledge accumulators. Cryptology ePrint Archive, Paper 2022/756, 2022. https://eprint.iacr.org/2022/756.

[8] Heewon Chung, Kyoohyung Han, Chanyang Ju, Myungsun Kim, and Jae Hong Seo. Bulletproofs+: Shorter proofs for privacy-enhanced distributed ledger. Cryptology ePrint Archive, Paper 2020/735, 2020.

[9] James W. Cooley and John W. Tukey. An algorithm for the machine calculation of complex fourier series. https://www.ams.org/journals/mcom/1965-19-090/S0025-5718-1965-0178586-1/S0025-5718-1965-0178586-1.pdf, 1965.

[10] Liam Eagen, Sanket Kanjalkar, Tim Ruffing, and Jonas Nick. Bulletproofs++: Next generation confidential transactions via reciprocal set membership arguments. Cryptology ePrint Archive, Paper 2022/510, 2022.

[11] Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems, 1986.

[12] Ariel Gabizon. From airs to raps - how plonk-style arithmetization works. Blog post on https://hackmd.io/@aztec-network/plonk-arithmetiization-air, 2024.

[13] Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. Plonk: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge. Cryptology ePrint Archive, Paper 2019/953, 2019. https://eprint.iacr.org/2019/953.

[14] Jens Groth. On the size of pairing-based non-interactive arguments. Cryptology ePrint Archive, Paper 2016/260, 2016. https://eprint.iacr.org/2016/260.

[15] Torben P. Pedersen. Non-interactive and information-theoretic secure verifiable secret sharing, 1991.

[16] Zcash. Halo2 book. https://zcash.github.io/halo2/index.html.

A Detailed benchmarks

Here are detailed benchmarks of zkFFT, including the improvements achieved with the aggregation and batching techniques.

n	prover time (ms)	proof size (kb)	verifier time (ms)	aggregated prover time (ms)	aggregated proof size (kb)	aggregated verifier time (ms)	batch verifier time (ms)
2^6	23.895	0.46875	4.507	25.176	3.59375	19.736	174
2^7	44.142	0.53125	7.9774	49.880	3.65625	22.090	339
2^8	87.585	0.59375	14.744	106.80	3.71875	28.588	687
2^9	186.07	0.65625	33.645	263.34	3.78125	47.97	1368
2^{10}	434.05	0.71875	94.776	753.11	3.84375	109.1	3253
2^{11}	1115.8	0.78125	295.28	5101.0	3.90625	316.13	8101

Table 1: Performance Metrics for zkFFT (Batch size = 100)

In the tables below you can find in-depth comparison of zkFFT and MSM on Curve Tree relation

- **n** is the number of elements in the tree
- **depth** is the depth of the tree
- 2^k is the number of elements in each node.

n	**(k, Depth)**	**Keygen Time (ms)**		**Prover Time (ms)**		**Proof Size (kb)**		**Verifier Time (ms)**	
		zkFFT	**MSM**	**zkFFT**	**MSM**	**zkFFT**	**MSM**	**zkFFT**	**MSM**
2^{16}	(4, 4)	3748	52586	7344	31188	18.125	93.875	337	110
2^{20}	(4, 5)	3794	52555	9345	39050	22.65625	117.3437	426	114
2^{24}	(4, 6)	3770	52365	10944	47112	27.1875	140.8125	523	121
2^{28}	(4, 7)	3755	53273	12803	54691	31.71875	164.28125	535	131
2^{32}	(4, 8)	3678	53024	14528	62136	36.25	187.75	915	173
2^{36}	(4, 9)	3709	52567	16614	70002	40.78125	211.21875	676	144
2^{40}	(4, 10)	3717	52481	18410	77250	45.3125	234.6875	799	197

Table 2: Performance Metrics for MSM and zkFFT (k = 4)

n	(k, Depth)	Keygen Time (ms)		Prover Time (ms)		Proof Size (kb)		Verifier Time (ms)	
		zkFFT	MSM	zkFFT	MSM	zkFFT	MSM	zkFFT	MSM
2^{15}	(5, 3)	3661	100177	5481	44583	13.59375	128.71875	272	172
2^{20}	(5, 4)	3707	101178	7536	56464	18.125	171.625	343	174
2^{25}	(5, 5)	3670	100849	9500	70885	22.65625	214.53125	414	280
2^{30}	(5, 6)	3699	101454	11010	84810	27.1875	257.4375	464	185
2^{35}	(5, 7)	3755	100103	13111	99197	31.71875	300.34375	559	207
2^{40}	(5, 8)	3768	100866	14784	113280	36.25	343.25	644	212

Table 3: Performance Metrics for MSM and zkFFT (k = 5)

n	(k, Depth)	Keygen Time (ms)		Prover Time (ms)		Proof Size (kb)		Verifier Time (ms)	
		zkFFT	MSM	zkFFT	MSM	zkFFT	MSM	zkFFT	MSM
2^{18}	(6, 3)	3773	206042	5799	105879	13.59375	245.15625	285	349
2^{24}	(6, 4)	3687	208391	7544	141408	18.125	326.875	336	348
2^{30}	(6, 5)	3717	208139	9450	176345	22.65625	408.59375	405	420
2^{36}	(6, 6)	3718	207579	11370	212208	27.1875	490.3125	471	603

Table 4: Performance Metrics for MSM and zkFFT (k = 6)

n	(k, Depth)	Keygen Time (ms)		Prover Time (ms)		Proof Size (kb)		Verifier Time (ms)	
		zkFFT	MSM	zkFFT	MSM	zkFFT	MSM	zkFFT	MSM
2^{21}	(7, 3)	4886	410746	8976	206982	13.96875	477.65625	515	783
2^{28}	(7, 4)	4818	411693	12120	277332	18.625	636.875	631	866
2^{35}	(7, 5)	4879	412268	15200	349405	23.28125	796.09375	761	815

Table 5: Performance Metrics for MSM and zkFFT (k = 7)

n	(k, Depth)	Keygen Time (ms)		Prover Time (ms)		Proof Size (kb)		Verifier Time (ms)	
		zkFFT	MSM	zkFFT	MSM	zkFFT	MSM	zkFFT	MSM
2^{16}	(8, 2)	8142	937983	10878	408988	9.5625	628.8125	821	2186
2^{24}	(8, 3)	8115	939068	16305	616857	14.34375	943.21875	1051	2219
2^{32}	(8, 4)	8265	935709	21840	822164	19.125	1257.625	1282	2217
2^{40}	(8, 5)	8205	936802	27280	1027705	23.90625	1572.03125	1804	2256

Table 6: Performance Metrics for MSM and zkFFT (k = 8)